

Lecture Notes
Mathematical Modeling

Prof. Boris Houska

Contents

1	Probability and Statistics Models	3
2	Multivariate Analysis Models	136
3	Linear Programming Models	203
4	Nonlinear Programming Models	291
5	Graph and Network Models	369
6	Ordinary Differential Equation Models	454

Chapter 1

Probability and Statistics Models

Probability and Statistics Models

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Why Probability and Statistics?

Many groundbreaking technologies rely on probabilistic thinking:

- **Artificial Intelligence:** Large language models (like DeepSeek) predict the next word based on probabilities learned from vast amounts of text.
- **Medicine:** Determining whether a new vaccine is effective requires statistical hypothesis testing on clinical trial data.
- **Self-driving cars:** Sensors are noisy – the car must constantly estimate its position and the probability of obstacles.
- **Recommendation systems:** Netflix and TikTok use probabilistic models to predict what you'll like next.

Why Probability and Statistics?

Probability and statistics give us the language to:

- **Summarize** complex data (descriptive statistics)
- **Draw conclusions** under uncertainty (inference)
- **Make predictions** that power modern technology

What is a Mathematical Model?

- A **mathematical model** is a description of a real-world phenomenon using mathematical language.
- It typically consists of:
 - Variables (inputs, outputs)
 - Equations or rules linking them
 - Parameters (constants to be estimated)
- Models help us understand, predict, and control systems.
- In this chapter we focus on **probability and statistics models**: models that involve randomness or data.

Contents

- Introduction
- **Descriptive Statistics – First Look**
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Descriptive Statistics

- When we have a set of data, we want to describe its main features.
- Two key aspects:
 1. **Central tendency**: where the data are centered.
 2. **Dispersion**: how spread out the data are.
- We'll look at common measures: mean, variance, standard deviation
 - first with intuition, then with formal definitions later.

Populations and Samples

- A **population** is the entire group we are interested in.
Example: all students in a university.
- A **sample** is a subset of the population (say, 5 students).
- We denote:
 - Population size: N (often very large or infinite)
 - Sample size: n (small, finite)
 - Population mean: μ (usually unknown)
 - Sample mean: $\bar{x}$ (computed from data)
- The goal of statistical inference is to use sample statistics to estimate population parameters.

Sample Mean (Average)

- For a sample $x_1, x_2, \dots, x_n$, the **sample mean** is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

- It estimates the population mean μ (if the sample is representative).
- Example: # of cups of coffee consumed yesterday by 5 students:

0, 1, 2, 3, 9.

The mean is

$$\bar{x} = \frac{0 + 1 + 2 + 3 + 9}{5} = \frac{15}{5} = 3.$$

- The one student who drank 9 cups (outlier) pulls the mean upward.

Variance and Standard Deviation – First Look

- Variance measures the average squared deviation from the mean.
- For a set of numbers, one natural way to approximate the variance of *the sample itself* is:

$$\text{Sample variance (descriptive)} \approx \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

- The standard deviation is the square root of the variance, giving spread in the original units.
- However, if we want to use a sample to estimate the *population* variance, the above formula tends to underestimate the true spread. We will see why and how to correct it (using $n - 1$ instead of n).

Variance and Standard Deviation – First Look

- For the coffee data 0, 1, 2, 3, 9 with $\bar{x} = 3$:

$$\frac{1}{5} \sum (x_i - \bar{x})^2 = \frac{9 + 4 + 1 + 0 + 36}{5} = 10,$$

so the approximate standard deviation is $\approx \sqrt{10} \approx 3.16$ cups.

- Interpretation: On average, observations deviate from the mean by about 3.16 cups (we'll later correct this to $\sqrt{12.5} \approx 3.54$ cups).
- Also note that most students are near the mean, but the 9-cup student causes the spread to be larger.

Thinking About Samples...

We have asked 5 students: "How many coffees did you drink yesterday?"

Answers: 0, 1, 2, 3, 9.

Question 1: Is 3 your best guess for the true average of all students in this university? How confident are you?

Thinking About Samples...

We have asked 5 students: "How many coffees did you drink yesterday?"

Answers: 0, 1, 2, 3, 9.

Question 2: If you meet a random student tomorrow, would you predict they drank exactly 3? If not, what would you predict?

Thinking About Samples...

We have asked 5 students: "How many coffees did you drink yesterday?"

Answers: 0, 1, 2, 3, 9.

Question 3: Suppose we had instead sampled 2, 2, 3, 3, 4 (same mean).

Does this change your answers? Why?

Thinking About Samples...

We have asked 5 students: "How many coffees did you drink yesterday?"

Answers: 0, 1, 2, 3, 9.

Question 4: Could the true university average be 2.5? Could it be 5?

How would you even know?

Thinking About Samples...

We have asked 5 students: "How many coffees did you drink yesterday?"

Answers: 0, 1, 2, 3, 9.

Question 5: That student who drank 9 — outlier or valuable information?

Thinking About Samples...

The above questions cut to the heart of statistics:

- Estimating parameters (Question 1)
- Predicting individual outcomes (Question 2)
- Quantifying uncertainty (Questions 3–4)
- Handling extreme observations (Question 5)

Contents

- Introduction
- Descriptive Statistics – First Look
- **Probability and Random Variables**
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Random Experiments and Events

- A random experiment is a process with uncertain outcome.
- Example: Tossing a fair coin.
- The set of all possible outcomes is the **sample space** Ω .
 - For a coin toss: $\Omega = \{\text{Heads, Tails}\}$ (often written $\{H, T\}$ or $\{0, 1\}$).
- An **event** is any subset of the sample space Ω .
 - Example: "Getting Heads" is the event $\{H\}$.
 - Example: "Getting a number greater than 4" when rolling a die is the event $\{5, 6\}$.

Probability of Events

- **Probability** $P(A)$ measures how likely event A is to occur.
 - We have $0 \leq P(A) \leq 1$.
 - $P(\Omega) = 1$ (something in the sample space must happen).
 - If A and B are mutually exclusive (cannot happen at the same time), then

$$P(A \cup B) = P(A) + P(B).$$

Here, $\cup$ denotes the union: $A \cup B$ is the event that A **or** B occurs.

Random Variables

- A **random variable** X is a numerical outcome of a random experiment. There are two types:
- **Discrete**: countable values (Example: number of heads in 3 tosses).
- **Continuous**: takes any value in an interval (Example: height, time).

Random Variables

- Probability distribution describes how probabilities are assigned to the values of a random variable.
- Discrete: probability mass function (pmf) $p(x) = P(X = x)$.
- Continuous: probability density function (pdf) $f(x)$ such that

$$P(a \leq X \leq b) = \int_a^b \rho(x) \, dx$$

Expectation (Mean) of a Random Variable

- The **expected value** (mean) of a random variable X is a weighted average of its possible values.
- For discrete X with pmf $p(x)$:

$$\mathbb{E}[X] = \sum_x x p(x).$$

- For continuous X with pdf $f(x)$:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x) dx.$$

- Interpretation: long-run average if experiment repeated many times.

Linearity of Expectation

- For any constants a, b and random variables X, Y ,

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

- This holds even if X and Y are dependent – a very powerful property!
- Example: If $X_1, \dots, X_n$ are independent with the same mean μ , then

$$\mathbb{E}\left[\frac{1}{n} \sum X_i\right] = \frac{1}{n} \sum \mathbb{E}[X_i] = \mu.$$

Example: Discrete Random Variable

- Toss a fair coin twice. Let X = number of heads.
- Possible outcomes: HH (2), HT (1), TH (1), TT (0).
- pmf: $p(0) = 1/4$, $p(1) = 1/2$, $p(2) = 1/4$.
- Expectation:

$$\mathbb{E}[X] = 0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 0 + 0.5 + 0.5 = 1.$$

- On average, we get 1 head in two tosses.

Variance of a Random Variable

- The **variance** measures spread around the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

- Standard deviation: $\sigma_X = \sqrt{\text{Var}(X)}$.
- An equivalent formula (often easier to compute) is

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

- Derivation:

$$\begin{aligned}\text{Var}(X) &= \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2 - 2\mu X + \mu^2] \\ &= \mathbb{E}[X^2] - 2\mu\mathbb{E}[X] + \mu^2 = \mathbb{E}[X^2] - \mu^2.\end{aligned}$$

Properties of Variance

- For independent random variables X and Y ,

$$\text{Var}(aX + bY) = a^2 \text{Var}(X) + b^2 \text{Var}(Y).$$

- If they are dependent, there is an extra covariance term.
- Example: If $X_1, \dots, X_n$ are i.i.d. with variance σ^2 , then

$$\text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}.$$

Computing the Variance

- For a discrete random variable with pmf $p(x)$:

$$\mathbb{E}[X^2] = \sum_x x^2 p(x), \quad \mu = \sum_x x p(x).$$

- For a continuous random variable with pdf $f(x)$:

$$\mathbb{E}[X^2] = \int_{-\infty}^{\infty} x^2 f(x) dx, \quad \mu = \int_{-\infty}^{\infty} x f(x) dx.$$

Computing the Variance

- **Example 1:** Coin toss (two coins). X = number of heads.

$$p(0) = \frac{1}{4}, p(1) = \frac{1}{2}, p(2) = \frac{1}{4}.$$

$$\mathbb{E}[X] = 0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 1,$$

$$\mathbb{E}[X^2] = 0^2 \cdot \frac{1}{4} + 1^2 \cdot \frac{1}{2} + 2^2 \cdot \frac{1}{4} = 0 + 0.5 + 1 = 1.5,$$

$$\text{Var}(X) = 1.5 - 1^2 = 0.5, \quad \sigma_X = \sqrt{0.5} \approx 0.707.$$

Computing the Variance

- **Example 2:** $X \sim \text{Uniform}(0, 1)$ with pdf $f(x) = 1$ on $[0, 1]$.

$$\mu = \int_0^1 x \, dx = \frac{1}{2},$$

$$\mathbb{E}[X^2] = \int_0^1 x^2 \, dx = \frac{1}{3},$$

$$\text{Var}(X) = \frac{1}{3} - \left(\frac{1}{2}\right)^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12} \approx 0.0833.$$

Binomial Distribution: Definition

- The Binomial distribution models the number of successes in a fixed number of independent trials.
- **Setting:**
 - We perform n independent trials.
 - Each trial has two outcomes: success (S) or failure (F).
 - Probability of success in a single trial is p (constant for all trials).
- Let X = total number of successes in n trials.
- We write $X \sim \text{Bin}(n, p)$.
- **Example:** Toss a fair coin 10 times. Let X = number of heads. Then $n = 10$, $p = 0.5$.

Binomial Distribution: Probability Mass Function

- What is $P(X = k)$ for $k = 0, 1, \dots, n$?
- We need exactly k successes and $n - k$ failures.
- Probability of a specific sequence with k successes: $p^k(1 - p)^{n-k}$.
- How many such sequences? Choose which k trials are successes: $\binom{n}{k}$.
- Therefore,

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n.$$

- Also recall the binomial formula (you know it from school!):

$$\sum_{k=0}^n P(X = k) = (p + (1 - p))^n = 1^n = 1.$$

Binomial Distribution: Expectation

- We want $\mathbb{E}[X]$.
- **Trick:** Write X as a sum of simpler random variables.
- Let $X_i =$ indicator of success on trial i :

$$X_i = \begin{cases} 1 & \text{if trial } i \text{ is a success,} \\ 0 & \text{otherwise.} \end{cases}$$

- Then $X = X_1 + X_2 + \cdots + X_n$.
- Each X_i is a Bernoulli random variable; that is,

$$P(X_i = 1) = p \quad \text{and} \quad P(X_i = 0) = 1 - p.$$

- $\mathbb{E}[X_i] = 1 \cdot p + 0 \cdot (1 - p) = p$.
- By linearity: $\mathbb{E}[X] = \mathbb{E}[X_1] + \mathbb{E}[X_2] + \cdots + \mathbb{E}[X_n] = np$.

Binomial Distribution: Variance

- First, find $\text{Var}(X_i)$ for a Bernoulli trial.
- $\mathbb{E}[X_i^2] = 1^2 \cdot p + 0^2 \cdot (1 - p) = p.$
- $\text{Var}(X_i) = \mathbb{E}[X_i^2] - (\mathbb{E}[X_i])^2 = p - p^2 = p(1 - p).$
- Since the trials are independent, the X_i are independent.
- Variance of a sum of independent variables is the sum of variances:

$$\text{Var}(X) = \text{Var}(X_1) + \text{Var}(X_2) + \cdots + \text{Var}(X_n) = np(1 - p).$$

- **Summary for Binomial:**

$$\mathbb{E}[X] = np \quad \text{and} \quad \text{Var}(X) = np(1 - p).$$

Poisson Distribution: Motivation

- The Poisson distribution models the number of rare events occurring in a fixed interval of time or space.
- **Examples:**
 - Number of emails received in an hour.
 - Number of customers arriving at a store in a day.
 - Number of typos on this slide.
- It arises as a limit of the Binomial distribution when n is large, p is small, and $np = \lambda$ is constant.
- Let $\lambda > 0$ be the average rate (mean number of events per interval).
- We write $X \sim \text{Poisson}(\lambda)$.

Poisson Distribution: Probability Mass Function

- The pmf of a Poisson random variable X is:

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

- Derivation sketch (from Binomial limit):

$$\begin{aligned} P(X = k) &= \lim_{n \rightarrow \infty} \binom{n}{k} \left(\frac{\lambda}{n}\right)^k \left(1 - \frac{\lambda}{n}\right)^{n-k} \\ &= \frac{\lambda^k}{k!} \lim_{n \rightarrow \infty} \frac{n!}{(n-k)! n^k} \left(1 - \frac{\lambda}{n}\right)^{n-k} \\ &= \frac{e^{-\lambda} \lambda^k}{k!}. \end{aligned}$$

- Also check: $\sum_{k=0}^{\infty} \frac{e^{-\lambda} \lambda^k}{k!} = e^{-\lambda} e^{\lambda} = 1.$

Poisson Distribution: Expectation

- We compute $\mathbb{E}[X]$ directly from the definition.

$$\mathbb{E}[X] = \sum_{k=0}^{\infty} k \cdot \frac{e^{-\lambda} \lambda^k}{k!}.$$

- The term for $k = 0$ is zero, so start from $k = 1$:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} \frac{e^{-\lambda} \lambda^k}{(k-1)!}.$$

- Let $j = k - 1$ (so j runs from 0 to ∞):

$$\mathbb{E}[X] = \sum_{j=0}^{\infty} \frac{e^{-\lambda} \lambda^{j+1}}{j!} = \lambda \sum_{j=0}^{\infty} \frac{e^{-\lambda} \lambda^j}{j!}.$$

- The sum is the total probability = 1. Thus,

$$\mathbb{E}[X] = \lambda.$$

Poisson Distribution: Variance

- First compute $\mathbb{E}[X^2]$.

$$\mathbb{E}[X^2] = \sum_{k=0}^{\infty} k^2 \frac{e^{-\lambda} \lambda^k}{k!}.$$

- Note that $\mathbb{E}[X^2] = \mathbb{E}[X(X-1)] + \mathbb{E}[X]$.
- Compute $\mathbb{E}[X(X-1)]$:

$$\mathbb{E}[X(X-1)] = \sum_{k=0}^{\infty} k(k-1) \frac{e^{-\lambda} \lambda^k}{k!} = \sum_{k=2}^{\infty} \frac{e^{-\lambda} \lambda^k}{(k-2)!}.$$

- Let $j = k - 2$:

$$\mathbb{E}[X(X-1)] = \sum_{j=0}^{\infty} \frac{e^{-\lambda} \lambda^{j+2}}{j!} = \lambda^2 \sum_{j=0}^{\infty} \frac{e^{-\lambda} \lambda^j}{j!} = \lambda^2.$$

Poisson Distribution: Variance

- Therefore, $\mathbb{E}[X^2] = \lambda^2 + \lambda$.
- Finally,

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = (\lambda^2 + \lambda) - \lambda^2 = \lambda.$$

- **Summary for Poisson:**

$$\mathbb{E}[X] = \lambda, \quad \text{Var}(X) = \lambda \quad (\text{mean} = \text{variance})$$

Normal (Gaussian) Distribution: Definition

- The Normal distribution is the most important continuous distribution in statistics.
- It is bell-shaped, symmetric, and characterized by two parameters: mean μ and variance σ^2 .
- We write $X \sim \mathcal{N}(\mu, \sigma^2)$.
- Probability density function (pdf):

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

- The constant $\frac{1}{\sqrt{2\pi\sigma^2}}$ ensures that $\int_{-\infty}^{\infty} f(x) dx = 1$.
- The distribution is symmetric about μ ; the peak occurs at $x = \mu$.

Standard Normal Distribution

- A special case: $\mu = 0, \sigma^2 = 1$.
- This is called the standard normal distribution, denoted $Z \sim \mathcal{N}(0, 1)$.
- Its pdf is

$$\phi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}.$$

- Any normal random variable can be transformed to standard normal:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1).$$

- This standardization is useful for computing probabilities using tables.

Quick Excursion: The Gaussian Integral

- The constant $\frac{1}{\sqrt{2\pi\sigma^2}}$ is chosen so that $\int_{-\infty}^{\infty} f(x) dx = 1$.
- This relies on the classic result:

$$I = \int_{-\infty}^{\infty} e^{-z^2/2} dz = \sqrt{2\pi}.$$

- Proof sketch (polar coordinates trick):

$$\begin{aligned} I^2 &= \int_{-\infty}^{\infty} e^{-x^2/2} dx \int_{-\infty}^{\infty} e^{-y^2/2} dy \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} e^{-(x^2+y^2)/2} dx dy \\ &= \int_0^{2\pi} \int_0^{\infty} e^{-r^2/2} r dr d\theta \quad (\text{polar coordinates}) \\ &= 2\pi \int_0^{\infty} e^{-u} du \quad (u = r^2/2) \\ &= 2\pi. \quad \text{Hence, } I = \sqrt{2\pi}. \end{aligned}$$

Normal Distribution: Expectation

- We want to show $\mathbb{E}[X] = \mu$.
- By definition,

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} dx.$$

- Change variables: $z = \frac{x-\mu}{\sigma}$, so $x = \mu + \sigma z$, $dx = \sigma dz$; and then

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} (\mu + \sigma z) \cdot \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz.$$

- Split the integral:

$$\mathbb{E}[X] = \mu \int_{-\infty}^{\infty} \phi(z) dz + \sigma \int_{-\infty}^{\infty} z\phi(z) dz.$$

- The first integral is 1 (total probability). The second integral is 0 because $z\phi(z)$ is an odd function integrated over symmetric limits.
- Hence, $\mathbb{E}[X] = \mu$.

Normal Distribution: Variance

- We want to show $\text{Var}(X) = \sigma^2$.
- $\text{Var}(X) = \mathbb{E}[(X - \mu)^2]$.
- Compute:

$$\mathbb{E}[(X - \mu)^2] = \int_{-\infty}^{\infty} (x - \mu)^2 \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} dx.$$

- Again set $z = \frac{x-\mu}{\sigma}$, so $x - \mu = \sigma z$, $dx = \sigma dz$.
- Then

$$\mathbb{E}[(X - \mu)^2] = \int_{-\infty}^{\infty} \sigma^2 z^2 \cdot \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz = \sigma^2 \int_{-\infty}^{\infty} z^2 \phi(z) dz.$$

- The integral $\int_{-\infty}^{\infty} z^2 \phi(z) dz$ is the variance of a standard normal, which is 1 (can be shown via integration by parts or using the fact that $\int z^2 e^{-z^2/2} dz = \sqrt{2\pi}$).
- Thus, $\text{Var}(X) = \sigma^2 \cdot 1 = \sigma^2$.

Chi-Square Distribution: Definition

- Let $Z_1, Z_2, \dots, Z_k$ be independent standard normal random variables:

$$Z_i \sim \mathcal{N}(0, 1).$$

- Define

$$X = Z_1^2 + Z_2^2 + \dots + Z_k^2.$$

- Then X follows a **chi-square distribution** with k degrees of freedom.
- Notation: $X \sim \chi_k^2$ or $\chi^2(k)$.
- The parameter k (degrees of freedom) determines the shape.

Chi-Square Distribution: Shape and Properties

- The chi-square distribution is defined only for $x \geq 0$.
- It is skewed to the right, especially for small k .
- As k increases, the distribution becomes more symmetric and approaches a normal distribution.

- **Mean:**

$$\mathbb{E}[X] = k.$$

- **Variance:**

$$\text{Var}(X) = 2k.$$

- These can be derived using properties of moments of standard normal variables.

Chi-Square Distribution: Derivation of Mean

- Since $X = \sum_{i=1}^k Z_i^2$, by linearity of expectation,

$$\mathbb{E}[X] = \sum_{i=1}^k \mathbb{E}[Z_i^2].$$

- For a standard normal, $\mathbb{E}[Z_i^2] = \text{Var}(Z_i) + (\mathbb{E}[Z_i])^2 = 1 + 0 = 1$.
- Hence $\mathbb{E}[X] = k \cdot 1 = k$.

Chi-Square Distribution: Derivation of Variance

- To find $\text{Var}(X)$, we compute $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$.
- First, $\mathbb{E}[X^2] = \mathbb{E}\left[\left(\sum Z_i^2\right)^2\right] = \mathbb{E}\left[\sum_i Z_i^4 + \sum_{i \neq j} Z_i^2 Z_j^2\right]$.
- For a standard normal, $\mathbb{E}[Z_i^4] = 3$ (fourth moment; can be derived via integration by parts or moment generating function).
- By independence, $\mathbb{E}[Z_i^2 Z_j^2] = \mathbb{E}[Z_i^2] \mathbb{E}[Z_j^2] = 1 \cdot 1 = 1$ for $i \neq j$.
- There are k terms with Z_i^4 and $k(k-1)$ cross terms.
- Thus $\mathbb{E}[X^2] = k \cdot 3 + k(k-1) \cdot 1 = 3k + k(k-1) = k^2 + 2k$.
- Therefore $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = (k^2 + 2k) - k^2 = 2k$.

Chi-Square Distribution: Probability Density Function

- The pdf of χ_k^2 is given by

$$f(x) = \frac{1}{2^{k/2} \Gamma(k/2)} x^{k/2-1} e^{-x/2}, \quad x > 0,$$

where $\Gamma(\cdot)$ is the Gamma function.

- For integer arguments, $\Gamma(n) = (n-1)!$.
- For half-integer arguments, $\Gamma(1/2) = \sqrt{\pi}$, and

$$\Gamma\left(m + \frac{1}{2}\right) = \frac{(2m)!}{4^m m!} \sqrt{\pi}, \quad m = 1, 2, 3, \dots$$

- A full derivation of this pdf can be found in the appendix.

Chi-Square Distribution: Applications

- **Goodness-of-fit tests:** Compare observed frequencies with expected frequencies under a hypothesized distribution.
- **Tests of independence:** In contingency tables, test whether two categorical variables are independent.
- **Confidence intervals for variance:** For a normal population, the sample variance follows a scaled chi-square distribution.
- **Building block** for the t-distribution and F-distribution.

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- **Central Limit Theorem**
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Central Limit Theorem (Formal Statement)

- Let $X_1, X_2, \dots, X_n$ be independent and identically distributed (= i.i.d.) random variables.
- Suppose that $\mu = \mathbb{E}[X_i] < \infty$ and $\sigma^2 = \text{Var}(X_i) < \infty$.
- Define the sample mean $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$.
- Then, as $n \rightarrow \infty$, the distribution of

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$$

converges to the standard normal distribution $\mathcal{N}(0, 1)$.

- This means: for any $a < b$,

$$\lim_{n \rightarrow \infty} P(a \leq Z_n \leq b) = \int_a^b \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz.$$

Moment Generating Functions (MGF)

- To prove the CLT, we use moment generating functions.
- For a random variable Y , the MGF is defined as

$$M_Y(t) = \mathbb{E}[e^{tY}], \quad \text{for } t \text{ in a neighborhood of } 0.$$

- If two distributions have the same MGF (in a neighborhood of 0), they are identical.
- Key properties:
 1. If $Y_1, \dots, Y_n$ are independent, then

$$M_{Y_1 + \dots + Y_n}(t) = M_{Y_1}(t) \cdots M_{Y_n}(t).$$

2. For constants a, b , $M_{aY+b}(t) = e^{bt} M_Y(at)$.
- For a standard normal $Z \sim \mathcal{N}(0, 1)$, we have

$$M_Z(t) = e^{t^2/2}.$$

MGF of the Standardized Sample Mean

- Let $Y_i = X_i - \mu$, so $\mathbb{E}[Y_i] = 0$ and $\text{Var}(Y_i) = \sigma^2$.

- Then

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} = \frac{1}{\sqrt{n}\sigma} \sum_{i=1}^n Y_i.$$

- By independence,

$$\begin{aligned} M_{Z_n}(t) &= \mathbb{E} \left[\exp \left(\frac{t}{\sqrt{n}\sigma} \sum_{i=1}^n Y_i \right) \right] = \prod_{i=1}^n \mathbb{E} \left[\exp \left(\frac{t}{\sqrt{n}\sigma} Y_i \right) \right] \\ &= \left[M_Y \left(\frac{t}{\sqrt{n}\sigma} \right) \right]^n, \end{aligned}$$

where M_Y is the MGF of Y_i (same for all i).

Taylor Expansion of M_Y

- Since $\mathbb{E}[Y] = 0$ and $\mathbb{E}[Y^2] = \sigma^2$, we can expand $M_Y(s)$ around $s = 0$:

$$M_Y(s) = M_Y(0) + M_Y'(0)s + \frac{M_Y''(0)}{2}s^2 + o(s^2) \quad \text{as } s \rightarrow 0.$$

- We have:

$$M_Y(0) = 1,$$

$$M_Y'(0) = \mathbb{E}[Y] = 0,$$

$$M_Y''(0) = \mathbb{E}[Y^2] = \sigma^2.$$

- Therefore,

$$M_Y(s) = 1 + \frac{\sigma^2}{2}s^2 + o(s^2).$$

Applying the Expansion to M_{Z_n}

- Set $s = \frac{t}{\sqrt{n}\sigma}$. Then as $n \rightarrow \infty$, $s \rightarrow 0$.

•

$$M_Y\left(\frac{t}{\sqrt{n}\sigma}\right) = 1 + \frac{\sigma^2}{2} \left(\frac{t}{\sqrt{n}\sigma}\right)^2 + o\left(\frac{t^2}{n\sigma^2}\right) = 1 + \frac{t^2}{2n} + o\left(\frac{1}{n}\right).$$

- Hence,

$$M_{Z_n}(t) = \left[1 + \frac{t^2}{2n} + o\left(\frac{1}{n}\right)\right]^n.$$

Taking the Limit

- Recall the exponential limit: for any constant c ,

$$\lim_{n \rightarrow \infty} \left(1 + \frac{c}{n} + o\left(\frac{1}{n}\right) \right)^n = e^c.$$

- Apply this with $c = \frac{t^2}{2}$:

$$\lim_{n \rightarrow \infty} M_{Z_n}(t) = e^{t^2/2}.$$

- But $e^{t^2/2}$ is exactly the MGF of a standard normal random variable.
- By the uniqueness of MGFs (if they exist in a neighborhood of 0), we conclude that Z_n converges in distribution to $\mathcal{N}(0, 1)$.

Remarks on the Proof

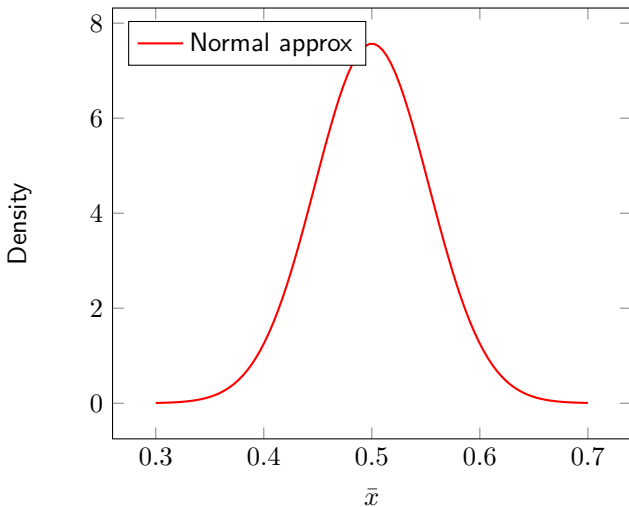
- The proof assumed that $M_Y(t)$ exists in a neighborhood of 0 (i.e., all moments are finite). This is not strictly necessary; the CLT holds under the weaker condition of finite variance (Lindeberg–Lévy CLT).
- The Taylor expansion used the fact that $\mathbb{E}[Y] = 0$ and $\mathbb{E}[Y^2] = \sigma^2$. Higher-order terms become negligible as n grows.
- The argument shows that the MGF of Z_n converges to the MGF of $\mathcal{N}(0, 1)$, which implies convergence in distribution (continuity theorem for MGFs).
- This proof is the classical “MGF proof” and is standard in introductory mathematical statistics.

Why is the CLT Important?

- It justifies using the normal distribution for many real-world averages.
- It allows us to make probability statements about sample means even if we don't know the population distribution.
- It is the foundation for confidence intervals and hypothesis tests for means.
- Rule of thumb: $n \geq 30$ is often enough for a good approximation, but it depends on the underlying distribution.

Illustration of CLT

Distribution of Sample Mean ($n = 30$) from Uniform[0,1]



Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- **Estimating Parameters from Samples**
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Unbiased Estimators

- An **estimator** is a statistic (a function of the sample) used to estimate a population parameter.
- Let $\hat{\theta}$ be an estimator of a parameter θ . We say $\hat{\theta}$ is **unbiased** if

$$\mathbb{E}[\hat{\theta}] = \theta.$$

- Unbiasedness means that on average (over many samples), the estimator hits the true value.
- Example: The sample mean $\bar{X}$ is an unbiased estimator of the population mean μ , because $\mathbb{E}[\bar{X}] = \mu$.

The Sample Mean Revisited

- For a sample $x_1, \dots, x_n$, the sample mean is $\bar{x} = \frac{1}{n} \sum x_i$.
- As a random variable $\bar{X}$,

$$\mathbb{E}[\bar{X}] = \mu, \quad \text{Var}(\bar{X}) = \frac{\sigma^2}{n}.$$

- This uses linearity of expectation and the variance property for independent variables.

Estimating the Variance – First Attempt

- We want an estimator of the population variance σ^2 .
- A natural idea is to use the average squared deviation from the sample mean:

$$\tilde{S}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2.$$

- But this estimator turns out to be **biased**: on average it underestimates σ^2 .
- We will compute its expected value using the tools from Part II.

Derivation of the Bias: Step 1

Let $X_1, \dots, X_n$ be i.i.d. with mean μ and variance σ^2 . We analyze

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right].$$

First, add and subtract μ inside the parentheses:

$$X_i - \bar{X} = (X_i - \mu) + (\mu - \bar{X}).$$

Derivation: Step 2 – Expand the Square

Square both sides and sum over i :

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n (X_i - \mu)^2 + \sum_{i=1}^n (\mu - \bar{X})^2 + 2 \sum_{i=1}^n (X_i - \mu)(\mu - \bar{X}).$$

The second term is $n(\mu - \bar{X})^2$, because $(\mu - \bar{X})^2$ does not depend on i .

For the cross term, note that $\sum_{i=1}^n (X_i - \mu) = n(\bar{X} - \mu)$. Hence

$$2(\mu - \bar{X}) \sum_{i=1}^n (X_i - \mu) = 2(\mu - \bar{X}) \cdot n(\bar{X} - \mu) = -2n(\bar{X} - \mu)^2.$$

Therefore,

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

Derivation: Step 3 – Take Expectations

Now take expectations on both sides. By definition of variance,

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \mu)^2 \right] = n\sigma^2.$$

For the second term, note that $\mathbb{E}[\bar{X}] = \mu$, so

$$\mathbb{E}[(\bar{X} - \mu)^2] = \text{Var}(\bar{X}) = \frac{\sigma^2}{n}.$$

Thus,

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = n\sigma^2 - n \left(\frac{\sigma^2}{n} \right) = (n-1)\sigma^2.$$

Derivation: Step 4 – The Unbiased Estimator

From the previous slide,

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = (n-1)\sigma^2.$$

Therefore,

$$\mathbb{E} \left[\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 \right] = \sigma^2.$$

This shows that the estimator

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

is **unbiased** for the population variance σ^2 .

Sample Variance and Standard Deviation

- The **sample variance** (unbiased) is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

- The denominator $n - 1$ is called **Bessel's correction**.
- Intuition: we have “used up” one degree of freedom by estimating the mean, so we average over the remaining $n - 1$ independent pieces of information.
- The **sample standard deviation** is $s = \sqrt{s^2}$ (same units as the data).
- For the coffee data 0, 1, 2, 3, 9 with $\bar{x} = 3$ (see our previous example):

$$\sum_{i=1}^5 (x_i - 3)^2 = 50 \quad \implies \quad s = \sqrt{\frac{50}{4}} \approx 3.54 \text{ cups.}$$

Summary: Mean and Variance Estimation

- The sample mean $\bar{X}$ is an unbiased estimator of μ .
- The naive variance estimator $\frac{1}{n} \sum (X_i - \bar{X})^2$ is biased; its expectation is $\frac{n-1}{n} \sigma^2$.
- Dividing by $n - 1$ instead of n removes the bias, giving the unbiased sample variance S^2 .
- This correction (Bessel's correction) is essential when estimating population variance from a sample.

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- **Interval Estimation**
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Point Estimates Leave Us Uncertain

- From our coffee data, we got $\bar{x} = 3$ cups as our estimate of the true university average μ .
- But is μ exactly 3? Probably not.
- How far off could we be? 0.1 cups? 1 cup? 5 cups?
- **Important:** a point estimate tells us nothing about precision.
- We need a way to say: " μ is likely somewhere between L and U ."

Confidence Intervals: The Basic Idea

- A **confidence interval** is a range $[L, U]$ that we believe contains the true parameter θ .
- The **confidence level**, $1 - \alpha$, tells us how "confident" we are.

False Interpretation:

"There is a $(1 - \alpha)$ probability that θ lies in this particular interval"

Important: The true parameter, θ , is not random.

Confidence Intervals: The Basic Idea

- A **confidence interval** is a range $[L, U]$ that we believe contains the true parameter θ .
- The **confidence level**, $1 - \alpha$, tells us how "confident" we are.

Correct Interpretation:

"If we repeated the sampling process many times, about $(1 - \alpha)$ of the intervals we construct would contain the true θ ".

Important: The true parameter, θ , is not random.

Computing Confidence Probabilities – A Warm-Up

- Suppose we know that $X \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 3$ and $\sigma = 2$.
- What is $P(1 \leq X \leq 5)$?
- Standardize: $Z = \frac{X-\mu}{\sigma} = \frac{X-3}{2}$.

$$x = 1 \quad \Rightarrow \quad z = \frac{1-3}{2} = -1,$$

$$x = 5 \quad \Rightarrow \quad z = \frac{5-3}{2} = 1.$$

- Then

$$\begin{aligned} P(1 \leq X \leq 5) &= P(-1 \leq Z \leq 1) \\ &= \int_{-1}^1 \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz \approx 0.6827 \end{aligned}$$

In summary, that's approximately 68%.

- So for a **known** distribution, we can compute exact probabilities.

From Known Distribution to Unknown Parameter

- In reality, we don't know μ . We had $\bar{x} = 3$ from our coffee sample.
- Could the true mean μ be 10? Possibly.
- Could it be 1000? Also possible?
- Without further assumptions, any value is possible.

The data alone don't tell us what's plausible.

- This is sometimes called the **"no free lunch" principle** in statistics: you cannot make precise statements about an unknown parameter without making assumptions about how the data were generated.

From Known Distribution to Unknown Parameter

- To make progress, we must assume a **probability model** for the data.
- The standard choice: assume $X_1, \dots, X_n$ are i.i.d. from a normal distribution $\mathcal{N}(\mu, \sigma^2)$.
- Is this assumption true? Probably not. But it's a useful approximation, and many real datasets are approximately Gaussian.
- And, yes: the CLT is a good motivation to work with Gaussians!
- Nevertheless: if the true distribution is completely different, our conclusions on the slides below will be wrong – that's the risk we take.

From Known Distribution to Unknown Parameter

- **Assume** that $X_1, \dots, X_n$ are i.i.d. $\mathcal{N}(\mu, \sigma^2)$ and that σ is **known**.
- Under this assumption, the sampling distribution of $\bar{X}$ is exactly

$$\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

- This distribution still depends on the unknown μ , so we cannot directly use it.
- But we can rearrange to eliminate μ from the distribution:

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0, 1).$$

- We can now make probability statements about Z , then invert them to get an interval for μ .

Deriving the Confidence Interval (Known σ)

- We know $\bar{X} \sim \mathcal{N}(\mu, \sigma^2/n)$. Standardize:

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0, 1).$$

- For a given confidence level $1 - \alpha$, find $z_{\alpha/2}$ such that

$$P(-z_{\alpha/2} \leq Z \leq z_{\alpha/2}) = 1 - \alpha.$$

- For $1 - \alpha = 95\%$, we have $z_{0.025} = 1.96$ (see Z-table on next slide).
- Rearranging the inequalities inside the probability yields

$$P\left(\bar{X} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}\right) = 1 - \alpha.$$

- Now, $[\bar{X} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}}, \bar{X} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}]$ contains μ with probability $1 - \alpha$.

Standard Normal Table (Z-table)

z	1.64	1.96	2.33	2.58
$P(Z > z)$	0.05	0.025	0.01	0.005

- For a standard normal random variable $Z \sim \mathcal{N}(0, 1)$, the table gives the right-tail probability $P(Z > z)$.
- Example: $z_{0.025} = 1.96$ because $P(Z > 1.96) = 0.025$.
- For a two-sided confidence level $1 - \alpha$, we need $z_{\alpha/2}$ such that $P(Z > z_{\alpha/2}) = \alpha/2$.
- These values come from solving $\int_{z_{\alpha/2}}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt = \alpha/2$.

Example: Known σ

- Suppose for our coffee data, we magically know that $\sigma = 2$ cups.
- $\bar{x} = 3$, $n = 5$, $\frac{\sigma}{\sqrt{n}} = \frac{2}{\sqrt{5}} \approx 0.894$.
- For 95% confidence, $z_{0.025} = 1.96$.
- Margin of error: $1.96 \times 0.894 \approx 1.75$.
- Hence, our 95% confidence interval (CI) is:

$$[3 - 1.75, 3 + 1.75] = [1.25, 4.75].$$

- Interpretation: We are 95% confident that the true mean μ lies between 1.25 and 4.75 cups.
- But wait – we don't actually know σ in real life...
- Much worse: the number of cups is discrete—definitely not Gaussian.

The Problem: Unknown σ

- In practice, we rarely know σ .
- Instead, we estimate it by $s = \frac{1}{n-1} \sum (X_i - \bar{X})^2$.
- If we simply replace σ with s , the statistic

$$\frac{\bar{X} - \mu}{s/\sqrt{n}}$$

is **not** normally distributed.

- Why? Because s is a random variable that fluctuates from sample to sample, adding extra uncertainty.
- For small samples, this extra uncertainty is significant.

Introducing the t-Distribution

- When σ is unknown, we replace it with s , but the ratio

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}$$

is no longer normal.

- William Sealy Gosset ("Student") derived its exact distribution under the assumption that $X_1, \dots, X_n$ are i.i.d. $\mathcal{N}(\mu, \sigma^2)$.
- Key fact: $\frac{(n-1)s^2}{\sigma^2} \sim \chi_{n-1}^2$ (a chi-square distribution with $n - 1$ degrees of freedom).

Introducing the t-Distribution

- In summary, T can be written as

$$T = \frac{(\bar{X} - \mu)/(\sigma/\sqrt{n})}{\sqrt{\frac{(n-1)s^2}{\sigma^2}/(n-1)}} = \frac{Z}{\sqrt{U/(n-1)}},$$

where $Z \sim \mathcal{N}(0, 1)$ and $U \sim \chi_{n-1}^2$ are independent.

- This ratio follows a **t-distribution** with $n - 1$ degrees of freedom.

Notation: $T \sim t_{n-1}$.

- The t-distribution looks like the standard normal but has **heavier tails** – it accounts for the extra uncertainty from estimating σ .

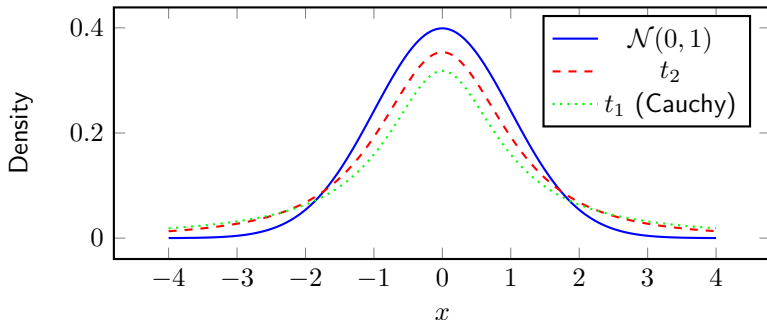
t-Distribution: Probability Density Function

- The pdf of t_k is

$$f(x) = \frac{\Gamma(\frac{k+1}{2})}{\sqrt{k\pi} \Gamma(\frac{k}{2})} \left(1 + \frac{x^2}{k}\right)^{-\frac{k+1}{2}}, \quad x \in \mathbb{R}.$$

- This formula is derived from the joint distribution of Z and U and a change of variables. (we skip the rather technical details here).
- Important features:
 - Symmetric about 0, bell-shaped, but with heavier tails than the normal.
 - As $k \rightarrow \infty$, t_k converges to $\mathcal{N}(0, 1)$.
 - For small k , the tails are much heavier.
- For our coffee example with $n = 5$, we have $k = n - 1 = 4$.

Visualizing the t-Distribution



- As n increases, t_{n-1} approaches $\mathcal{N}(0, 1)$.
- For small n , the tails are much heavier – reflecting greater uncertainty.

Confidence Interval with t-Distribution

- Let $t_{\alpha/2, n-1}$ be such that $P(T \geq t_{\alpha/2, n-1}) = \alpha/2$ for $T \sim t_{n-1}$.

- Then

$$P\left(-t_{\alpha/2, n-1} \leq \frac{\bar{X} - \mu}{s/\sqrt{n}} \leq t_{\alpha/2, n-1}\right) = 1 - \alpha.$$

- Rearranging:

$$\bar{X} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}.$$

- This interval is **wider** than the one using z – the price we pay for not knowing σ .

Student's t-table (Critical Values)

df \ α (right-tail)	0.10	0.05	0.025	0.01	0.005
1	3.078	6.314	12.706	31.821	63.657
2	1.886	2.920	4.303	6.965	9.925
3	1.638	2.353	3.182	4.541	5.841
4	1.533	2.132	2.776	3.747	4.604
5	1.476	2.015	2.571	3.365	4.032
10	1.372	1.812	2.228	2.764	3.169
20	1.325	1.725	2.086	2.528	2.845
30	1.310	1.697	2.042	2.457	2.750
∞ (normal)	1.282	1.645	1.960	2.326	2.576

- The table gives the value $t_{\alpha,df}$ such that $P(T > t_{\alpha,df}) = \alpha$ for a t-distribution with df degrees of freedom.

Example: Coffee Data (Unknown σ)

- Coffee data: 0, 1, 2, 3, 9, $n = 5$, $\bar{x} = 3$, $s \approx 3.54$.
- $s/\sqrt{n} = 3.54/\sqrt{5} \approx 1.583$.
- Degrees of freedom: $n - 1 = 4$.
- For 95% confidence, $t_{0.025,4} = 2.776$ (from t-table).
- Margin of error: $2.776 \times 1.583 \approx 4.40$.
- The corresponding 95% confidence interval is given by

$$[3 - 4.40, 3 + 4.40] = [-1.40, 7.40].$$

- This interval includes negative values— clearly a limitation of our false assumption that the data is Gaussian!

When Can We Use These Intervals?

- **Assumption:** The data are drawn from a **Gaussian distribution**.
- If the data are not Gaussian, the t -interval may not be accurate, especially for small samples.
- For large samples ($n \gtrsim 30$), often the Central Limit Theorem kicks in:
 - The average value $\bar{X}$ is approximately normal even if the data aren't.
 - The t -interval is approximately correct—even for non-normal data.
- This is why the t -interval is called "robust" to moderate departures from normality.

Summary: Confidence Intervals for the Mean

- **Known σ , normal data:**

$$\bar{x} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}, \quad z \sim \mathcal{N}(0, 1).$$

- **Unknown σ , normal data:**

$$\bar{x} \pm t_{\alpha/2, n-1} \frac{s}{\sqrt{n}}, \quad t \sim t_{n-1}.$$

- The t-interval is wider – it accounts for the extra uncertainty from estimating σ .
- For large n , $t_{\alpha/2, n-1} \approx z_{\alpha/2}$, so the two intervals become similar.
- Always check assumptions! The t-interval assumes (approximately) normal data or large n .

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- **Hypothesis Testing**
- Non-parametric Tests and Chi-Square
- Summary
- Appendix: Derivation of the Chi-Square PDF

Recall: Our Coffee Data

- Earlier we surveyed 5 students about their coffee consumption:

$$0, 1, 2, 3, 9 \implies \bar{x} = 3, s \approx 3.54.$$

- The university administration claims:
"The average student drinks 2 cups of coffee per day."
- Question: Does our sample provide evidence against this claim?
- This is a classic **hypothesis testing problem**.

Setting Up the Hypotheses

- Let μ be the true average coffee consumption of *all* students.
- The administration's claim is: $\mu = 2$.
- This becomes our **null hypothesis**:

$$H_0 : \mu = 2.$$

- The **alternative hypothesis** is what we suspect if the claim is false:

$$H_1 : \mu \neq 2 \quad (\text{two-sided alternative}).$$

- We want to see if the data are consistent with H_0 , or if they point toward H_1 .

The Logic of Hypothesis Testing

- Assume for a moment that H_0 is true ($\mu = 2$).
- Under this assumption, we ask: How likely are we to observe a sample mean as extreme as $\bar{x} = 3$?
- If such an extreme value is *very unlikely* under H_0 , that casts doubt on H_0 .
- If it's reasonably likely, we have no grounds to reject H_0 .
- This is like a "proof by contradiction" – we assume H_0 is true and see if the data contradict it.

What Do We Need to Quantify "Unlikely"?

- To assess how extreme $\bar{x} = 3$ is under H_0 , we need the **sampling distribution** of $\bar{X}$.
- But the sampling distribution depends on how the data were generated – and we don't know that.
- This brings us to a fundamental truth: **we must make assumptions.**
- For now, we assume:
 1. The data come from a normal distribution.
 2. The population standard deviation σ is known.Let's say previous large studies found $\sigma = 2.5$ cups.

What Do We Need to Quantify "Unlikely"?

- Are the above assumptions realistic for our coffee data? They are not.
- **But here's the key:** Statistics is about making the best possible progress under uncertainty. Without any assumptions, we're stuck.
- This is the "no free lunch" principle.
- All models are wrong, but some are useful.
- The above assumptions, while imperfect, let us do exact calculations.
To which extent these calculations lead to plausible conclusions, is something that we will have to discuss later.

The Test Statistic (z-test)

- Under H_0 , we have $\bar{X} \sim \mathcal{N}(\mu_0, \sigma^2/n)$ with $\mu_0 = 2$, $\sigma = 2.5$, $n = 5$.
- Standardize to get a **test statistic** that follows a standard normal distribution under H_0 :

$$Z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}}.$$

- For our sample, $\bar{x} = 3$, so

$$z = \frac{3 - 2}{2.5/\sqrt{5}} = \frac{1}{1.118} \approx 0.894.$$

- If H_0 is true, Z should be close to 0. Our observed $z = 0.894$ is positive but not huge.

Measuring "Extremeness": The p-value

- The **p-value** is the probability, under H_0 , of observing a test statistic as extreme as (or more extreme than) the one we got.
- For a two-sided test, "more extreme" means $|Z| \geq |z|$.
- So:

$$\text{p-value} = P(|Z| \geq 0.894) = 2 \times P(Z \geq 0.894).$$

- From the standard normal table: $P(Z \geq 0.894) \approx 0.186$.
- Thus $\text{p-value} \approx 2 \times 0.186 = 0.372$.

Interpreting the p-value

- If H_0 is true ($\mu = 2$), there is about a 37% chance of seeing a sample mean as far from 2 as 3 (or farther).
- That's not particularly unlikely – it happens more than one-third of the time by random chance alone.
- Therefore, our data do not provide strong evidence against H_0 .
- We say: we **fail to reject** H_0 .
- Important: This does *not* mean we have proven $\mu = 2$! It means the data are consistent with H_0 .

Setting a Threshold: Significance Level α

- In practice, we often choose a cutoff α (say, 5%) to decide when the evidence is strong enough.
- If $p\text{-value} < \alpha$, we reject H_0 ; otherwise, we fail to reject.
- The cutoff α is also called the **significance level**.
- It represents the risk we are willing to take of making a **Type I error**: rejecting H_0 when it is actually true.
- For our coffee example, $p\text{-value} = 0.372 > 0.05$, so we fail to reject H_0 at the 5% level.

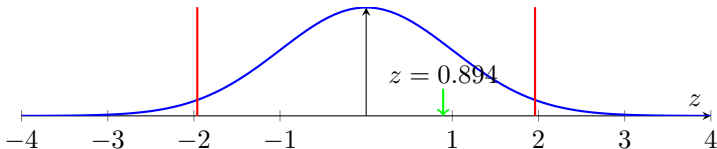
The Two Types of Errors

Reality	Decision	
	Fail to reject H_0	Reject H_0
H_0 true	Correct	Type I error (prob. α)
H_0 false	Type II error (prob. β)	Correct (prob. $1 - \beta$)

- We choose α (say 5%) to control Type I error.
- In our coffee example, failing to reject H_0 is very like a Type II error—the true μ might actually not be exactly 2.

Critical Value Approach

- Instead of computing a p-value, we can pre-specify a **rejection region**.
- For $\alpha = 0.05$, we find $z_{\alpha/2} = z_{0.025} = 1.96$ from the Z-table.
- Reject H_0 if $|z| > 1.96$.
- Our $z = 0.894$ is not in the rejection region, so we fail to reject.
- This is equivalent to the p-value approach: we reject if p-value $< \alpha$.



Summary of Our Coffee Example

- We tested $H_0 : \mu = 2$ against $H_1 : \mu \neq 2$ using a sample mean of 3 cups.
- Under the assumptions (normal data, known $\sigma = 2.5$), we got $z = 0.894$, $p\text{-value} = 0.372$.
- At $\alpha = 0.05$, we fail to reject H_0 .
- Conclusion: The data do not provide strong evidence that the true average coffee consumption differs from 2 cups.
- But wait – our assumptions are questionable. The data are not normal (outlier 9), and we don't really know σ .

What If σ Is Unknown? The t-Test

- In reality, we rarely know σ . We estimate it by the sample standard deviation.
- The test statistic becomes:

$$T = \frac{\bar{X} - \mu_0}{s/\sqrt{n}}.$$

- Under H_0 and the normality assumption, T follows a **t-distribution** with $n - 1$ degrees of freedom.
- For our coffee data: $\bar{x} = 3$, $s \approx 3.54$, $n = 5$, so

$$t = \frac{3 - 2}{3.54/\sqrt{5}} \approx \frac{1}{1.583} \approx 0.632.$$

- Critical value for $\alpha = 0.05$, $df=4$: $t_{0.025,4} = 2.776$. $|t| < 2.776$, so we still fail to reject.

When Are These Tests Valid?

- Both z-test and t-test assume the data are a random sample from a normal distribution.
- Our coffee data have an outlier (9 cups) and are clearly not normal. For such a small sample, the test results may be unreliable.
- With larger samples ($n \gtrsim 30$), the Central Limit Theorem makes the tests approximately valid even for non-normal data.
- If the data are very non-normal and n is small, nonparametric tests (like the sign test) are safer.
- In practice, always check your assumptions!

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- **Non-parametric Tests and Chi-Square**
- Summary
- Appendix: Derivation of the Chi-Square PDF

Motivation: Testing a Coin for Fairness

- We flip a coin n times and observe O_1 heads and $O_2 = n - O_1$ tails.
- Is the coin fair? That is, do we really have $p = P(\text{heads}) = 0.5$?
- This is a simple example of a goodness-of-fit problem: we have observed counts and want to see if they match a hypothesized distribution.
- We'll first develop a test that works for this case. Later we generalize this to more categories.

Two Categories: Heads and Tails

- Under the null hypothesis $H_0 : p = p_0$ (e.g., $p_0 = 0.5$), the expected counts are

$$E_1 = np_0, \quad E_2 = n(1 - p_0).$$

- A natural measure of discrepancy is the sum of squared, scaled differences,

$$\chi^2 = \frac{(O_1 - E_1)^2}{E_1} + \frac{(O_2 - E_2)^2}{E_2}.$$

This measure is also known as Pearson's χ^2 -statistic.

- We will simplify this expression and show that it equals the square of the familiar z-statistic.

Simplifying the χ^2 Statistic

$$\begin{aligned}\chi^2 &= \frac{(O_1 - np_0)^2}{np_0} + \frac{((n - O_1) - n(1 - p_0))^2}{n(1 - p_0)} \\&= \frac{(O_1 - np_0)^2}{np_0} + \frac{(-(O_1 - np_0))^2}{n(1 - p_0)} \\&= (O_1 - np_0)^2 \left(\frac{1}{np_0} + \frac{1}{n(1 - p_0)} \right) \\&= (O_1 - np_0)^2 \cdot \frac{1}{n} \left(\frac{1}{p_0} + \frac{1}{1 - p_0} \right) \\&= (O_1 - np_0)^2 \cdot \frac{1}{n} \cdot \frac{1}{p_0(1 - p_0)} \\&= \left(\frac{O_1 - np_0}{\sqrt{np_0(1 - p_0)}} \right)^2.\end{aligned}$$

Connection to the z-Test

- The quantity inside the square is exactly the z-statistic for a single proportion (see also the next slide for details):

$$z = \frac{O_1 - np_0}{\sqrt{np_0(1 - p_0)}}.$$

- Hence $\chi^2 = z^2$.
- Under H_0 , for large n , the Central Limit Theorem tells us that z is approximately standard normal.
- Therefore χ^2 is approximately the square of a standard normal – which is, by definition, a χ^2 distribution with 1 degree of freedom.
- So for two categories, Pearson's χ^2 test is equivalent to a two-sided z-test, and we have a clear derivation of its asymptotic distribution.

The Central Limit Theorem in This Context

- Let $X_i = 1$ if toss i is heads, 0 otherwise. Then $O_1 = \sum X_i$, and the X_i are i.i.d. Bernoulli(p_0).
- Mean $\mu = p_0$, variance $\sigma^2 = p_0(1 - p_0)$.
- By the CLT,

$$\frac{\bar{X} - p_0}{\sigma/\sqrt{n}} = \frac{O_1/n - p_0}{\sqrt{p_0(1 - p_0)/n}} = \frac{O_1 - np_0}{\sqrt{np_0(1 - p_0)}} \xrightarrow{d} \mathcal{N}(0, 1).$$

- This convergence justifies the approximation for large n .
- **Assumption:** n is sufficiently large. There is no free lunch – we must assume something, here it's that the sample size is large enough for the CLT to be accurate. A common rule of thumb is that $np_0 \geq 5$ and $n(1 - p_0) \geq 5$.

Goodness-of-Fit for Multiple Categories

- Now suppose we have k categories, with observed counts $O_1, \dots, O_k$ and expected counts $E_1, \dots, E_k$ under H_0 (with $\sum E_i = n$).
- Pearson's chi-square statistic generalises naturally:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}.$$

- For large n , and provided all E_i are not too small, this statistic approximately follows a χ^2 distribution with $k - 1$ degrees of freedom.
- The $k - 1$ degrees of freedom reflect the constraint $\sum O_i = n$.
Once we know $k - 1$ of the counts, the last is determined.

Where Does the Chi-Square Distribution Come From?

- A full derivation is mathematically involved (multivariate CLT, covariance structure, quadratic forms), but the idea is similar to the two-category case:

For large n , the sum of squares behaves like the sum of squares of $k - 1$ independent standard normals; that is, χ_{k-1}^2 .

- This is a mathematical fact, not magic—but it requires advanced tools to prove.
- For practical work, we rely on the approximation and use the rule $E_i \geq 5$ (sometimes $E_i \geq 1$ with caution) to ensure its validity.

Example 1: Is a Die Fair?

- Roll a die 60 times. Under fairness (H_0), each face has probability $1/6$, so $E_i = 10$ for $i = 1, \dots, 6$.

- Observed counts: $O = (8, 9, 11, 10, 12, 10)$.

- Compute

$$\chi^2 = \frac{(8-10)^2}{10} + \frac{(9-10)^2}{10} + \frac{(11-10)^2}{10} + 2\frac{(10-10)^2}{10} + \frac{(12-10)^2}{10} = 1.$$

- Degrees of freedom: $6 - 1 = 5$. From χ^2 tables, $\chi_{0.05,5}^2 = 11.07$.

- Since $1.0 < 11.07$, we fail to reject H_0 .

In other words, there is no evidence that the die is biased.

- Since, $E_i \geq 5$, the assumptions are satisfied—so, this test is plausible.

Chi-Square Table (Critical Values)

df \ α (right-tail)	0.10	0.05	0.025	0.01	0.005
1	2.706	3.841	5.024	6.635	7.879
2	4.605	5.991	7.378	9.210	10.597
3	6.251	7.815	9.348	11.345	12.838
4	7.779	9.488	11.143	13.277	14.860
5	9.236	11.070	12.833	15.086	16.750
10	15.987	18.307	20.483	23.209	25.188
20	28.412	31.410	34.170	37.566	40.000
30	40.256	43.773	46.979	50.892	53.672

- The table gives the value $\chi^2_{\alpha,df}$ such that $P(\chi^2_{df} > \chi^2_{\alpha,df}) = \alpha$ for a chi-square distribution with df degrees of freedom.

Example 2: Smoking and Lung Disease

- Suppose we have data on 200 individuals, classified by smoking status and lung disease:

	Lung Disease	No Disease	Total
Smoker	20	80	100
Non-smoker	5	95	100
Total	25	175	200

Is there an association between smoking and lung disease?

Example 2: Setting Up the Hypotheses

- Define: A = event "person is a smoker".
- Also define: B = event "person has lung disease".
- **Null hypothesis** H_0 : Smoking and lung disease are independent,

$$P(A \cap B) = P(A) \times P(B).$$

- **Alternative hypothesis** H_1 : There is an association (dependence).
- We will use a chi-square test of independence.

Example 2: Expected Counts Under Independence

- If H_0 is true, we can estimate probabilities from the margins:

$$\hat{P}(A) = \frac{100}{200} = 0.5, \quad \hat{P}(B) = \frac{25}{200} = 0.125.$$

- Under independence, the expected count in cell (Smoker, Disease) is

$$E_{11} = n \times \hat{P}(A) \times \hat{P}(B) = 200 \times 0.5 \times 0.125 = 12.5.$$

- Obviously, the above formula generalizes to

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{col } j \text{ total})}{\text{grand total}}.$$

- Hence, we also compute the remaining expected counts:

$$E_{12} = 87.5, \quad E_{21} = 12.5, \quad E_{22} = 87.5.$$

Example 2: Computing the Chi-Square Statistic

- Pearson's chi-square statistic compares observed (O) and expected (E) counts:

$$\chi^2 = \sum_{\text{all cells}} \frac{(O - E)^2}{E}.$$

- For our table:

$$\begin{aligned}\chi^2 &= \frac{(20 - 12.5)^2}{12.5} + \frac{(80 - 87.5)^2}{87.5} \\ &\quad + \frac{(5 - 12.5)^2}{12.5} + \frac{(95 - 87.5)^2}{87.5} \\ &= \frac{56.25}{12.5} + \frac{56.25}{87.5} + \frac{56.25}{12.5} + \frac{56.25}{87.5} \\ &= 4.5 + 0.643 + 4.5 + 0.643 = 10.286.\end{aligned}$$

Example 2: Degrees of Freedom

- For a contingency table with r rows and c columns, degrees of freedom are

$$df = (r - 1)(c - 1).$$

- Why? Once we know the row and column totals, only $(r - 1)(c - 1)$ cells can be filled freely; the rest are determined.
- For our 2×2 table:

$$df = (2 - 1)(2 - 1) = 1.$$

- This means the chi-square statistic will be compared to a χ^2_1 distribution.

Example 2: Making a Decision

- Choose significance level, say, $\alpha = 0.05$.
- From the chi-square table, the critical value for $df = 1$ is

$$\chi_{0.05,1}^2 = 3.84.$$

- Our computed $\chi^2 = 10.286$ is **greater than** 3.84.
- Therefore we **reject the null hypothesis** of independence.
- **Conclusion:** There is statistically significant evidence of an association between smoking and lung disease.
- **Caution:** Association does not imply causation. This test does not tell us that smoking causes lung disease – only that they are related in this data.

Checking Assumptions

- The chi-square test of independence relies on:
 1. **Independence of observations:** Each individual is unrelated to others (satisfied by random sampling).
 2. **Expected counts not too small:** All $E_{ij} \geq 5$ is a common rule of thumb. Here we had $E_{ij} \geq 12.5$ — satisfied.
- If expected counts are too small, the χ^2 approximation may be poor.

The “No Free Lunch” Principle Strikes Again

- We have replaced strong parametric assumptions (e.g., normality) with weaker, but still non-trivial, assumptions.
- We assumed:
 - The data are counts from independent trials.
 - The sample is large enough that the χ^2 approximation is accurate.
 - Under H_0 , the expected counts can be computed (either from a fully specified distribution or from margins).
- If these assumptions are violated, the test may give misleading results.
- There is no such thing as an assumption-free test – we always pay for inference with some assumptions.
- The art of statistics is choosing a test whose assumptions are plausible for the data at hand.

Summary: Chi-Square Tests

- The chi-square test is a versatile tool for categorical data.
- For two categories, it reduces to the familiar z-test (or its square) and is justified by the CLT.
- For multiple categories, the same idea extends, though the derivation is more advanced.
- Two common applications:
 - **Goodness-of-fit:** compare observed counts to a theoretical distribution.
 - **Test of independence:** check if two categorical variables are related.
- Always check the expected-count condition and the independence of observations.
- Remember: assumptions are not optional – they are the price we pay for making inferences.

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- **Summary**
- Appendix: Derivation of the Chi-Square PDF

Summary

In this lecture we covered:

- Descriptive statistics: mean, variance, standard deviation
- Probability basics: random variables, expectation, variance
- Important distributions: binomial, Poisson, normal, t, chi-square
- Central Limit Theorem and its importance
- Interval estimation: confidence intervals for mean
- Hypothesis testing: z-test, t-test, p-values
- Non-parametric tests: chi-square goodness-of-fit, test of independence

Contents

- Introduction
- Descriptive Statistics – First Look
- Probability and Random Variables
- Central Limit Theorem
- Estimating Parameters from Samples
- Interval Estimation
- Hypothesis Testing
- Non-parametric Tests and Chi-Square
- Summary
- **Appendix: Derivation of the Chi-Square PDF**

Why the Moment Generating Function (MGF)

Determines a Distribution

- For a random variable X , the MGF is defined as $M_X(t) = \mathbb{E}[e^{tX}]$, provided the expectation exists for t in a neighbourhood of 0.
- **Uniqueness theorem:** If two random variables have the same MGF (for all t in some interval containing 0), then they have the same cumulative distribution function.
- This follows from the fact that the MGF is essentially the two-sided Laplace transform of the density (or a generating function for discrete probabilities), and Laplace transforms are injective. The details regarding such Laplace transforms are discussed in our IMS courses on calculus and complex analysis.

MGF of a Squared Standard Normal

- Let $Z \sim \mathcal{N}(0, 1)$. We want $M_{Z^2}(t) = \mathbb{E}[e^{tZ^2}]$.
- Compute by integration:

$$M_{Z^2}(t) = \int_{-\infty}^{\infty} e^{tz^2} \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(1-2t)z^2} dz.$$

- The integral converges for $t < \frac{1}{2}$. It is a Gaussian integral:

$$\int_{-\infty}^{\infty} e^{-az^2} dz = \sqrt{\frac{\pi}{a}}, \quad a > 0.$$

- Here $a = \frac{1-2t}{2} > 0$, so

$$M_{Z^2}(t) = \frac{1}{\sqrt{2\pi}} \cdot \sqrt{\frac{2\pi}{1-2t}} = (1-2t)^{-1/2}.$$

MGF of a Chi-Square Variable

- Let $Q_k = Z_1^2 + Z_2^2 + \cdots + Z_k^2$ with independent $Z_i \sim \mathcal{N}(0, 1)$.
- The MGF of Q_k is the product of the individual MGFs:

$$M_{Q_k}(t) = \prod_{i=1}^k M_{Z_i^2}(t) = \left[(1 - 2t)^{-1/2} \right]^k = (1 - 2t)^{-k/2}.$$

- This MGF is defined for $t < \frac{1}{2}$.
- The gamma distribution with shape $\alpha > 0$ and rate $\beta > 0$ has pdf

$$f(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0,$$

and MGF $M(t) = \left(1 - \frac{t}{\beta}\right)^{-\alpha}$ for $t < \beta$.

- Comparing with $M_{Q_k}(t) = (1 - 2t)^{-k/2}$, we see that $\beta = \frac{1}{2}$ and $\alpha = \frac{k}{2}$.

The Chi-Square PDF

- Therefore Q_k follows a gamma distribution with shape $\alpha = k/2$ and rate $\beta = 1/2$.
- Substituting into the gamma pdf gives

$$f_{Q_k}(x) = \frac{(1/2)^{k/2}}{\Gamma(k/2)} x^{k/2-1} e^{-x/2}, \quad x > 0.$$

- This is the probability density function of the chi-square distribution with k degrees of freedom.
- Often it is written as

$$f_{\chi_k^2}(x) = \frac{1}{2^{k/2} \Gamma(k/2)} x^{k/2-1} e^{-x/2}, \quad x > 0.$$

Chapter 2

Multivariate Analysis Models

Multivariate Analysis Models

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

From Univariate to Multivariate

- In Chapter 1 we studied single random variables. Now we need to handle **vectors** of random variables.
- Example: For each student we measure study hours X_1 and exam score X_2 . The pair (X_1, X_2) is a bivariate random vector.
- A p -dimensional random vector $X = (X_1, \dots, X_p)^\top$ is described by its joint probability distribution.

Joint Probability Density Function (pdf)

- For continuous random vectors, the joint pdf $f_X(x)$ satisfies

$$P(X \in A) = \int_A f_X(x) dx.$$

- The marginal density of a single component is obtained by integrating out the others:

$$f_{X_1}(x_1) = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} f_X(x_1, \dots, x_p) dx_2 \cdots dx_p.$$

Expectation Vector and Covariance Matrix

- The **expectation vector** (mean vector) of X is

$$\mu = \mathbb{E}[X] = (\mathbb{E}[X_1], \dots, \mathbb{E}[X_p])^\top.$$

As for the scalar case, $\mathbb{E}\{\cdot\}$ is linear.

- The **covariance matrix** Σ (also called variance-covariance matrix) is a $p \times p$ symmetric matrix with entries

$$\Sigma_{ij} = \text{Cov}(X_i, X_j) = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)].$$

- Diagonal entries are variances: $\Sigma_{ii} = \text{Var}(X_i)$.
- For independent components, Σ is diagonal (all off-diagonals zero).

Covariance and Correlation – Intuition

- Consider two variables, e.g., study hours X_1 and exam score X_2 .
- If students who study more tend to score higher, we expect a **positive covariance** $\text{Cov}(X_1, X_2) > 0$.
- If there were no relationship, covariance would be close to zero.
- The **correlation coefficient** standardises covariance to $[-1, 1]$:

$$\rho_{12} = \frac{\text{Cov}(X_1, X_2)}{\sqrt{\text{Var}(X_1) \text{Var}(X_2)}}.$$

- $\rho = 1$ means perfect positive linear relationship; $\rho = -1$ perfect negative; $\rho = 0$ no linear relationship.
- Example: In our exam data, we would expect $\rho > 0$ – more study hours correlate with higher scores.

Recall: Univariate Normal Distribution

- A single random variable X has a normal distribution $\mathcal{N}(\mu, \sigma^2)$ if its pdf is

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

- The normal distribution is completely described by its mean μ and variance σ^2 .
- It is symmetric, bell-shaped, and appears everywhere thanks to the Central Limit Theorem.

From Univariate to Multivariate: Independent Case

- Suppose we have p independent random variables $X_1, \dots, X_p$, each normally distributed:

$$X_i \sim \mathcal{N}(\mu_i, \sigma_i^2).$$

- Because of independence, the joint pdf is the product of the individual pdfs:

$$f_{X_1, \dots, X_p}(x_1, \dots, x_p) = \prod_{i=1}^p \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right).$$

- This can be written more compactly in vector form.

Vector Notation for Independent Normals

- Define the vector $X = (X_1, \dots, X_p)^\top$, the mean vector $\mu = (\mu_1, \dots, \mu_p)^\top$, and the covariance matrix

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_p^2 \end{pmatrix}.$$

- Then the joint pdf becomes

$$f_X(x) = \frac{1}{(2\pi)^{p/2}(\det \Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right).$$

- This is exactly the multivariate normal density – for the special case of independent components.

The General Multivariate Normal Distribution

- For dependent normal variables, the covariance matrix Σ has non-zero off-diagonal entries.
- The same formula still holds:

$$f_X(x) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right).$$

- Here $|\Sigma|$ denotes the determinant of Σ , and Σ must be positive definite (so that the density is well-defined).
- The off-diagonal entries $\Sigma_{ij} = \text{Cov}(X_i, X_j)$ capture dependencies between variables.
- If Σ is diagonal, we recover the independent case.

Intuition: Shape of the Multivariate Normal

- For $p = 2$, the density looks like a bell-shaped surface above the (x_1, x_2) -plane.
- Contours of constant density are ellipses centered at μ .
- The orientation and shape of the ellipses are determined by Σ :
 - If $\sigma_1 = \sigma_2$ and $\text{Cov}(X_1, X_2) = 0$, the contours are circles.
 - If variances differ, the ellipses are stretched in the direction of larger variance.
 - Positive covariance tilts the ellipse along the line $x_1 = x_2$; negative covariance tilts the opposite way.
- The density is highest at the mean μ and decreases exponentially as we move away.

Key Property: Linear Transformations

- One of the most important features of the multivariate normal is its behaviour under linear transformations.
- Let $X \sim \mathcal{N}(\mu, \Sigma)$ and define $Y = AX + b$, where A is a matrix and b a vector (both of appropriate dimensions).
- Then Y is also multivariate normal:

$$Y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top).$$

- This is a direct consequence of the properties of expectation and covariance:

$$\mathbb{E}[Y] = A\mathbb{E}[X] + b = A\mu + b,$$

$$\text{Cov}(Y) = A \text{Cov}(X) A^\top = A\Sigma A^\top.$$

Example: Sum of Two Correlated Normals

- Suppose (X_1, X_2) is bivariate normal with mean (μ_1, μ_2) , variances σ_1^2, σ_2^2 , and covariance σ_{12} .
- Consider $Y = X_1 + X_2$. Write $Y = AX$ with $A = (1, 1)$.
- Then Y is univariate normal with

$$\mathbb{E}[Y] = \mu_1 + \mu_2,$$

$$\text{Var}(Y) = (1, 1)\Sigma(1, 1)^\top = \sigma_1^2 + \sigma_2^2 + 2\sigma_{12}.$$

- This matches an earlier formula for the variance of the sum of independent variables. Now we know: in general the variance of a sum includes a covariance term.

The Multivariate Central Limit Theorem

- In Chapter 1 we saw that the sample mean of i.i.d. univariate random variables is approximately normal for large n .
- The same idea extends to vectors: if $X_1, \dots, X_n$ are i.i.d. random vectors with mean μ and covariance Σ , then for large n ,

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \quad \text{is approximately} \quad \mathcal{N}\left(\mu, \frac{\Sigma}{n}\right).$$

- This is the **multivariate Central Limit Theorem**.
- It justifies using the multivariate normal distribution in many practical settings, even when the original data are not normal, provided the sample size is large enough.

Contents

- Multivariate Random Variables
- **Least-Squares Regression**
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

Can we predict exam scores from study habits?

- We have data from 10 students. For each student i we record:
 - h_i = hours studied before the exam,
 - t_i = number of practice tests taken,
 - b_i = final exam score (out of 100).

- We suspect an approximate **linear relationship**:

$$b_i = x_0 + x_1 h_i + x_2 t_i + \varepsilon_i,$$

- Here, x_0, x_1, x_2 are unknown coefficients (parameters),
- and ε_i models the error.
- Goal: estimate the coefficients and understand how well the model fits the data.

The Data

h_i (hours)	t_i (tests)	b_i (score)
2	1	72
3	2	78
4	3	84
5	4	88
6	5	91
3	1	70
4	2	80
5	3	85
6	4	89
7	5	94

We will use this dataset to illustrate all concepts.

Matrix Notation

- Stack the observations into a vector $b \in \mathbb{R}^{10}$ and a matrix $A \in \mathbb{R}^{10 \times 3}$:

$$b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_{10} \end{pmatrix}, \quad A = \begin{pmatrix} 1 & h_1 & t_1 \\ 1 & h_2 & t_2 \\ \vdots & \vdots & \vdots \\ 1 & h_{10} & t_{10} \end{pmatrix}.$$

- The parameter vector is $x = (x_0, x_1, x_2)^\top \in \mathbb{R}^3$.
- The error vector is $\varepsilon = (\varepsilon_1, \dots, \varepsilon_{10})^\top$.
- The model becomes:

$$b = Ax + \varepsilon.$$

The Idea of Least Squares

- We want to choose x so that Ax is as close as possible to the observed b .
- For a candidate x , the vector of residuals is $r = b - Ax$; its i -th component r_i is the error we would make for student i .
- A natural measure of overall misfit is the sum of squared residuals:

$$f(x) = \sum_{i=1}^{10} r_i^2 = \|b - Ax\|^2.$$

- This is called the **least squares objective**. It is a quadratic function of x , so we can find its minimizer analytically.

General Least Squares Problem

- In general, we have m observations and n parameters.
- This means A is a $m \times n$ matrix; $x \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$ are vectors.
- The least squares problem is:

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|^2.$$

- This is a convex quadratic optimization problem.
- No assumptions on the errors are needed yet. For now, we merely fit a linear model to data.
- We will discuss statistical assumptions later.

Deriving the Optimality Condition

- We introduce the **objective function**

$$f(x) = (b - Ax)^\top (b - Ax).$$

- Its gradient (vector of partial derivatives) is given by

$$\nabla f(x) = -2A^\top b + 2A^\top Ax.$$

- Setting $\nabla f(x) = 0$ gives the **normal equations**:

$$A^\top A x = A^\top b.$$

- This is a linear system for x .

Solving the Normal Equations

- If $A^T A$ is invertible ($= A$ has full rank), the unique solution is

$$\hat{x} = \underbrace{(A^T A)^{-1} A^T}_{=A^\dagger} b.$$

- $\hat{x}$ is called a least squares (LSQ) estimator.
- For our example, A is 10×3 . As long as the columns are linearly independent, a unique solution exists. In this case, the matrix

$$A^\dagger = (A^T A)^{-1} A^T$$

is a left inverse of A .

- We can compute $\hat{x}$ numerically once we have the data.

Geometric Interpretation

- The fitted values $\hat{b} = A\hat{x}$ are the orthogonal projection of b onto the column space of A .
- The corresponding projection matrix, $H = AA^\dagger$, satisfies

$$H^2 = (AA^\dagger)(AA^\dagger) = A(A^\dagger A)A^\dagger = AA^\dagger = H;$$

and we have $\hat{b} = Hb$.

- Residuals $r = b - \hat{b}$ satisfy $A^\top r = 0$ (orthogonal to all columns of A).
- This orthogonality is exactly the normal equations.

Back to Our Example...

- Construct A and b from the table:

$$b = \begin{pmatrix} 72 \\ 78 \\ 84 \\ 88 \\ 91 \\ 70 \\ 80 \\ 85 \\ 89 \\ 94 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 3 \\ 1 & 5 & 4 \\ 1 & 6 & 5 \\ 1 & 3 & 1 \\ 1 & 4 & 2 \\ 1 & 5 & 3 \\ 1 & 6 & 4 \\ 1 & 7 & 5 \end{pmatrix}.$$

- Compute $A^T A$ and $A^T b$, then solve $(A^T A)\hat{x} = A^T b$.

Back to Our Example...

- The result is

$$\hat{x} = \begin{pmatrix} 65.85 \\ 1 \\ 4.25 \end{pmatrix}.$$

- Interpretation: baseline score $\hat{x}_0 \approx 65.85$, each additional hour adds about 1 points, each practice test adds about 4.25 points.

Questions: Is this result plausible?

Can you predict your scores in the final exam based on that?

Contents

- Multivariate Random Variables
- Least-Squares Regression
- **Statistical Interpretation of Least Squares**
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

The Linear Model as an Assumption

- In the previous section we simply minimised $\|b - Ax\|^2$ to find a vector $\hat{x}$ that fits the data well.
- But if we want to make statements about the "true" underlying relationship, we need a **probability model**.
- We assume that the data were generated by

$$b = Ax^* + \varepsilon.$$

- Here, $x^* \in \mathbb{R}^n$ is the **true** (but unknown) parameter vector,
- and $\varepsilon \in \mathbb{R}^m$ is a random vector of errors.
- Additionally, the matrix A is known—we designed the experiment.
- **Caution:** we are postulating that the true relationship is exactly linear. In reality this is almost never true!

Assumptions on the Errors

- To make progress, we place assumptions on the error vector ε :
 1. **Zero mean:** $\mathbb{E}[\varepsilon] = 0$. This ensures that the model is correct on average.
 2. **Homoscedasticity and uncorrelatedness:** $\text{Var}(\varepsilon) = \sigma^2 I$, i.e. all errors have the same variance σ^2 and are uncorrelated.
 3. (Optional, for exact inference) **Normality:** $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$.
- These are strong assumptions. In practice, we have to check them.
- But: remember the no-free-lunch principle—without assumptions, we cannot quantify uncertainty.

Why Is $\hat{x}$ Random?

- The data b are random because they contain the random errors ε .
- Our estimator $\hat{x} = (A^\top A)^{-1} A^\top b$ is a linear function of b , hence it is also a random vector.
- Different samples would give different estimates.
- We now study the properties of $\hat{x}$ as an estimator of the true x^* .

Unbiasedness of the LSQ Estimator

- Let us first compute the expectation

$$\mathbb{E}[\hat{x}] = \mathbb{E}[(A^\top A)^{-1} A^\top b] = (A^\top A)^{-1} A^\top \mathbb{E}[b].$$

- From the model $b = Ax^* + \varepsilon$ and $\mathbb{E}[\varepsilon] = 0$, we have $\mathbb{E}[b] = Ax^*$.
- Hence, we have

$$\mathbb{E}[\hat{x}] = (A^\top A)^{-1} A^\top Ax^* = x^*.$$

- So $\hat{x}$ is an **unbiased estimator** of x^* : on average it hits the true value.

Variance of the LSQ Estimator

- The covariance matrix of $\hat{x}$ is derived using $\text{Var}(b) = \sigma^2 I$:

$$\text{Var}(\hat{x}) = \text{Var}((A^\top A)^{-1} A^\top b) = (A^\top A)^{-1} A^\top \text{Var}(b) A (A^\top A)^{-1}.$$

- Substitute $\text{Var}(b) = \sigma^2 I$:

$$\text{Var}(\hat{x}) = (A^\top A)^{-1} A^\top (\sigma^2 I) A (A^\top A)^{-1} = \sigma^2 (A^\top A)^{-1} (A^\top A) (A^\top A)^{-1}.$$

- Simplify:

$$\text{Var}(\hat{x}) = \sigma^2 (A^\top A)^{-1}.$$

- The diagonal entries give the variances of individual $\hat{x}_j$; off-diagonals give covariances between estimates.

The Gauss–Markov Theorem

- Consider any other linear unbiased estimator $\tilde{x} = Cb$ (with C such that $CA = I$ to ensure unbiasedness).
- Gauss–Markov theorem: The LSQ estimator $\hat{x}$ has the smallest variance among all linear unbiased estimators. (Proof: see next slide)
- In detail, $\text{Var}(\tilde{x}) - \text{Var}(\hat{x})$ is positive semidefinite – meaning every linear combination of $\tilde{x}$ has variance at least as large as that of $\hat{x}$.
- This is why LSQ is called **BLUE** (Best Linear Unbiased Estimator).
- The theorem holds under assumptions 1–2 only; normality is not required.

Sketch of the Gauss–Markov Proof

- Any linear unbiased estimator can be written as $\tilde{x} = \hat{x} + Db$ for some matrix D with $DA = 0$ (to preserve unbiasedness).
- Then $\text{Var}(\tilde{x}) = \text{Var}(\hat{x}) + \sigma^2 DD^\top$, because the cross-term vanishes.
- Since $DD^\top$ is positive semidefinite, $\text{Var}(\tilde{x}) - \text{Var}(\hat{x})$ is positive semidefinite.
- Hence $\hat{x}$ has minimum variance.
- This elegant proof shows that any deviation from $\hat{x}$ adds extra variance.

Estimating the Error Variance σ^2

- In practice σ^2 is unknown. We need to estimate it from the data.
- The residual sum of squares $\text{RSS} = \|b - A\hat{x}\|^2$ is a natural candidate.
- It can be shown that $\mathbb{E}[\text{RSS}] = (m - n)\sigma^2$. (see next slide for details)
- Therefore an **unbiased estimator** of σ^2 is

$$s^2 = \frac{\text{RSS}}{m - n}.$$

- This is the multivariate analogue of Bessel's correction from Lecture 1: we divide by $m - n$ (degrees of freedom) instead of m .
- Intuition: we have estimated n parameters, losing n degrees of freedom, so we average the squared residuals over the remaining $m - n$ independent pieces of information.

Formal Derivation

- First check that $\text{RSS} = \varepsilon^\top (I - H)\varepsilon$, with $H = A(A^\top A)^{-1}A^\top$.
- $I - H$ is a projection matrix with trace $m - n$ (its rank).
- For a vector ε with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2 I$, we have

$$\mathbb{E}[\text{RSS}] = \mathbb{E}[\varepsilon^\top (I - H)\varepsilon] = \sigma^2 \text{trace}(I - H) = \sigma^2(m - n).$$

- Hence $\mathbb{E}[\text{RSS}/(m - n)] = \sigma^2$.
- This is the generalisation of the scalar case where $\sum (x_i - \bar{x})^2$ had expectation $(n - 1)\sigma^2$.

Back to our Example...

- The data matrix A and observation vector b for our exam data were given on the slides.
- We first compute $A^\top A$ and $A^\top b$:

$$A^\top A = \begin{pmatrix} 10 & 45 & 30 \\ 45 & 225 & 155 \\ 30 & 155 & 110 \end{pmatrix}, \quad A^\top b = \begin{pmatrix} 831 \\ 3847 \\ 2598 \end{pmatrix}.$$

- Solving $(A^\top A)\hat{x} = A^\top b$ gives:

$$\hat{x} = \begin{pmatrix} \hat{x}_0 \\ \hat{x}_1 \\ \hat{x}_2 \end{pmatrix} = \begin{pmatrix} 65.85 \\ 1.0 \\ 4.25 \end{pmatrix}.$$

Computing Residuals and Estimating σ^2

- The fitted values $\hat{b} = A\hat{x}$ and residuals $r = b - \hat{b}$ can be used to compute the residual sum of squares,

$$\text{RSS} = \sum r_i^2 = 21.15.$$

- Unbiased estimate of error variance:

$$s^2 = \frac{\text{RSS}}{m - n} = \frac{21.15}{10 - 3} \approx 3.02.$$

- The estimated standard deviation of the errors is

$$s = \sqrt{3.02} \approx 1.74 \text{ points.}$$

Standard Errors of the Coefficients

- The estimated covariance matrix of $\hat{x}$ is $\widehat{\text{Var}}(\hat{x}) = s^2(A^\top A)^{-1}$.
- First compute $(A^\top A)^{-1}$ (we can invert the 3×3 matrix):

$$(A^\top A)^{-1} \approx \begin{pmatrix} 1.45 & -0.6 & 0.45 \\ -0.6 & 0.4 & -0.4 \\ 0.45 & -0.4 & 0.45 \end{pmatrix}.$$

- We multiply this by $s^2 \approx 3.02$ to find

$$\widehat{\text{Var}}(\hat{x}) \approx \begin{pmatrix} 4.38 & -1.81 & 1.35964 \\ -1.81 & 1.20 & -1.20857 \\ 1.35 & -1.20 & 1.35964 \end{pmatrix}.$$

- Standard errors (square roots of diagonal entries):

$$\text{se}(\hat{x}_0) \approx 2.1, \quad \text{se}(\hat{x}_1) \approx 1.1, \quad \text{se}(\hat{x}_2) \approx 1.2.$$

Standard Errors of the Coefficients

- In practice, we often use the following intuitive notation to summarize our result for the estimator:

$$\hat{x} = \begin{pmatrix} 65.85 \pm 2.1 \\ 1.0 \pm 1.1 \\ 4.25 \pm 1.2 \end{pmatrix},$$

listing estimated nominal values and estimated standard deviation.

Question: How would you interpret these results?

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- **Inference in Linear Regression**
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

Distribution of $\hat{x}$ under Normality

- If $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, then $\hat{x} \sim \mathcal{N}(x^*, \sigma^2 (A^\top A)^{-1})$.
- This allows us to construct confidence intervals and hypothesis tests.
- For a single coefficient $\hat{x}_j$, its standard error is $s\sqrt{[(A^\top A)^{-1}]_{jj}}$.
- The ratio $\frac{\hat{x}_j - x_j^*}{s\sqrt{[(A^\top A)^{-1}]_{jj}}}$ follows a t_{m-n} distribution.

Confidence Intervals and Hypothesis Tests

- A 95% confidence interval for x_j^* is

$$\hat{x}_j \pm t_{0.025, m-n} s \sqrt{[(A^\top A)^{-1}]_{jj}}.$$

- To test $H_0 : x_j^* = 0$ (predictor has no effect), compute

$$t = \frac{\hat{x}_j}{s \sqrt{[(A^\top A)^{-1}]_{jj}}} \text{ and compare to } t_{\alpha/2, m-n}.$$

- For our exam data, we could test whether practice tests (x_2) significantly improve the model after accounting for study hours.

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- **Model Selection and the Bias–Variance Trade-off**
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

The Problem: How Complex Should Our Model Be?

- In our exam score example, we used two predictors:
hours and practice tests.
- But what if the true relationship is not linear? Perhaps the benefit of extra study hours diminishes (a curved relationship).
- We could add a quadratic term: hours^2 , or even higher powers.
- More complex models (more parameters) will always fit the **training data** better – they can bend and wiggle to match every point.
- But the real goal is to predict **new, unseen data**. A model that is too complex may overfit: it learns the noise, not the signal.
- This is the fundamental tension in model building: we must choose the right level of complexity.

The Bias–Variance Trade-off

- Imagine we could repeatedly sample new datasets from the same population and fit the same model each time.
- **Bias** is the systematic error – how far the average prediction (over many datasets) is from the true value.
- **Variance** is how much the predictions fluctuate between datasets.
- Simple models (e.g., linear) tend to have:
 - High bias: they may miss important patterns (underfitting).
 - Low variance: they are stable across samples.
- Complex models (e.g., high-degree polynomials) tend to have:
 - Low bias: they can capture intricate patterns.
 - High variance: small changes in the data can cause big changes in the model (overfitting).
- Total prediction error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.

Illustration: Polynomial Fit to Study Hours

- To see this clearly, let's temporarily ignore practice tests and model exam score as a polynomial of degree d in study hours alone.
- We have 10 data points; we'll fit polynomials of degree $d = 1, 2, 3, \dots, 9$.

d	Description	Behaviour
1	Linear	Smooth, may miss curvature (high bias)
2	Quadratic	Can capture one bend
3	Cubic	More flexible
$\vdots$		
9	Degree 9	Can pass exactly through all 10 points

What Happens as Degree Increases?

- For $d = 1$ (linear): the fit is a straight line. It might systematically underpredict at low and high hours – that's bias.
- For $d = 2$ (quadratic): the line can curve. Fit improves, bias decreases.
- As d increases further, the curve starts to wiggle more to pass near every point.
- For $d = 9$, the polynomial can interpolate exactly – it goes through every data point. Training error is zero.
- But if we test on new data, the $d = 9$ model will likely perform terribly – it has learned the noise, not the true pattern. Its variance is huge.

Measuring Fit: Total Sum of Squares

- To understand how well our model fits, we compare it to the simplest possible model: just the mean.
- **Total Sum of Squares (TSS)** measures the total variability,

$$\text{TSS} = \sum_{i=1}^m (b_i - \bar{b})^2,$$

where $\bar{b} = \frac{1}{m} \sum_{i=1}^m b_i$ is the overall mean.

- In contrast, the **Residual Sum of Squares (RSS)** measures the variability left after using our model:

$$\text{RSS} = \sum_{i=1}^m (b_i - \hat{b}_i)^2 = \|b - A\hat{x}\|^2.$$

- RSS is always smaller than TSS (unless the model is useless).

Coefficient of Determination

- R^2 (pronounced "R-squared") is defined as:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}.$$

- Interpretation: the proportion of variance in b explained by the model.
- If the model fits perfectly, $\text{RSS} = 0$ and $R^2 = 1$.
- If the model is no better than the mean, $\text{RSS} \approx \text{TSS}$ and $R^2 \approx 0$.

The Problem with Ordinary R^2

- Adding any extra predictor to the model (even a completely random one) will never increase RSS – it will either stay the same or decrease.
- Therefore, R^2 can only increase (or stay the same) when we add more terms.
- A quadratic term (hours²) will always give a higher R^2 than a purely linear model, even if the true relationship is linear.
- If we add enough terms, we can drive R^2 arbitrarily close to 1 – but the model becomes overfitted and predicts new data poorly.
- Ordinary R^2 does not penalize complexity. It will always prefer the most complex model.

Adjusted R^2 : Penalizing Complexity

- **Adjusted R^2** modifies ordinary R^2 to

$$R_{\text{adj}}^2 = 1 - \frac{\text{RSS}/(m - n)}{\text{TSS}/(m - 1)}, \quad \text{where}$$

m = “number of observations” and “ n = number of parameters”.

- The denominator $\text{TSS}/(m - 1)$ is the total variance estimate, which uses $m - 1$ degrees of freedom.
- The numerator $\text{RSS}/(m - n)$ is the residual variance estimate – this is exactly our unbiased estimator s^2 from earlier.
- If we add a useless predictor, RSS decreases slightly, but $m - n$ also decreases (because n increases). The ratio $\text{RSS}/(m - n)$ may increase, causing R_{adj}^2 to drop.

Interpreting Adjusted R^2

- Adjusted R^2 can be thought of as an estimate of how much variance the model would explain in new data (though it's still optimistic).
- Unlike ordinary R^2 , adjusted R^2 can decrease when we add an irrelevant predictor.
- For model selection (e.g., choosing polynomial degree), we can compute R^2_{adj} for each candidate model and pick the one with the highest value.
- However, adjusted R^2 is still only a heuristic – it doesn't directly measure prediction performance on truly new data. Cross-validation is more reliable but computationally heavier.
- In practice, both adjusted R^2 and cross-validation are useful tools; they often give similar guidance.

A More Direct Approach: Cross-Validation

- Idea: mimic how well the model will predict new data by testing it on data not used for fitting.
- **Leave-one-out cross-validation (LOOCV):**
 1. Remove one data point.
 2. Fit the model on the remaining $m - 1$ points.
 3. Predict the held-out point and record the squared error.
 4. Repeat for every point, average the errors.
- This gives a nearly unbiased estimate of prediction error.
- For small datasets like ours ($m = 10$), LOOCV is feasible. For larger datasets, we often use k -fold cross-validation (e.g., split data into 5 or 10 folds).

Applying Cross-Validation to the Polynomial Example

- For each degree $d = 1, 2, \dots, 8$, we would:
 - For $i = 1$ to 10:
 - Remove student i from the data.
 - Fit a degree- d polynomial to the remaining 9 points.
 - Predict score for student i , compute $(b_i - \hat{b}_i)^2$.
 - Average the 10 squared errors $\rightarrow$ CV error for degree d .
- We would then choose the degree d with the smallest CV error.
- Typically, a low-degree model (e.g., $d = 1$ or 2) wins – the extra wiggles of high-degree polynomials hurt prediction.

Bias–Variance Trade-off in Our Full Model

- Returning to our original two-predictor model, we might ask:
 - Should we add an interaction term (hours $\times$ practice tests)?
 - Should we add quadratic terms (hours², tests²)?
- Adding these terms would reduce bias (if the true relationship is curved/interactive) but increase variance.
- With only 10 data points, adding many extra terms could lead to severe overfitting.
- Cross-validation or adjusted R^2 can guide us: compute these measures for the model with and without the extra terms, and choose the model that predicts best.

Key Takeaway: The Art of Modeling

- There is no single "correct" model – only models that are more or less useful for prediction or explanation.
- The bias–variance trade-off is a fundamental principle: increasing model complexity reduces bias but increases variance.
- We must choose complexity to minimise total prediction error on new data. In practice, this is often not easy.
- Tools like adjusted R^2 and cross-validation help us make this choice.
- In practice, start simple, check residuals, and only add complexity if it clearly improves predictive performance.
- For our exam data, a simple linear model with hours and practice tests already gives interpretable coefficients. Adding complexity would likely overfit with only 10 observations.

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- **Weighted Least Squares and Extensions**
- Rank Deficiency and Regularization
- Summary

When Errors Are Not Homoscedastic

- Recall our assumption $\text{Var}(\varepsilon) = \sigma^2 I$: all errors have the same variance (homoscedastic) and are uncorrelated.
- In many real situations, this fails. For example:
 - Measurements may have different known accuracies.
 - Errors might be correlated over time.
- Suppose instead $\text{Var}(\varepsilon) = \sigma^2 V$, where V is a known positive definite matrix.
- V describes the structure: diagonal entries give relative variances, off-diagonals give correlations.

The Weighted Least Squares (WLS) Idea

- If errors have different variances, observations with smaller variance (more precise) should count more in the fit.
- We want to minimise a **weighted** sum of squares:

$$f_V(x) = (b - Ax)^\top V^{-1}(b - Ax).$$

- The matrix V^{-1} gives less weight to high-variance observations.
- This is called the **weighted least squares** (WLS) problem.

Solution of the WLS Problem

- Expanding $f_V(x)$ gives a quadratic form; its gradient is

$$\nabla f_V(x) = -2A^\top V^{-1}b + 2A^\top V^{-1}Ax.$$

- Setting to zero yields the **weighted normal equations**:

$$A^\top V^{-1}Ax = A^\top V^{-1}b.$$

- If A has full rank, the unique solution is

$$\hat{x}_V = (A^\top V^{-1}A)^{-1}A^\top V^{-1}b.$$

- This is the WLS estimator.

Connection to Ordinary Least Squares

- Observe that V^{-1} is positive definite, so we can factor it as $V^{-1} = C^{\top} C$ (e.g. Cholesky decomposition).
- Define transformed variables:

$$\tilde{b} = Cb, \quad \tilde{A} = CA.$$

- Then the WLS objective becomes

$$(b - Ax)^{\top} V^{-1} (b - Ax) = (Cb - CAx)^{\top} (Cb - CAx) = \|\tilde{b} - \tilde{A}x\|^2.$$

- This is an ordinary LSQ problem in the transformed data!
- Therefore, WLS is equivalent to:
 1. Transform the data using C (a square root of V^{-1}).
 2. Perform ordinary least squares on the transformed data.

Properties of the WLS Estimator

- Under the model $b = Ax^* + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2 V$:
 - $\hat{x}_V$ is **unbiased**: $\mathbb{E}[\hat{x}_V] = x^*$.
 - Its covariance matrix is $\text{Var}(\hat{x}_V) = \sigma^2 (A^\top V^{-1} A)^{-1}$.
- The Gauss–Markov theorem generalises: among all linear unbiased estimators, $\hat{x}_V$ has the smallest variance (it is BLUE for this error structure).
- In practice, V is rarely known exactly. Often we assume a simple structure (e.g. V diagonal with known relative variances) and estimate σ^2 from the residuals.

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- Summary

What If A Does Not Have Full Rank?

- If predictors are linearly dependent, $A^T A$ is singular.
- Then there are infinitely many solutions, all giving the same fitted values.
- In this case, regularization (replace $A^T A$ by $A^T A + \tau I$) can stabilize estimates. This introduces bias to reduce variance.
- Alternatively, one can use the Moore–Penrose pseudoinverse,

$$A^\dagger = \lim_{\tau \rightarrow 0^+} (A^T A + \tau I)^{-1} A^T,$$

to get the minimum-norm solution.

Contents

- Multivariate Random Variables
- Least-Squares Regression
- Statistical Interpretation of Least Squares
- Inference in Linear Regression
- Model Selection and the Bias–Variance Trade-off
- Weighted Least Squares and Extensions
- Rank Deficiency and Regularization
- **Summary**

What We Learned

- Multivariate random variables: mean vector, covariance matrix, multivariate normal.
- Linear model $b = Ax + \varepsilon$, assumptions needed for inference.
- LSQ: normal equations, geometric interpretation, unbiasedness, Gauss–Markov.
- Inference: t-tests, confidence intervals.
- Model selection and the bias–variance trade-off.
- Weighted least squares problems.
- Rank deficiency and the pseudoinverse.

Chapter 3

Linear Programming Models

Linear Programming

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

Optimization in Everyday Life

- We often face decisions: how many hours to study, which route to take, how much to produce.
- We want the **best** possible outcome.
For example: maximize profit, minimize time, reduce pollution, ...
- This "best" is subject to **limitations**: time, money, resources.
- An optimization problem captures this mathematically.

What Goes Into an Optimization Problem?

- **Decision variables:** the quantities we can choose.
 - Example: how many units of each product to produce.
 - We denote them as $x = (x_1, x_2, \dots, x_n)$.
- **Objective function:** what we want to achieve.
 - Maximize profit, minimize cost, minimize time, etc.
 - It is a function $f_0(x)$ of the decision variables.
- **Constraints:** limitations or requirements.
 - Inequality constraints: $f_i(x) \leq 0$ (Example: resource limits).
 - Equality constraints: $h_j(x) = 0$ (Example: exact balance equations).

The Full Problem Formulation

- We write an optimization problem in a standard way:

$$\begin{array}{ll}\text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p,\end{array}$$

where $x \in \mathbb{R}^n$ is the vector of decision variables.

- "Minimize" can be replaced by "maximize" if we flip the sign of f_0 .
- This notation is compact and used everywhere in optimization.

Feasible Set and Optimal Value

- A point x is called **feasible** if it satisfies *all* constraints.
- The set of all feasible points is the **feasible set** (or feasible region):

$$\mathcal{F} = \left\{ x \in \mathbb{R}^n \left| \begin{array}{l} \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, p\}, \\ f_i(x) \leq 0, h_j(x) = 0 \end{array} \right. \right\}.$$

- The **optimal value** p^* is the smallest value of f_0 over the feasible set:

$$p^* = \inf_{x \in \mathcal{F}} f_0(x).$$

- If there exists $x^* \in \mathcal{F}$ such that $f_0(x^*) = p^*$, then x^* is an **optimal solution** (or minimizer).
- Not all problems have an optimal solution. For example, the feasible set might be empty, or the objective could be unbounded below.

Guiding Example: A Production Planning Model

- A factory produces two products. Let x_1 = number of units of product A, x_2 = units of product B.
- Selling price per unit: product A \$13, product B \$17.
- Production cost (materials, labor costs, energy costs, ...) per unit: product A \$10, product B \$12.
- Therefore, **net profit per unit**: product A \$3, product B \$5.
- Labor: each unit of A requires 1 hour, each unit of B requires 2 hours. Total available: 100 hours.
- Material: each unit of A requires 3 kg, each unit of B requires 1 kg. Total available: 120 kg.
- Non-negativity: $x_1 \geq 0$, $x_2 \geq 0$.
- **Question:** How to choose x_1, x_2 to maximize total net profit?

Mathematical Formulation

- **Decision variables:** x_1, x_2 (production quantities).
- **Objective:** maximize total net profit

$$3x_1 + 5x_2.$$

- **Constraints:**

$$x_1 + 2x_2 \leq 100 \quad (\text{labor hours})$$

$$3x_1 + x_2 \leq 120 \quad (\text{material kg})$$

$$x_1, x_2 \geq 0 \quad (\text{non-negativity}).$$

- This is a **linear program** (LP) because:
 - The objective is a linear function of x_1, x_2 .
 - All constraints are linear inequalities.
 - The variables are continuous (can take any real value).

Contents

- What is an Optimization Problem?
- **Linear Programming Formulation**
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

Standard Form of a Linear Program

- A linear program (LP) in standard form is:

$$\begin{array}{ll}\text{minimize} & c^\top x \\ \text{subject to} & Ax = b \\ & x \geq 0,\end{array}$$

where $x \in \mathbb{R}^n$, $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

- "Minimize" can be changed to "maximize" by flipping the sign of c .
- Any LP can be transformed to this form (see next slides).

Transforming to Standard Form

- Recall our original problem (maximize profit):

$$\text{maximize} \quad 3x_1 + 5x_2$$

$$\text{subject to} \quad x_1 + 2x_2 \leq 100$$

$$3x_1 + x_2 \leq 120$$

$$x_1 \geq 0, \quad x_2 \geq 0.$$

- Standard form requires:
 - Minimization (not maximization).
 - Only equality constraints.
 - All variables non-negative.
- We need to transform our problem accordingly.

Transforming to Standard Form

- **Step 1: Convert maximization to minimization.**

- Maximizing $3x_1 + 5x_2$ is equivalent to minimizing $-3x_1 - 5x_2$.
- The optimal x_1, x_2 will be the same; only the objective value changes.

- **Step 2: Turn inequalities into equalities.**

- For each $\leq$ constraint, add a non-negative **slack variable**.
- Slack variable represents unused resources.

- For labor: $x_1 + 2x_2 \leq 100$ becomes

$$x_1 + 2x_2 + s_1 = 100, \quad s_1 \geq 0.$$

- For material: $3x_1 + x_2 \leq 120$ becomes

$$3x_1 + x_2 + s_2 = 120, \quad s_2 \geq 0.$$

Transforming to Standard Form

- Now we have a new set of variables:

$$x = (x_1, x_2, s_1, s_2)^T \in \mathbb{R}^4.$$

- All variables are non-negative: $x_1 \geq 0, x_2 \geq 0, s_1 \geq 0, s_2 \geq 0$.
- The constraints are now equalities:

$$\begin{array}{cccccccl} x_1 & + & 2x_2 & + & s_1 & & = & 100 \\ 3x_1 & + & x_2 & & & + & s_2 & = & 120 \end{array}$$

Transforming to Standard Form

- Write the system in matrix form $Ax = b$:

$$A = \begin{pmatrix} 1 & 2 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 100 \\ 120 \end{pmatrix}.$$

- The cost vector c (for minimization) is:

$$c = (-3, -5, 0, 0)^{\top}.$$

- Our LP in standard form is:

$$\begin{array}{ll} \text{minimize} & c^{\top} x \\ \text{subject to} & Ax = b \\ & x \geq 0. \end{array}$$

- This is now ready for linear programming algorithms.

Software Tools for Optimization

- In this lecture, we transformed our production problem to standard form by hand – adding slack variables, converting to minimization.
- This was educational, but in practice, **modeling languages** do this automatically.
- You describe the problem naturally (maximize profit, subject to constraints), and the software:
 - Adds slack variables automatically.
 - Converts maximization to minimization.
 - Calls a solver (simplex, interior-point) and returns the solution.

Popular Modeling Tools – A Quick Look

- YALMIP, CVX, CVXPY, and many other tools:
 - Let you write constraints like $x_1 + 2x_2 \leq 100$ directly.
 - Automatically handle transformations to standard form.
 - Interface with solvers like Gurobi, MOSEK, SeDuMi.
- The theory you learn helps you:
 - Formulate problems correctly.
 - Interpret solver output (shadow prices, etc.).

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- **Geometry of Linear Programs**
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

Polyhedra: The Feasible Set of an LP

- In an LP, each constraint is a linear inequality $a^\top x \leq b$ or equality $a^\top x = b$ (with $a \neq 0$).
- A linear inequality $a^\top x \leq b$ defines a **halfspace** – all points on one side of a line (in 2D) or a hyperplane (in higher dimensions).
- A linear equality $a^\top x = b$ defines a **hyperplane** – a flat surface of dimension $n - 1$.
- The feasible set of an LP is the intersection of finitely many halfspaces and hyperplanes.
- Such a set is called a **polyhedron**.
- A **polytope** is a bounded polyhedron (finite in extent).

Our Example as a Polyhedron

- Recall our production planning problem (before adding slacks):

$$x_1 + 2x_2 \leq 100 \quad (\text{labor})$$

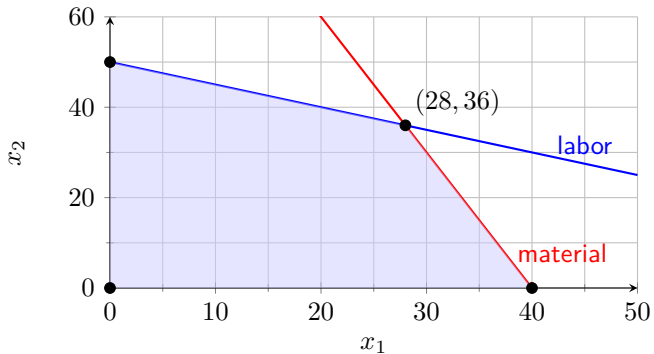
$$3x_1 + x_2 \leq 120 \quad (\text{material})$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

- Each inequality defines a halfspace. The feasible set is the intersection of these four halfspaces.

Feasible Region of Our Example



Vertices (Extreme Points)

- A point in a convex set is called a **vertex** (or **extreme point**) if it cannot be written as a convex combination of two other distinct points in the set.
- In a polyhedron, vertices are the "corners" where enough constraints are active (hold with equality) to determine the point uniquely.
- For a polyhedron in $\mathbb{R}^n$, a vertex must have at least n linearly independent active constraints.

Faces of a Polyhedron

- A **face** of a polyhedron is the intersection of the polyhedron with a supporting hyperplane – a hyperplane that touches the polyhedron without cutting through it.
- Faces are themselves polyhedra of lower dimension.
- Classification by dimension:
 - **Vertices** are 0-dimensional faces.
 - **Edges** are 1-dimensional faces.
 - **Facets** are $(n - 1)$ -dimensional faces (the "sides" of the polyhedron).

Active Set and Active Constraint Matrix

Definition

For a feasible point x of an LP with constraints $a_i^\top x \leq b_i$ ($i = 1, \dots, m$), the **active set** is

$$\mathcal{A}(x) = \{i \mid a_i^\top x = b_i\}.$$

The **active constraint matrix** A_{act} consists of the rows $a_i^\top$ for $i \in \mathcal{A}(x)$, and b_{act} is the corresponding vector of right-hand sides. At x , we have $A_{\text{act}}x = b_{\text{act}}$.

Example

In our example (2D):

- Vertex $(0, 0)$: active constraints $x_1 \geq 0$ and $x_2 \geq 0$.
- Vertex $(0, 50)$: active constraints $x_1 \geq 0$ and $x_1 + 2x_2 \leq 100$.
- Vertex $(40, 0)$: active constraints $x_2 \geq 0$ and $3x_1 + x_2 \leq 120$.
- Vertex $(28, 36)$: active constraints $x_1 + 2x_2 \leq 100$ and $3x_1 + x_2 \leq 120$.
- Edges: the four line segments connecting these vertices.
- Facets: the entire edges (since in 2D, facets are 1D faces).

Evaluating Profit at Each Vertex

- Profit function $P = 3x_1 + 5x_2$.
- Vertex A $(0, 0)$: $P = 0$.
- Vertex B $(0, 50)$: $P = 3 \cdot 0 + 5 \cdot 50 = 250$.
- Vertex C $(40, 0)$: $P = 3 \cdot 40 + 5 \cdot 0 = 120$.
- Vertex D $(28, 36)$: $P = 3 \cdot 28 + 5 \cdot 36 = 84 + 180 = 264$.
- The maximum profit is 264 at $(28, 36)$.

Fundamental Theorem of Linear Programming

Theorem.

If a linear program has a nonempty bounded feasible set and an optimal solution exists, then there exists an optimal solution at a vertex of the feasible polyhedron.

- This theorem tells us we only need to check vertices – not every point in the feasible set.
- It drastically simplifies the search for an optimum.
- The simplex method exploits this by moving from vertex to vertex.
- For our example, the optimum is at vertex $(28, 36)$ with profit 264.
- A proof of this theorem is developed over the next few slides...

Recall: Active Constraints and Vertices

- At a feasible point x , a constraint is **active** if it holds with equality.
- In $\mathbb{R}^n$, a point is a **vertex** if it has at least n linearly independent active constraints.
- Example: Our vertices have two active constraints each.
- If a point is *not* a vertex, the set of active constraints has rank less than n .
- Then there exists a nonzero direction d that is orthogonal to all active constraint gradients (that is, it lies in the nullspace of the active constraint matrix).

Key Lemma: Existence of a Feasible Direction

Lemma.

Let x be a feasible point that is not a vertex. Then there exists a nonzero vector d such that for all sufficiently small $\varepsilon > 0$, both $x + \varepsilon d$ and $x - \varepsilon d$ are feasible.

Proof.

Let A_{act} be the matrix of active constraint rows at x (including equality constraints). Since x is not a vertex, A_{act} has rank $< n$. Hence there exists $d \neq 0$ with $A_{\text{act}}d = 0$. For inactive constraints, $a_i^\top x < b_i$, so for small ε , $a_i^\top (x \pm \varepsilon d) \leq b_i$ still holds. Thus $x \pm \varepsilon d$ remains feasible. $\square$

Optimality Implies Constant Objective Along d

- Let x^* be an optimal solution. If x^* is not a vertex, pick d as in the lemma.
- For small ε , $x^* \pm \varepsilon d$ are feasible.
- The objective value at these points is $c^\top x^* \pm \varepsilon c^\top d$.
- Since x^* is optimal, we cannot have $c^\top d > 0$ (otherwise $x^* + \varepsilon d$ would be better) nor $c^\top d < 0$ (otherwise $x^* - \varepsilon d$ would be better).
- Therefore $c^\top d = 0$. So moving along d does not change the objective.

Moving to a New Constraint

- Now consider moving from x^* in direction $+d$ (or $-d$). Since the feasible set is bounded, we cannot go arbitrarily far.
- Let $\varepsilon^* > 0$ be the largest ε such that $x^* + \varepsilon d$ remains feasible.
- At $\varepsilon = \varepsilon^*$, at least one new constraint becomes active (otherwise we could go further).
- Denote $x' = x^* + \varepsilon^* d$. Then:
 - x' is feasible.
 - $c^\top x' = c^\top x^*$ (since $c^\top d = 0$).
 - x' has at least one more active constraint than x^* .

Reaching a Vertex

- Starting from any optimal solution x^* , if it is not a vertex, we can construct a new optimal solution x' with strictly more active constraints.
- Since there are only finitely many constraints, after a finite number of steps we must reach a point where no further increase in active constraints is possible – that is, a vertex.
- This vertex is also optimal.
- Thus, there always exists an optimal solution at a vertex.
- This completes our proof. □

Applying the Proof to Our Example

- In our production example, the feasible set is bounded.
- The optimal point $(28, 36)$ is actually a vertex, so the theorem applies directly.
- But suppose the optimal point were somewhere along the edge between $(28, 36)$ and $(40, 0)$ – say $(34, 18)$. Could that happen?
- According to the proof, if such an interior point were optimal, then both endpoints $(28, 36)$ and $(40, 0)$ would have the same objective value. Let's check:

$$3 \cdot 28 + 5 \cdot 36 = 84 + 180 = 264, \quad 3 \cdot 40 + 5 \cdot 0 = 120.$$

They are different, so no interior point on that edge can be optimal.

- This illustrates the power of the theorem.

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- **Duality in Linear Programming**
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

The Idea of Duality

- Every linear program has a twin problem called the **dual**.
- The dual provides:
 - Lower bounds on the optimal value (for minimization).
 - Economic interpretation (shadow prices).
 - A way to check optimality.
- The original LP is called the **primal**.
- Duality reveals a beautiful symmetry in linear programming.

Constructing a Lower Bound

- Recall that our primal linear program is the minimization problem

$$(P) \quad V^* = \min_x c^\top x \quad \{ x \geq 0, Ax = b \}$$

- For the equality constraints $Ax = b$, for any $\lambda \in \mathbb{R}^m$ we have

$$\lambda^\top (b - Ax) = 0 \quad \text{for any feasible } x.$$

- For the nonnegativity constraints $x \geq 0$, for any $\mu \geq 0$ we have

$$-\mu^\top x \leq 0 \quad \text{for any feasible } x.$$

- Adding these to the objective gives the Lagrangian lower bound

$$L(x, \lambda, \mu) \stackrel{\text{def}}{=} c^\top x + \lambda^\top (b - Ax) - \mu^\top x \leq c^\top x \quad \text{for any feasible } x.$$

Maximizing the Lower Bound

- To get a lower bound valid for *all* feasible x , we first take the minimum over x :

$$g(\lambda, \mu) \stackrel{\text{def}}{=} \min_x L(x, \lambda, \mu) \leq V^*.$$

By construction, this inequality holds for any λ, μ with $\mu \geq 0$.

- However, since we have

$$L(x, \lambda, \mu) = b^\top \lambda + x^\top (c - A^\top \lambda - \mu),$$

it turns out that $g(\lambda, \mu) \neq -\infty$ only if $c - A^\top \lambda - \mu = 0$.

- Moreover, if $c - A^\top \lambda - \mu = 0$, then $g(\lambda, \mu) = b^\top \lambda$.
- Hence, the best possible lower bound is given by

$$V^* \geq \max_{\lambda, \mu} b^\top \lambda \quad \text{s.t.} \quad \{ \mu \geq 0, c - A^\top \lambda - \mu = 0 \}.$$

The Dual Problem

- After eliminating μ , using $0 \leq \mu = c - A^\top \lambda$, the best possible lower bound can be found by solving the dual problem

$$\begin{array}{ll} \max_{\lambda} & b^\top \lambda \\ \text{s.t.} & A^\top \lambda \leq c \end{array}$$

- Theorem (Weak Duality):** For any feasible primal x and any feasible dual λ , we have $c^\top x \geq b^\top \lambda$.

The Primal-Dual Pair Summary

Primal (P)	Dual (D)
$\min_x c^\top x$	$\max_\lambda b^\top \lambda$
s.t. $Ax = b$	s.t. $A^\top \lambda \leq c$
$x \geq 0$	λ unrestricted

- Notice the beautiful symmetry:
 - Primal constraints $Ax = b$ give dual variables λ unrestricted.
 - Primal variables $x \geq 0$ give dual constraints $A^\top \lambda \leq c$.
 - Objective coefficients swap: c and b exchange places.
- This is the standard primal-dual pair for LPs.
- Due to the symmetry, the dual of the dual problem is again equivalent to the primal problem.

Applying Duality to Our Production Example

- Recall our primal in standard form (minimization):

$$\begin{array}{ll}\min_x & -3x_1 - 5x_2 \\ \text{s.t.} & x_1 + 2x_2 + s_1 = 100 \\ & 3x_1 + x_2 + s_2 = 120 \\ & x_1, x_2, s_1, s_2 \geq 0.\end{array}$$

- Hence, we have

$$A = \begin{pmatrix} 1 & 2 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 100 \\ 120 \end{pmatrix}, \quad c = \begin{pmatrix} -3 & -5 & 0 & 0 \end{pmatrix}^{\top}.$$

Writing the Dual Constraints

- Compute $A^T \lambda$:

$$A^T \lambda = \begin{pmatrix} 1 & 3 \\ 2 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} \lambda_1 + 3\lambda_2 \\ 2\lambda_1 + \lambda_2 \\ \lambda_1 \\ \lambda_2 \end{pmatrix}.$$

- The constraints $A^T \lambda \leq c$ become:

$$\lambda_1 + 3\lambda_2 \leq -3$$

$$2\lambda_1 + \lambda_2 \leq -5$$

$$\lambda_1 \leq 0$$

$$\lambda_2 \leq 0.$$

- So λ_1 and λ_2 are both non-positive.

Simplifying with Nonnegative Variables

- Since $\lambda_1 \leq 0$ and $\lambda_2 \leq 0$, define nonnegative variables:

$$y_1 = -\lambda_1 \geq 0, \quad y_2 = -\lambda_2 \geq 0.$$

- Substituting $\lambda_1 = -y_1$, $\lambda_2 = -y_2$:

$$\lambda_1 + 3\lambda_2 = -y_1 - 3y_2 \leq -3 \implies y_1 + 3y_2 \geq 3,$$

$$2\lambda_1 + \lambda_2 = -2y_1 - y_2 \leq -5 \implies 2y_1 + y_2 \geq 5.$$

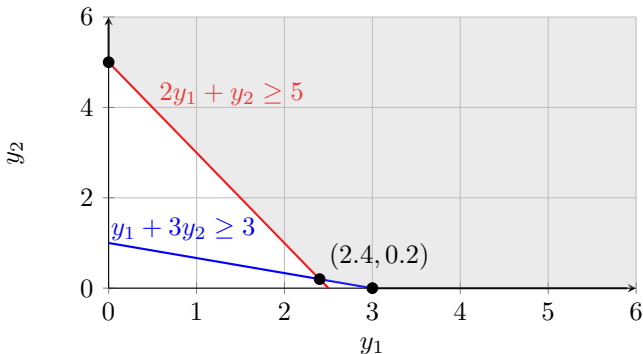
- The nonnegativity constraints become $y_1 \geq 0$, $y_2 \geq 0$.
- Objective: $100\lambda_1 + 120\lambda_2 = -100y_1 - 120y_2$.
- Maximizing this is equivalent to minimizing $100y_1 + 120y_2$.

The Dual in Clean Form

Primal (P)	Dual (D)
$\min -3x_1 - 5x_2$	$\min 100y_1 + 120y_2$
s.t.	s.t.
$x_1 + 2x_2 + s_1 = 100$	$y_1 + 3y_2 \geq 3$
$3x_1 + x_2 + s_2 = 120$	$2y_1 + y_2 \geq 5$
$x_1, x_2, s_1, s_2 \geq 0$	$y_1, y_2 \geq 0$

- The dual variables y_1, y_2 are the **shadow prices** – marginal value of labor and material.

Solving the Dual Graphically



- Vertex A $(0, 5)$: dual value $D = 120 \times 5 = 600$.
- Vertex B $(2.4, 0.2)$: dual value $D = 100 \times 2.4 + 120 \times 0.2 = 264$.
- Vertex C $(3, 0)$: dual value $D = 100 \times 3 = 300$.
- Optimal dual solution: $y_1^* = 2.4$, $y_2^* = 0.2$, optimal value 264.

Interpreting the Shadow Prices

- $y_1^* = 2.4$: One additional hour of labor increases profit by \$2.40.
- $y_2^* = 0.2$: One additional kg of material increases profit by \$0.20.
- Why is labor more valuable?

- At the optimum (28, 36), both constraints are strongly active,

$$28 + 2 \cdot 36 = 100, \quad 3 \cdot 28 + 36 = 120.$$

- But the profit contribution of each product differs:
 - Product A: profit \$3, uses 1h labor, 3kg material.
 - Product B: profit \$5, uses 2h labor, 1kg material.
- Labor is the scarcer resource in terms of profit generation – hence it has a higher shadow price.
- Shadow prices help us to decide what we would be willing to pay for an extra unit of a resource.

Strong Duality

Theorem (Strong Duality).

If the primal LP has an optimal solution x^* , then the dual also has an optimal solution λ^* , and

$$c^\top x^* = b^\top \lambda^*.$$

Remarks:

- This holds for our example: both optimal values are equal to 264.
- The proof is not difficult but technical. This is not part of this course, but it can be found in all textbooks on LPs or Convex Optimization.
- It is the foundation for many algorithms and economic interpretations.

Duality as a Diagnostic Tool

- The relationship between primal and dual can even give us information about the problem status even if no solution exists. In general:

Primal status	Dual status
Optimal	Optimal
Unbounded	Infeasible
Infeasible	Unbounded or infeasible

- A formal proof of this status table relies on Farkas' lemma, which is not discussed here. It can be found in the literature.

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- **KKT Optimality Conditions**
- Algorithms for Linear Programming
- Extensions of Linear Programming
- Summary

Recall: Strong Duality

- For our primal (P) and dual (D), strong duality tells us:

If x^* optimal for (P) and λ^* optimal for (D), then $c^\top x^* = b^\top \lambda^*$.

- Primal feasibility means $Ax^* = b$ and $x^* \geq 0$.
- Dual feasibility means $A^\top \lambda^* \leq c$.
- We'll use this equality to derive so called Karush-Kuhn Tucker conditions that characterize optimality for (LPs).

Step 1: Rewrite the Strong Duality Equality

- Start with $c^\top x^* = b^\top \lambda^*$.
- Since x^* is primal feasible, $Ax^* = b$. Therefore:

$$b^\top \lambda^* = (Ax^*)^\top \lambda^* = (x^*)^\top A^\top \lambda^*.$$

- Substitute back:

$$c^\top x^* = (x^*)^\top A^\top \lambda^*.$$

- Bring all terms to one side:

$$c^\top x^* - (x^*)^\top A^\top \lambda^* = 0.$$

- Factor x^* (note: $c^\top x^* = (x^*)^\top c$ since it's a scalar):

$$(x^*)^\top (c - A^\top \lambda^*) = 0.$$

Step 2: Introduce Slack Variables for Dual Constraints

- From dual feasibility, we have $A^T \lambda^* \leq c$.
- Define the dual slack variables:

$$\mu^* = c - A^T \lambda^* \geq 0.$$

- Then the equation $(x^*)^T (c - A^T \lambda^*) = 0$ becomes:

$$(x^*)^T \mu^* = 0.$$

Step 3: Complementary Slackness

- We have $x^* \geq 0$ and $\mu^* \geq 0$, and $(x^*)^\top \mu^* = 0$.
- For two nonnegative vectors, their dot product is zero if and only if each component product is zero:

$$x_j^* \cdot \mu_j^* = 0 \quad \text{for all } j = 1, \dots, n.$$

- This is **complementary slackness** for the primal variables.
- Interpretation: For each variable x_j , either
 - $x_j^* > 0$ and then $\mu_j^* = 0$, meaning the corresponding dual constraint is active ($a_j^\top \lambda^* = c_j$), or
 - $\mu_j^* > 0$ and then $x_j^* = 0$, meaning the variable is at its lower bound.

Step 4: The Complete KKT Conditions

- Putting everything together, we obtain the **Karush-Kuhn-Tucker (KKT) conditions** for LP:
 1. **Primal feasibility:** $Ax^* = b, x^* \geq 0$.
 2. **Dual feasibility:** $A^\top \lambda^* \leq c$ (or equivalently $\mu^* = c - A^\top \lambda^* \geq 0$).
 3. **Complementary:** $x_j^* \mu_j^* = 0$ for all j .

Necessity: If x^* and λ^* are optimal, then:

- Strong duality gives $c^\top x^* = b^\top \lambda^*$.
- From the derivation in Step 3, this implies $(x^*)^\top (c - A^\top \lambda^*) = 0$.
- With $x^* \geq 0$ and $c - A^\top \lambda^* \geq 0$, we get $x_j^* (c_j - a_j^\top \lambda^*) = 0$ for all j .
- Defining $\mu^* = c - A^\top \lambda^*$ gives the complementary slackness condition.

Step 4: The Complete KKT Conditions

- Putting everything together, we obtain the **Karush-Kuhn-Tucker (KKT) conditions** for LP:
 1. **Primal feasibility:** $Ax^* = b, x^* \geq 0$.
 2. **Dual feasibility:** $A^\top \lambda^* \leq c$ (or equivalently $\mu^* = c - A^\top \lambda^* \geq 0$).
 3. **Complementary:** $x_j^* \mu_j^* = 0$ for all j .

Sufficiency: If (x^*, λ^*, μ^*) satisfy these conditions, then:

- x^* is primal feasible and λ^* is dual feasible.
- From complementary: $(x^*)^\top \mu^* = 0$; that is, $(x^*)^\top (c - A^\top \lambda^*) = 0$.
- Rearranging: $c^\top x^* = (x^*)^\top A^\top \lambda^* = (Ax^*)^\top \lambda^* = b^\top \lambda^*$.
- Thus the duality gap is zero, so x^* and λ^* are optimal.

Summary: KKT and Duality

- The KKT conditions for LPs are simply:
 - Primal feasibility
 - Dual feasibility
 - Complementary
- They follow directly from strong duality and nonnegativity.
- They are necessary and sufficient for optimality.
- Duality also provides a "status report": unboundedness or infeasibility of one problem tells us about the other.
- These insights are fundamental for both theory and algorithms.

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- **Algorithms for Linear Programming**
- Extensions of Linear Programming
- Summary

How Do We Solve LPs in Practice?

- Two main families of algorithms:
 1. **Simplex method** (1947, Dantzig): walks along the edges of the feasible polyhedron from vertex to vertex.
 2. **Interior-point methods** (1984, Karmarkar): moves through the interior of the feasible set, approaching the optimum along a smooth path.
- Both are widely used in software. Simplex is often efficient for small to medium problems; interior-point methods excel for large-scale LPs.

In this course, we'll look at the core ideas conceptually.

Details are covered in advanced courses on optimization.

Simplex Method: Overview

- The simplex method solves LPs by moving from one vertex to a better adjacent vertex.
- It uses the algebraic description of vertices in standard form.
- At each step:
 1. Identify current vertex (basic feasible solution).
 2. Check if current vertex is optimal.
 3. If not, choose an edge to move along that improves the objective.
 4. Move along that edge until a new vertex is reached.
 5. Repeat.
- We'll see how this works using our production example.

Step 1: Basic Feasible Solutions

- In standard form $\min c^\top x$ s.t. $Ax = b$, $x \geq 0$, with A $m \times n$, $\text{rank } m$.
- At a vertex, exactly n constraints are active: the m equalities $Ax = b$ and $n - m$ nonnegativity constraints $x_j = 0$.
- The variables set to zero are **nonbasic**.
- The remaining m variables are **basic**.
- Let B be the matrix of basic columns of A .
- Then $Ax = b$ becomes $Bx_B = b$ and $x_B = B^{-1}b$.
- For simplicity, we assume B is invertible (no dependencies).
- If $x_B \geq 0$, we have a **basic feasible solution (BFS)** – a vertex.

Example: Basic Feasible Solution (BFS)

- Back to our production problem in standard form with

$$x = (x_1, x_2, s_1, s_2):$$

$$A = \begin{pmatrix} 1 & 2 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{pmatrix}, \quad b = (100, 120)^\top.$$

- At the vertex $(0, 0)$, we have $x_1 = 0, x_2 = 0$.
- From $Ax = b$, we find $s_1 = 100, s_2 = 120$.
- Hence, our basic variables are s_1 and s_2 , with basis matrix $B = I$.
- Because we have $x_B = (100, 120) \geq 0$, we have a BFS.

Step 2: Moving Along an Edge

- To move to an adjacent vertex, we increase one nonbasic variable.
- Suppose we increase x_q (nonbasic) by $t \geq 0$. All other nonbasic variables stay zero.
- To keep $Ax = b$, the basic variables must adjust. Write a_q for column q of A . Since $Ax = b$, we find $Bx_B + ta_q = b$.
- Since $Bx_B^0 = b$ at current vertex, we have $B(x_B - x_B^0) + ta_q = 0$, so

$$x_B(t) = x_B^0 - tB^{-1}a_q.$$

- The direction vector is $d = \begin{pmatrix} -B^{-1}a_q \\ e_q \end{pmatrix}$.

Step 3: How the Objective Changes

- Objective along this direction: $c^\top x(t) = c_B^\top (x_B^0 - tB^{-1}a_q) + c_q t$.
- The rate of change (derivative at $t = 0$) is:

$$\Delta_q = c_q - c_B^\top B^{-1}a_q.$$

- This is the **reduced cost** of variable q .
- For minimization:
 - If $\Delta_q < 0$, increasing x_q decreases the objective $\rightarrow$ good candidate!
 - If $\Delta_q > 0$, increasing x_q increases objective $\rightarrow$ not desirable.
 - If $\Delta_q = 0$, moving along this edge doesn't change objective.

Example: Reducing Cost

- At $(0, 0)$, basis $B = I$, $x_B = (100, 120)$, $c_B = (0, 0)$.

- For x_1 : $a_1 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$, $B^{-1}a_1 = a_1$.

$$\Delta_1 = c_1 - c_B^\top a_1 = -3 - 0 = -3.$$

- For x_2 : $a_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$, $\Delta_2 = -5 - 0 = -5$.

- Both negative. We can choose either.
- Usually, we'd pick the most negative (x_2) for faster improvement.
- But let's pick x_1 — just for illustration, no other reason.

Step 4: How Far Can We Go? – Ratio Test

- As we increase x_q , basic variables change: $x_B(t) = x_B^0 - tB^{-1}a_q$.
- We need $x_B(t) \geq 0$. For each basic variable i :

$$(x_B^0)_i - t(B^{-1}a_q)_i \geq 0.$$

- If $(B^{-1}a_q)_i > 0$, this gives $t \leq \frac{(x_B^0)_i}{(B^{-1}a_q)_i}$.
- If $(B^{-1}a_q)_i \leq 0$, the inequality holds for all $t \geq 0$.
- The maximum step we can take is:

$$t_{\max} = \min_{i: (B^{-1}a_q)_i > 0} \frac{(x_B^0)_i}{(B^{-1}a_q)_i}.$$

- The index where the minimum occurs corresponds to the leaving variable – it becomes zero at $t = t_{\max}$.

Example: Ratio Test

- $x_B^0 = (100, 120)$, $B^{-1}a_1 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$.
- For s_1 : $100/1 = 100$.
- For s_2 : $120/3 = 40$.
- Minimum is 40, so $t_{\max} = 40$, leaving variable is s_2 .
- New solution:
 - $x_1 = 40$,
 - $s_1 = 100 - 40 \cdot 1 = 60$,
 - $x_2 = 0$, $s_2 = 0$.
- This is vertex $(40, 0)$ in original variables.

Step 5: The Pivot – Updating the Basis

- After the move, variable x_1 enters the basis, s_2 leaves.
- New basis matrix B now consists of columns for x_1 and s_1 :

$$B = \begin{pmatrix} 1 & 1 \\ 3 & 0 \end{pmatrix}.$$

- New basic variables: $x_1 = 40$, $s_1 = 60$.
- Nonbasic: $x_2 = 0$, $s_2 = 0$.
- We now have a new vertex. The process repeats: compute reduced costs, etc.

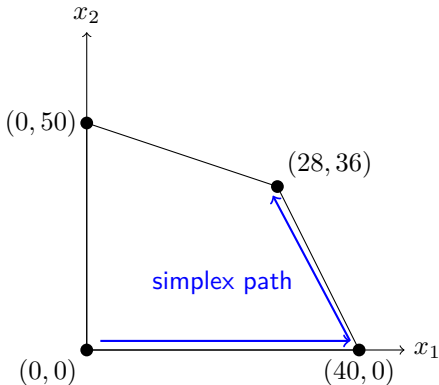
Step 6: Optimality Condition

- At any vertex, if $\Delta_j \geq 0$ (for minimization), then no move improves the objective – we are optimal.
- Reason: any feasible direction is a nonnegative combination of edge directions, and if all reduced costs are nonnegative, moving in any feasible direction cannot decrease the objective.
- For our example, after reaching $(28, 36)$, all reduced costs become nonnegative, confirming optimality.

Simplex Algorithm Summary

1. Start with a basic feasible solution (vertex).
2. Compute reduced costs for all nonbasic variables.
3. If all reduced costs satisfy $\Delta_j \geq 0$, stop – current solution is optimal.
4. Else, choose an entering variable q with $\Delta_q < 0$.
5. Compute $d = B^{-1}a_q$. Perform ratio test to find leaving variable (index p) and step $t_{\max}$.
6. Update solution: $x_q = t_{\max}$, $x_B = x_B - t_{\max}d$.
7. Update basis: replace column p with column q .
8. Go to step 2.

Visualizing the Simplex Path



The simplex method moves along edges, always improving the objective, until it reaches the optimal vertex.

Simplex Method: Practical Considerations

- **What if B is not invertible?**
 - Usually, at a vertex the basic columns are linearly independent, so B is invertible. Otherwise, there is a redundant constraint that can be removed.
 - Nevertheless, in numerical implementations, near-singular matrices can still occur due to rounding errors. Techniques like **pivoting with tolerance** or **refactorization** are used to maintain stability.
 - If linear independence is lost, the algorithm may stall or fail – modern solvers detect and correct this (details are beyond the scope of this course).

Simplex Method: Practical Considerations

- **How do we find an initial BFS?**
 - Many LPs do not have an obvious starting vertex (for example, $x = 0$ may not be feasible).
 - The **two-phase method** is used:
 1. Phase I: Introduce artificial variables to create an obvious BFS, then minimize their sum. If the minimum is zero, we have a feasible basis for the original problem.
 2. Phase II: Use that basis to solve the original LP.
 - If Phase I fails (minimum > 0), the original problem is infeasible.

Simplex Method: Practical Considerations

- **What if more than one variable ties in the ratio test?**
 - This is called **degeneracy** – a basic variable becomes zero, leading to a degenerate vertex.
 - In such cases, the next pivot may not improve the objective, and cycling (repeating the same sequence) can occur.
 - Anti-cycling rules (e.g., Bland's rule: always choose the smallest subscript among eligible variables) guarantee termination.
 - Degeneracy is common in practice but usually does not cause serious problems. Modern solvers handle all of this automatically.

Interior-Point Methods – A Glimpse

- Instead of sticking to vertices, interior-point methods take a different approach:
 - They stay strictly inside the feasible region (hence $x > 0$) and approach the optimum gradually.
 - A logarithmic barrier term is added to the objective to keep away from boundaries:

$$\min_x c^\top x - \tau \sum_{j=1}^n \log x_j \quad \text{s.t.} \quad Ax = b.$$

- For a large parameter τ , the solution is well inside; as $\tau \rightarrow 0$, the solution converges to an optimal vertex.
- The path traced by these solutions is called the **central path**.
- Newton's method is used to follow this path efficiently.

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- **Extensions of Linear Programming**
- Summary

When Linear Isn't Enough

- Linear programming is powerful, but many real-world problems require more:
 - **Diminishing returns:** doubling production might not double profit – the relationship is curved.
 - **Risk management:** in finance, we care about both expected return AND variance.
 - **Discrete decisions:** we can't build half a factory or dispatch 2.7 trucks.
- Two important extensions address these needs:
 1. **Quadratic Programming (QP)** – for curved objectives.
 2. **Mixed-Integer Linear Programming (MILP)** – for discrete decisions.
- Both build on LP ideas and are widely used in practice.

Quadratic Programming – What Is It?

- A Quadratic Program (QP) has a quadratic objective and linear constraints:

$$\min_x \frac{1}{2}x^\top Hx + g^\top x \quad \text{s.t.} \quad Gx \geq b.$$

- The matrix H captures interactions between variables. If H is positive definite, the objective is a convex bowl – then the problem is "easy" (similar difficulty to LP).
- If $H = 0$, we recover a linear program.
- The $x^\top Hx$ term allows us to model:
 - Quadratic costs (e.g., energy, penalties).
 - Variance (risk) in portfolio optimization.
 - Distance from a target.

Quadratic Programming – Real-World Examples

- **Portfolio optimization (Markowitz model):**

- Variables: how much to invest in each asset.
- Objective: maximize expected return minus a penalty for risk (variance).
- The variance term is quadratic: $x^T \Sigma x$, where Σ is the covariance matrix.

- **Support Vector Machines (SVMs):**

- Find the hyperplane that best separates two classes of data.
- The objective involves maximizing the margin, which leads to a quadratic program.

- **Model Predictive Control:**

- In robotics or chemical processes, we want to follow a trajectory.
- Quadratic penalties on deviations and control effort give a QP.
- Many LP algorithms extend to QP – active-set and interior-point methods – and are available in software like CVX, Gurobi, MOSEK.

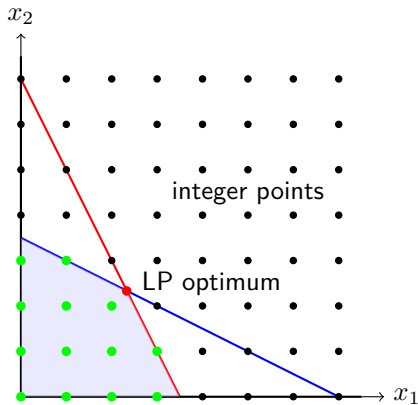
Mixed-Integer Linear Programming

- In many real problems, decisions aren't continuous:
 - How many trucks to dispatch? (must be integer)
 - Should we build a factory in location A or B? (yes/no)
 - Which warehouse should serve which customer? (assignment)
- A Mixed-Integer Linear Program (MILP) is an LP with some variables restricted to integers:

$$\min c^\top x \quad \text{s.t.} \quad Ax = b, x \geq 0, x_j \in \mathbb{Z} \quad \text{for some } j.$$

- "Mixed" means some variables are continuous, some are integer.
- This seemingly small change makes the problem much harder – MILP is NP-hard in general.

Why MILP Is Hard – A Visualization



- LP: feasible region is a continuous polygon (shaded area).
- MILP: feasible points are only the integer lattice points (green).

How MILP Is Solved in Practice

- Modern MILP solvers combine several techniques:
 - **Branch and bound:**
 1. Solve the LP relaxation (ignore integrality).
 2. If the solution has fractional integers, pick one and create two subproblems: one with $x_j \leq \lfloor x_j \rfloor$, one with $x_j \geq \lceil x_j \rceil$.
 3. Recursively explore until all integer variables are integer.
 - **Cutting planes:** Add constraints that cut off fractional solutions without removing any integer feasible points.
 - **Heuristics:** Quickly find good integer solutions to prune the search tree.
- State-of-the-art solvers (Gurobi, CPLEX, SCIP) combine these and can solve problems with millions of variables.

MILP – Real-World Examples

- **Airline crew scheduling:**

- Assign pilots and flight attendants to flights.
- Each crew member works a sequence of flights (a pairing).
- Decision: which pairings to select? Integer variables for each pairing.

- **Supply chain network design:**

- Where to build warehouses? Binary decisions.
- How much to ship from each warehouse to each customer? Continuous decisions.

- **Production planning:**

- Which products to make in each period? Integer batch sizes.
- Inventory levels are continuous.

- **Scheduling:** Timetabling, job shop scheduling, sports league scheduling – all involve integer decisions.

Summary: LP Extensions

- **Quadratic Programming (QP):**
 - Adds quadratic terms to the objective.
 - Convex QPs are "easy" – similar complexity to LPs.
 - Used in finance, machine learning, control.
- **Mixed-Integer Linear Programming (MILP):**
 - Adds integrality constraints on some variables.
 - Much harder (NP-hard) but solvers have made enormous progress.
 - Used in scheduling, logistics, design.
- Both build on LP concepts: duality, KKT, active sets, and algorithms extend to these more general problems.
- In practice, you model your problem and let sophisticated solvers do the work – understanding the theory helps you formulate better models.

Contents

- What is an Optimization Problem?
- Linear Programming Formulation
- Geometry of Linear Programs
- Duality in Linear Programming
- KKT Optimality Conditions
- Algorithms for Linear Programming
- Extensions of Linear Programming
- **Summary**

What We Learned: LP Basics and Geometry

- **Linear Programming (LP):**

- Minimize or maximize a linear function subject to linear constraints.
- Standard form: $\min c^T x$ s.t. $Ax = b, x \geq 0$.
- All LPs can be transformed to this form (slack variables, etc.).

- **Geometry of LPs:**

- Each linear inequality defines a halfspace; the feasible set is a **polyhedron** (intersection of halfspaces).
- A bounded polyhedron is called a **polytope**.
- The Fundamental Theorem of LP: if an optimal solution exists, there is one at a **vertex** (extreme point).

What We Learned: KKT Conditions

- **Karush-Kuhn-Tucker (KKT) conditions for LP:**
 1. **Primal feasibility:** $Ax = b, x \geq 0$.
 2. **Dual feasibility:** $A^T \lambda \leq c$ (or $\mu = c - A^T \lambda \geq 0$).
 3. **Complementary:** $x_j \mu_j = 0$ for all j .
- The KKT conditions are **necessary and sufficient** for optimality in Linear Programming problems.

What We Learned: Duality Theory

- **Duality:** Every LP has a twin problem called the **dual**.
 - For primal (P): $\min c^\top x$ s.t. $Ax = b, x \geq 0$.
 - Dual (D): $\max b^\top \lambda$ s.t. $A^\top \lambda \leq c, \lambda$ unrestricted.
- **Weak duality:** For any feasible primal x and dual λ , $c^\top x \geq b^\top \lambda$.
 - Dual values give lower bounds on primal optimum.
- **Strong duality:** If primal has optimal x^* , dual has optimal λ^* with $c^\top x^* = b^\top \lambda^*$.
- Duality also detects infeasibility/unboundedness:
 - Primal unbounded $\Rightarrow$ dual infeasible.
 - Primal infeasible $\Rightarrow$ dual unbounded or infeasible.

What We Learned: Algorithms

- **Simplex method:**

- Moves from vertex to adjacent vertex along edges.
- Uses **reduced costs** $\Delta_q = c_q - c_B^\top B^{-1} a_q$ to select entering variable.
- **Ratio test** determines how far to move and which variables leave.
- Continues until all reduced costs ≥ 0 (optimality).

- **Practical considerations:**

- Phase I finds initial BFS; Phase II solves original LP.
- Degeneracy can cause cycling; anti-cycling rules guarantee termination.

- **Interior-point methods:**

- Follow central path through interior using barrier functions and Newton's method.
- Efficient for large-scale problems.

What We Learned: Extensions

- **Extensions:**
 - **Quadratic Programming (QP):** quadratic objective, linear constraints (portfolio optimization, SVMs).
 - **Mixed-Integer Linear Programming (MILP):** some variables integer (scheduling, logistics). Much harder – solved with branch-and-bound, cutting planes.
- **Key takeaway:** LP is the foundation of optimization. Understanding its geometry, duality, and algorithms prepares you for more advanced models and real-world applications.

Chapter 4

Nonlinear Programming Models

Nonlinear Programming

- Introduction
- Optimality Conditions
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- Summary

Contents

- Introduction
- Optimality Conditions
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- Summary

Nonlinear Optimization Problem

- In Lecture 3 we studied linear programs, where all functions are affine:

$$\min_x c^\top x \quad \text{s.t.} \quad Ax = b, x \geq 0.$$

- However, many real-world problems involve nonlinearities.
 - Quadratic cost (Examples: energy, risk)
 - Nonlinear constraints (Example: geometric design)
 - Complex models (Example: parameter estimation / machine learning)
- This leads to **nonlinear programming** (NLP):

$$\min_x f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, h_j(x) = 0,$$

usually considered over finite index sets, $i \in \mathcal{I}, j \in \mathcal{J}$.

A Guiding Example: Fitting a Nonlinear Model

- Suppose we have measurements (t_i, y_i) and a model

$$y = e^{x_1 t} \cos(x_2 t) + \text{noise}.$$

- We want to estimate the parameters $x = (x_1, x_2)$ by minimizing the sum of squared errors:

$$\min_x F(x), \quad F(x) = \frac{1}{2} \sum_{i=1}^m (y_i - e^{x_1 t_i} \cos(x_2 t_i))^2.$$

- This is a **nonlinear least squares** problem – a special case of NLP.
- The objective is nonlinear; there is no closed-form solution. We need iterative methods.

Contents

- Introduction
- **Optimality Conditions**
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- Summary

Derivatives for Functions of One Variable

- For a function $f : \mathbb{R} \rightarrow \mathbb{R}$, the **first derivative** $f'(x)$ measures slope.
- The **second derivative** $f''(x)$ measures curvature.
- At a local minimizer x^* :
 - Necessary: $f'(x^*) = 0$ (flat tangent).
 - Sufficient: $f'(x^*) = 0$ and $f''(x^*) > 0$ (strictly convex).
- Example: $f(x) = x^2$ has $f'(x) = 2x$, $f''(x) = 2$.
At $x = 0$, $f'(0) = 0$, $f''(0) > 0 \implies$ local minimum.

From Scalar to Multivariate: Gradient

- For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** $\nabla f(x)$ is a vector of partial derivatives:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix}.$$

- It points in the direction of steepest ascent,
its negative points to steepest descent.
- Example: $f(x, y) = x^2 + y^2$ has $\nabla f(x, y) = (2x, 2y)^\top$.
- At a local minimizer, we expect $\nabla f(x^*) = 0$ (stationary point).

The Hessian Matrix

- The **Hessian** $\nabla^2 f(x)$ is an $n \times n$ matrix of second partial derivatives:

$$\nabla^2 f(x)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}(x).$$

- For twice continuously differentiable functions f , $\nabla^2 f$ is symmetric.
- Example: $f(x, y) = x^2 + y^2$ has Hessian

$$\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix},$$

which is positive definite.

- The Hessian captures curvature in multiple directions.

First-Order Necessary Condition

Theorem If x^* is a local minimizer of a continuously differentiable function f , then

$$\nabla f(x^*) = 0.$$

- Points with $\nabla f(x) = 0$ are called **stationary points**.
- They can be local minima, local maxima, or saddle points.
- Example: $f(x, y) = x^2 - y^2$ has $\nabla f = (2x, -2y)$. At $(0, 0)$, gradient is zero, but it's a saddle point, not a minimum.

Second-Order Conditions

Second-Order Necessary Condition

If x^* is a local minimizer and f is twice continuously differentiable, then

$$\nabla^2 f(x^*) \succeq 0 \quad (\text{positive semidefinite}).$$

Second-Order Sufficient Condition

If $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succ 0$ (positive definite), then x^* is a strict local minimizer.

- Positive definiteness means all eigenvalues are > 0 ; it implies the function curves upward in every direction.

Scalar Examples

- $f(x) = x^2$:
 - $\nabla f(x) = 2x$, $\nabla^2 f(x) = 2$.
 - Stationary point at $x = 0$; $\nabla^2 f(0) = 2 > 0 \rightarrow$ strict local minimum.
- $f(x) = -x^2$:
 - $\nabla f(x) = -2x$, $\nabla^2 f(x) = -2$.
 - $x = 0$ is stationary, but Hessian negative definite $\rightarrow$ strict local max.
- $f(x) = x^3$:
 - $\nabla f(x) = 3x^2$, $\nabla^2 f(x) = 6x$.
 - $x = 0$ stationary; $\nabla^2 f(0) = 0$ — in this case a saddle point.

Multivariate Example 1: Quadratic Bowl

$$f(x, y) = x^2 + y^2.$$

- Gradient: $\nabla f = (2x, 2y)$.
- Hessian: $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$ (positive definite everywhere).
- Stationary point: $(0, 0)$.
- Since Hessian is positive definite, $(0, 0)$ is a strict local minimum.

Multivariate Example 2: Saddle Point

$$f(x, y) = x^2 - y^2.$$

- Gradient: $\nabla f = (2x, -2y)$.
- Hessian: $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$ (one positive eigenvalue, one negative).
- Stationary point: $(0, 0)$.
- Because the Hessian is indefinite, $(0, 0)$ is a saddle point – not a min or max.

Multivariate Example 3: Local Maximum

$$f(x, y) = -x^2 - y^2.$$

- Gradient: $\nabla f = (-2x, -2y)$.
- Hessian: $\nabla^2 f = \begin{pmatrix} -2 & 0 \\ 0 & -2 \end{pmatrix}$ (negative definite).
- Stationary point: $(0, 0)$.
- Negative definite Hessian implies $(0, 0)$ is a strict local maximum.

Important Caveat

- The sufficient condition $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succ 0$ guarantees a strict local minimizer.
- However, a local minimizer can exist even if the Hessian is only positive semidefinite.
- Example: $f(x) = x^4$ at $x = 0$:

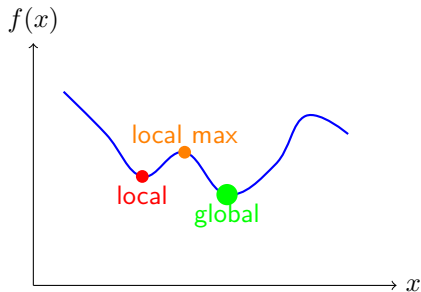
$$f'(0) = 0, \quad f''(0) = 0 \text{ (not positive definite),}$$

but $x = 0$ is still a strict local minimum.

- In such cases, higher-order derivatives determine the nature.

Local versus Global Minimizers

- **Local minimizer:** x^* such that $f(x^*) \leq f(x)$ for all x in a small neighborhood around x^* .
- **Global minimizer:** x^* such that $f(x^*) \leq f(x)$ for all x in the entire domain.



First and second order conditions merely indicate local minimizers.

In general, locating global minimizers is much more difficult.

Lagrange Multipliers in Calculus

- In multivariable calculus, you may have seen problems like:

$$\min_{x,y} f(x,y) \quad \text{s.t.} \quad g(x,y) = 0.$$

- The method of Lagrange multipliers introduces a variable λ and solves:

$$\nabla f(x,y) + \lambda \nabla g(x,y) = 0, \quad g(x,y) = 0.$$

- This is the foundation for general equality-constrained optimization.
- We now generalize to n variables and m constraints.

General Equality-Constrained Problem

- We consider:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad h(x) = 0,$$

where $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$, with $m \leq n$.

- The constraints are nonlinear in general.
- Example: minimize distance from a point to a surface.
- We need conditions that characterize optimal points.

The Lagrangian Function

- Introduce a vector of **Lagrange multipliers** $\lambda \in \mathbb{R}^m$.
- Define the **Lagrangian**:

$$L(x, \lambda) = f(x) + \lambda^\top h(x).$$

- Why this form? At a feasible point, $h(x) = 0$, so $L(x, \lambda) = f(x)$.
- The multipliers allow us to "absorb" the constraints into a single function.
- The gradient of L with respect to x combines the objective gradient and a weighted sum of constraint gradients.

First-Order Necessary Conditions (KKT)

Karush-Kuhn-Tucker (KKT) Conditions If x^* is a local minimizer and the constraints are "regular" at x^* (we'll explain this soon), then there exists $\lambda^* \in \mathbb{R}^m$ such that:

1. **Stationarity:** $\nabla_x L(x^*, \lambda^*) = \nabla f(x^*) + \nabla h(x^*)^\top \lambda^* = 0$.
 2. **Primal feasibility:** $h(x^*) = 0$.
- These are $n + m$ equations in $n + m$ unknowns (x^*, λ^*) .
 - They generalize the Lagrange multiplier method from calculus.
 - The conditions are necessary for optimality (under regularity).

Understanding Stationarity

- Stationarity says: at the optimum, the gradient of f is a linear combination of the gradients of the constraints.

$$\nabla f(x^*) = - \sum_{i=1}^m \lambda_i^* \nabla h_i(x^*).$$

- Geometry: $\nabla f(x^*)$ lies in the span of the constraint normals.
- Moving along any direction tangent to the constraints does not change f to first order.

Example 1: Quadratic Objective, Linear Constraint

- Problem: $\min_x x_1^2 + x_2^2$ subject to $x_1 + x_2 = 1$.
- Lagrangian: $L(x, \lambda) = x_1^2 + x_2^2 + \lambda(x_1 + x_2 - 1)$.
- Stationarity:

$$\frac{\partial L}{\partial x_1} = 2x_1 + \lambda = 0, \quad \frac{\partial L}{\partial x_2} = 2x_2 + \lambda = 0.$$

- Consequently, $x_1 = x_2 = -\lambda/2$.
- Feasibility implies $x_1 + x_2 = 1 \implies \lambda = -1, x_1 = x_2 = 0.5$.
- The point $(0.5, 0.5)$ satisfies KKT. Is it a minimizer?
→ Yes, because the objective is convex.

Example 2: Minimization on a Circle

- Problem: $\min_x (x_1 - 1)^2 + x_2^2$ subject to $x_1^2 + x_2^2 = 1$.
- Lagrangian: $L(x, \lambda) = (x_1 - 1)^2 + x_2^2 + \lambda(x_1^2 + x_2^2 - 1)$.
- Stationarity:

$$\frac{\partial L}{\partial x_1} = 2(x_1 - 1) + 2\lambda x_1 = 0, \quad \frac{\partial L}{\partial x_2} = 2x_2 + 2\lambda x_2 = 0.$$

- The second equation yields $2x_2(1 + \lambda) = 0$
 $\implies$ either $x_2 = 0$ or $\lambda = -1$.

Example 2: Solving the Cases

- **Case 1:** $x_2 = 0$. Then constraint gives $x_1^2 = 1$, so $x_1 = \pm 1$.
 - For $(1, 0)$: from stationarity: $2(1 - 1) + 2\lambda \cdot 1 = 0 \implies \lambda = 0$.
 - For $(-1, 0)$: $2(-1 - 1) + 2\lambda(-1) = 0 \implies -4 - 2\lambda = 0 \implies \lambda = -2$.
- **Case 2:** $\lambda = -1$. The first stationarity equation yields
$$2(x_1 - 1) + 2(-1)x_1 = 0 \implies 2x_1 - 2 - 2x_1 = -2 = 0$$
$$\implies \text{impossible.}$$
- Candidate points: $(1, 0)$ with $\lambda = 0$, and $(-1, 0)$ with $\lambda = -2$.
- Evaluate objective:
 - At $x = (1, 0)$: $f(x) = (1 - 1)^2 + 0^2 = 0$.
 - At $x = (-1, 0)$: $f(x) = (-1 - 1)^2 + 0^2 = 4$.
- Thus $(1, 0)$ is the minimizer; $(-1, 0)$ is actually a maximizer.

When Do the KKT Conditions Hold?

- The KKT conditions are necessary for optimality only under certain regularity conditions.
- First, we assume that f and h are **continuously differentiable**.
- Additionally, we need a **constraint qualification** (CQ) at x^* .
- The most common is the

Linear Independence Constraint Qualification (LICQ):

The gradients $\nabla h_1(x^*), \nabla h_2(x^*), \dots, \nabla h_m(x^*)$ are linearly independent.

- LICQ ensures that the feasible set is "regular" (a manifold) around x^* and that the Lagrange multipliers λ^* are unique.
- If LICQ fails, a minimizer might still exist, but it might not satisfy the KKT conditions (or multipliers might not be unique).

Example 1: Failure of LICQ

- Consider: $\min x_1 + x_2$ subject to $x_1^2 + x_2^2 = 0$.
- The only feasible point is $x^* = (0, 0)$.
- Constraint gradient: $\nabla h(x) = (2x_1, 2x_2)$. At x^* , this is $(0, 0)$ – not linearly independent (it's zero).
- LICQ fails.
- Try to write KKT: $\nabla f + \lambda \nabla h = (1, 1) + \lambda(0, 0) = (1, 1) \neq 0$.
- There is no λ satisfying stationarity, yet $(0, 0)$ is the global minimizer.
- This shows that KKT may not hold when constraints are degenerate.

Example 2: Non-Unique Multipliers

- Consider the problem:

$$\min_{x \in \mathbb{R}^2} x_1^2 + x_2^2 \quad \text{s.t.} \quad \begin{aligned} h_1(x) &= x_1 + x_2 = 0 \\ h_2(x) &= 2x_1 + 2x_2 = 0. \end{aligned}$$

- The unique minimizer is $x^* = (0, 0)$, but LICQ fails:
the gradients $\nabla h_1 = (1, 1)$ and $\nabla h_2 = (2, 2)$ are linearly dependent.
- Lagrangian: $L(x, \lambda_1, \lambda_2) = x_1^2 + x_2^2 + \lambda_1(x_1 + x_2) + \lambda_2(2x_1 + 2x_2)$.
- Stationarity at x^* :

$$\nabla_x L = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} + \lambda_1 \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \lambda_2 \begin{pmatrix} 2 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

At $x^* = (0, 0)$, this reduces to $\lambda_1 + 2\lambda_2 = 0$.

Example 2: Non-Unique Multipliers (continued)

$$\min_{x \in \mathbb{R}^2} x_1^2 + x_2^2 \quad \text{s.t.} \quad \begin{aligned} h_1(x) &= x_1 + x_2 = 0 \\ h_2(x) &= 2x_1 + 2x_2 = 0. \end{aligned}$$

- The equation $\lambda_1 + 2\lambda_2 = 0$ has infinitely many solutions.
- Thus the Lagrange multipliers are **not unique** – any pair satisfying $\lambda_1 = -2\lambda_2$ works.
- However, the point x^* is still a valid minimizer; the KKT conditions hold (for example, with $\lambda_1 = \lambda_2 = 0$), but uniqueness is lost due to constraint dependency.

Second-Order Conditions (Briefly)

- The KKT conditions are necessary but not sufficient – a point satisfying them could be a minimizer, maximizer, or saddle.
- **Second-order sufficient condition:** If x^* satisfies KKT and the Hessian of the Lagrangian projected onto the tangent space of the constraints is positive definite, then x^* is a strict local minimizer.
- Formally: Let Z be a basis for the null space of $\nabla h(x^*)^\top$. Then $Z^\top \nabla_{xx}^2 L(x^*, \lambda^*) Z \succ 0$ guarantees a local minimum.
- This is the nonlinear analogue of checking that the reduced Hessian is positive definite.

Summary: Equality-Constrained Optimality

- For $\min f(x)$ s.t. $h(x) = 0$, we introduce multipliers λ and the Lagrangian $L = f + \lambda^\top h$.
- **KKT conditions** (necessary, under LICQ):

$$\nabla_x L = 0, \quad h(x) = 0.$$

- Geometric meaning: ∇f is a linear combination of constraint gradients.
- LICQ ensures the multipliers exist and are unique.
- Second-order conditions help distinguish minima from maxima.
- These concepts are the foundation for algorithms like SQP.

Problem with Inequality Constraints

- We now consider the general nonlinear program:

$$\begin{array}{ll}\text{minimize} & f(x) \\ & x \in \mathbb{R}^n \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \dots, p \\ & h_j(x) = 0, \quad j = 1, \dots, q.\end{array}$$

- Assumptions:
 - f , g_i , and h_j are continuously differentiable (for first-order conditions).
 - For second-order sufficient conditions, we require twice continuous differentiability.
- This is the most general form we will consider.

Active and Inactive Constraints

- At a feasible point x , an inequality constraint $g_i(x) \leq 0$ is called:
 - **Active** if $g_i(x) = 0$ (it binds at the optimum).
 - **Inactive** if $g_i(x) < 0$ (strictly satisfied).
- Inactive constraints do not locally influence optimality – they can be ignored near x .
- Only active constraints matter for the optimality conditions.
- The set of active indices is $\mathcal{A}(x) = \{i \mid g_i(x) = 0\}$.

KKT Conditions for Inequality Constraints

Karush-Kuhn-Tucker (KKT) Conditions

If x^* is a local minimizer and a suitable constraint qualification holds, then there exist multipliers $\lambda_i \geq 0$ (for inequalities) and $\mu_j \in \mathbb{R}$ (for equalities) such that:

1. **Stationarity:** $\nabla f(x^*) + \sum_{i=1}^p \lambda_i \nabla g_i(x^*) + \sum_{j=1}^q \mu_j \nabla h_j(x^*) = 0.$
2. **Primal feasibility:** $g_i(x^*) \leq 0, h_j(x^*) = 0.$
3. **Dual feasibility:** $\lambda_i \geq 0$ for all $i.$
4. **Complementary:** $\lambda_i g_i(x^*) = 0$ for all $i.$

Constraint Qualifications for Inequalities

- As in the equality case, we need a **constraint qualification** (CQ) for the KKT conditions to be necessary.
- The most common is the **Linear Independence Constraint Qualification (LICQ)**:

The gradients of all active inequality constraints

$\nabla g_i(x^*)$ ($i \in \mathcal{A}(x^*)$) and all equality constraints $\nabla h_j(x^*)$

are linearly independent.

- LICQ ensures that the feasible set has a well-behaved tangent cone and that multipliers are unique.
- Other CQs exist (MFCQ, Slater's condition for convex problems), but LICQ is the easiest to state and verify.

Geometric Interpretation

- At an optimal point x^* , the negative gradient $-\nabla f(x^*)$ must lie in the cone generated by:
 - The gradients of active inequality constraints $\nabla g_i(x^*)$ (pointing outward from the feasible region).
 - The gradients of equality constraints $\nabla h_j(x^*)$ (allowing both directions).
- This is exactly the same geometry we saw in linear programming, but now the gradients can vary with x .
- Complementary ensures that inactive constraints (which do not shape the feasible set locally) contribute nothing.

Example 1: Minimization in a Disk

- Problem: $\min_{x \in \mathbb{R}^2} x_1^2 + x_2^2$ subject to $g(x) = x_1^2 + x_2^2 - 1 \leq 0$.
- Feasible set: the unit disk (including interior and boundary).
- The unique global minimizer is $x^* = (0, 0)$ with $f(x^*) = 0$.
- At x^* , $g(x^*) = -1 < 0$ (inactive), so $\mathcal{A}(x^*) = \emptyset$.
- Stationarity condition: $\nabla f(0) + \lambda \nabla g(0) = (0, 0) + \lambda(0, 0) = (0, 0)$ holds for any λ .
- Complementary slackness requires $\lambda g(0) = \lambda(-1) = 0$, which forces $\lambda = 0$.
- Thus the KKT point is $(0, 0)$ with $\lambda = 0$, satisfying all conditions.

Example 2: Point on the Boundary (Not a Minimizer)

- Consider the same problem, but now examine $x^* = (1, 0)$ on the boundary where $g(1, 0) = 0$ (active).
- Is $(1, 0)$ a local minimizer? No – moving slightly inside the disk reduces the objective.
- Stationarity: $\nabla f(1, 0) + \lambda \nabla g(1, 0) = (2, 0) + \lambda(2, 0) = (2 + 2\lambda, 0)$.
- Setting this to zero gives $2 + 2\lambda = 0$, so $\lambda = -1$.
- Dual feasibility requires $\lambda \geq 0$, which is violated.
- Therefore $(1, 0)$ does not satisfy KKT, consistent with it not being a minimizer.

Example 3: A True Boundary Minimizer

- Problem: $\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 - 1)^2$ subject to $g(x) = x_1^2 + x_2^2 - 1 \leq 0$.
- The minimizer lies on the boundary, $x^* = (1/\sqrt{2}, 1/\sqrt{2})$.
- At x^* , $g(x^*) = 0$ (active).
- Gradients:

$$\nabla f(x^*) = (2(x_1^* - 1), 2(x_2^* - 1)) = (\sqrt{2} - 2, \sqrt{2} - 2),$$

$$\nabla g(x^*) = (2x_1^*, 2x_2^*) = (\sqrt{2}, \sqrt{2}).$$

- Stationarity $\nabla f + \lambda \nabla g = 0$ gives $\sqrt{2} - 2 + \sqrt{2}\lambda = 0$.
- Solving yields $\lambda = \sqrt{2} - 1 \approx 0.414 > 0$, satisfying dual feasibility.
- Hence, all KKT conditions are satisfied at x^* .

Second-Order Sufficient Conditions

- The KKT conditions are necessary but not sufficient – a point satisfying them could be a local minimizer, maximizer, or saddle point.
- **Second-order sufficient condition:** If x^* satisfies KKT and the Hessian of the Lagrangian projected onto the tangent space of the active constraints is positive definite, then x^* is a strict local minimizer.
- Formally, let Z be a basis for the null space of the Jacobian of all active constraints. Then

$$Z^\top \nabla_{xx}^2 L(x^*, \lambda^*, \mu^*) Z \succ 0$$

implies a strict local minimum.

Contents

- Introduction
- Optimality Conditions
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- Summary

Newton's Method for Root Finding

- We want to solve $\nabla f(x) = 0$ (first-order optimality condition).
- Newton's method for nonlinear equations: given current x_k , solve the linearized model

$$\nabla f(x_k) + \nabla^2 f(x_k)(x_{k+1} - x_k) = 0.$$

- If $\nabla^2 f(x_k)$ is invertible,

$$x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

- This is the **Newton step** for optimization.

Quadratic Model Interpretation

- At x_k , build a quadratic approximation of f :

$$m_k(d) = f(x_k) + \nabla f(x_k)^\top d + \frac{1}{2} d^\top \nabla^2 f(x_k) d.$$

- Minimizing $m_k(d)$ over d gives the Newton direction

$$d_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

- If $\nabla^2 f(x_k)$ is positive definite, d_k is a descent direction.

Newton-Type Methods (Hessian Approximation)

- In practice, we often replace $\nabla^2 f(x_k)$ with a symmetric positive definite approximation B_k :

$$x_{k+1} = x_k - B_k^{-1} \nabla f(x_k).$$

Examples:

- Regularized Newton: $B_k = \nabla^2 f(x_k) + \tau I$ with a small $\tau > 0$.
- Gauss-Newton: $B_k = J(x_k)^\top J(x_k)$ for least squares (see below).
- Quasi-Newton methods (see book by Nocedal and Wright for details).

Important: The choice of B_k affects the convergence rate!

General Convergence Analysis

Assumptions

1. f is twice continuously differentiable near x^* , with $\nabla f(x^*) = 0$.
2. The Hessian is locally Lipschitz continuous; that is,
$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq L\|x - y\| \text{ for all } x, y \text{ near } x^*.$$
3. The approximations B_k are uniformly bounded and positive definite, and satisfy $\|B_k - \nabla^2 f(x_k)\| \leq \Delta_k$ for some $\Delta_k \geq 0$.

Remarks

- The Lipschitz condition controls how fast the Hessian can change.
- Δ_k measures how well B_k approximates the true Hessian.

Error Recursion

- Starting from $x_{k+1} = x_k - B_k^{-1} \nabla f(x_k)$, we subtract x^* to get

$$\begin{aligned} x_{k+1} - x^* &= x_k - x^* - B_k^{-1} \nabla f(x_k) \\ &= B_k^{-1} [B_k(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))]. \end{aligned}$$

- Using the integral form of the mean value theorem, we find

$$\nabla f(x_k) - \nabla f(x^*) = \underbrace{\left(\int_0^1 \nabla^2 f(x^* + t(x_k - x^*)) dt \right)}_{=: J_k} (x_k - x^*).$$

- After substituting this expression, we have

$$x_{k+1} - x^* = B_k^{-1} [(B_k - \nabla^2 f(x_k))(x_k - x^*) + (\nabla^2 f(x_k) - J_k)(x_k - x^*)].$$

Bounding the Terms

- Take norms and use $\|B_k^{-1}\| \leq \beta$ (since B_k is bounded and p.d.):

$$\|x_{k+1} - x^*\| \leq \beta [\|B_k - \nabla^2 f(x_k)\| + \|\nabla^2 f(x_k) - J_k\|] \|x_k - x^*\|.$$

- The first term is bounded by Δ_k .
- For the second term, we use Lipschitz continuity to bound

$$\begin{aligned} \|\nabla^2 f(x_k) - J_k\| &\leq \int_0^1 \|\nabla^2 f(x_k) - \nabla^2 f(x^* + t(x_k - x^*))\| dt \\ &\leq L \|x_k - x^*\| \int_0^1 (1 - t) dt = \frac{L}{2} \|x_k - x^*\|. \end{aligned}$$

- Hence, we have

$$\|x_{k+1} - x^*\| \leq \beta \left(\Delta_k + \frac{L}{2} \|x_k - x^*\| \right) \|x_k - x^*\|.$$

Convergence Rates

Theorem (Local Convergence)

Under the above assumptions, with $\|B_k^{-1}\| \leq \beta$, L denoting the Lipschitz constant of the Hessian, and if we have sufficiently small approximation errors $\|B_k - \nabla^2 f(x_k)\| \leq \Delta_k$, there exists a neighborhood of x^* such that in that neighborhood,

$$\|x_{k+1} - x^*\| \leq \beta \Delta_k \|x_k - x^*\| + \frac{\beta L}{2} \|x_k - x^*\|^2.$$

Remark 1:

- If $B_k = \nabla^2 f(x_k)$ (exact Newton), then $\Delta_k = 0$ and we have local quadratic convergence,

$$\|x_{k+1} - x^*\| \leq \frac{\beta L}{2} \|x_k - x^*\|^2.$$

Convergence Rates

Theorem (Local Convergence)

Under the above assumptions, with $\|B_k^{-1}\| \leq \beta$, L denoting the Lipschitz constant of the Hessian, and if we have sufficiently small approximation errors $\|B_k - \nabla^2 f(x_k)\| \leq \Delta_k$, there exists a neighborhood of x^* such that in that neighborhood,

$$\|x_{k+1} - x^*\| \leq \beta \Delta_k \|x_k - x^*\| + \frac{\beta L}{2} \|x_k - x^*\|^2.$$

Remark 2:

- If B_k is a “good” approximation (with Δ_k small), the first term dominates when $\|x_k - x^*\|$ is small, leading to a local convergence rate that approaches $\limsup_{k \rightarrow \infty} \beta \Delta_k$.

Convergence Rates

Theorem (Local Convergence)

Under the above assumptions, with $\|B_k^{-1}\| \leq \beta$, L denoting the Lipschitz constant of the Hessian, and if we have sufficiently small approximation errors $\|B_k - \nabla^2 f(x_k)\| \leq \Delta_k$, there exists a neighborhood of x^* such that in that neighborhood,

$$\|x_{k+1} - x^*\| \leq \beta \Delta_k \|x_k - x^*\| + \frac{\beta L}{2} \|x_k - x^*\|^2.$$

Remark 3:

- If $\Delta_k \rightarrow 0$ as $k \rightarrow \infty$ (satisfied for many quasi-Newton methods), superlinear convergence can be achieved.

Interpretation

- The error bound separates two effects:
 1. The **approximation error** Δ_k : how well B_k matches the true Hessian.
 2. The **nonlinearity** captured by L : curvature changes limit how fast we can reduce error.
- Near a solution, the linear term $\beta \Delta_k \|x_k - x^*\|$ usually dominates—at least if we have $\Delta_k > 0$.
- To achieve fast convergence, we need B_k to be a sufficiently accurate approximation of $\nabla^2 f(x_k)$.
- Quasi-Newton methods (BFGS) attempt to achieve $\Delta_k \rightarrow 0$ as $k \rightarrow \infty$, which leads to a superlinear convergence.

Why Globalization?

- Newton steps are derived from a local quadratic model; they may not decrease f when far from a solution.
- Even with a good search direction d_k , we need to choose a step length α_k that guarantees progress.
- **Line search** methods compute a step $x_{k+1} = x_k + \alpha_k d_k$ with $\alpha_k > 0$ chosen to satisfy a **sufficient decrease condition**.
- This ensures global convergence to a stationary point under mild assumptions.

The Armijo Condition (Sufficient Decrease)

- Let d_k be a descent direction at x_k , i.e., $\nabla f(x_k)^\top d_k < 0$.
- The **Armijo condition** requires that for a fixed constant $c \in (0, 1)$ (typically $c = 10^{-4}$):

$$f(x_k + \alpha d_k) \leq f(x_k) + c \alpha \nabla f(x_k)^\top d_k.$$

- This guarantees that the decrease in f is at least a fraction c of the decrease predicted by the linear model.
- The right-hand side is a line through $(0, f(x_k))$ with slope $c \nabla f(x_k)^\top d_k$.

Backtracking Line Search

- In practice, we start with $\alpha = 1$ (the full Newton step) and repeatedly reduce it (e.g., $\alpha \leftarrow \rho\alpha$ with $\rho \in (0, 1)$) until the Armijo condition holds.
- This simple procedure always terminates because for sufficiently small α , the first-order Taylor expansion guarantees the condition (since f is differentiable).
- The resulting step satisfies:

$$f(x_{k+1}) \leq f(x_k) + c \alpha_k \nabla f(x_k)^\top d_k.$$

Global Convergence Theorem

Assumptions

1. f is continuously differentiable and bounded.
2. The search directions d_k are gradient-related: there exist positive constants c_1, c_2 such that for all k ,

$$\nabla f(x_k)^\top d_k \leq -c_1 \|\nabla f(x_k)\|^2, \quad \|d_k\| \leq c_2 \|\nabla f(x_k)\|.$$

3. The step lengths α_k are chosen by backtracking Armijo.
 - The first condition ensures we stay in a compact region where the gradient is well-behaved.
 - The second condition is satisfied by many choices: steepest descent ($d_k = -\nabla f(x_k)$), Newton-type steps with bounded B_k^{-1} , etc.

Proof Sketch of Global Convergence

- Assume, for contradiction, that $\|\nabla f(x_k)\|$ does not converge to zero.
- Then there exists $\epsilon > 0$ and subsequence $\{x_{k_j}\}$ with $\|\nabla f(x_{k_j})\| \geq \epsilon$.
- From the gradient-related condition, $\nabla f(x_{k_j})^\top d_{k_j} \leq -c_1 \epsilon^2$.
- The Armijo condition gives:

$$f(x_{k_j+1}) \leq f(x_{k_j}) + c \alpha_{k_j} \nabla f(x_{k_j})^\top d_{k_j} \leq f(x_{k_j}) - c \alpha_{k_j} c_1 \epsilon^2.$$

- It can be shown that α_{k_j} is bounded below by a positive constant.
- Hence the decrease in f is at least a fixed positive amount each time we visit this subsequence.
- But f is bounded below, so this cannot happen infinitely often.
Contradiction.
- Therefore $\|\nabla f(x_k)\| \rightarrow 0$.

Remarks on Global Convergence

- The above argument guarantees convergence to a **stationary point** ($\nabla f = 0$), not necessarily a local minimum.
- If f is convex, any stationary point is a global minimizer.
- For nonconvex problems, additional technical conditions are needed to guarantee convergence to a local minimum, but in practice line search methods often work well.
- The gradient-related condition is crucial; it ensures that the search direction always points "sufficiently downhill".
- Newton's method with Hessian modification (for example, after adding a regularization to make B_k positive definite) satisfies this.

Nonlinear Least Squares Problem

- We aim to minimize

$$F(x) = \frac{1}{2} \|r(x)\|^2 = \frac{1}{2} \sum_{i=1}^m r_i(x)^2,$$

where $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a residual vector (typically $m \geq n$).

- This problem arises frequently in data fitting: $r_i(x) = y_i - h(x, t_i)$.
- Gradient: $\nabla F(x) = J(x)^\top r(x)$, where $J(x) \in \mathbb{R}^{m \times n}$ is the Jacobian of r .
- Hessian:

$$\nabla^2 F(x) = J(x)^\top J(x) + \sum_{i=1}^m r_i(x) \nabla^2 r_i(x).$$

- The first term $J^\top J$ is always positive semidefinite; the second term involves second derivatives of r .

Motivation: Small Residual Problems

- If the residuals $r_i(x)$ are small at the solution (i.e., the model fits the data well), the second term in the Hessian is negligible.
- In many applications, we are close to a good fit, so $\|r(x^*)\| \approx 0$.
- This suggests approximating the Hessian by $B_k = J(x_k)^\top J(x_k)$, ignoring the second-derivative term.
- This approximation has several advantages:
 - Only first derivatives are needed (no Hessians of r_i).
 - B_k is automatically positive semidefinite (and often positive definite if J has full rank).
 - The resulting step is the solution of a linear least squares problem.

The Gauss-Newton Step

- At iterate x_k , we solve the linearized least squares problem:

$$d_k = \arg \min_d \frac{1}{2} \|J(x_k)d + r(x_k)\|^2.$$

- This is a linear least squares problem with solution satisfying the normal equations:

$$J(x_k)^\top J(x_k)d_k = -J(x_k)^\top r(x_k).$$

- Note that $\nabla F(x_k) = J(x_k)^\top r(x_k)$, so the update can be written as

$$d_k = -[J(x_k)^\top J(x_k)]^{-1} \nabla F(x_k),$$

provided $J(x_k)$ has full column rank (so $J^\top J$ is invertible).

- This is exactly a Newton-type step with $B_k = J(x_k)^\top J(x_k)$.

Connection to General Newton-Type Analysis

- Recall the general error bound for Newton-type methods:

$$\|x_{k+1} - x^*\| \leq \beta \Delta_k \|x_k - x^*\| + \frac{\beta L}{2} \|x_k - x^*\|^2,$$

where $\Delta_k = \|B_k - \nabla^2 F(x_k)\|$ measures the Hessian approximation error.

- For Gauss-Newton, $B_k = J(x_k)^\top J(x_k)$.
- The approximation error is

$$\Delta_k = \|J(x_k)^\top J(x_k) - \nabla^2 F(x_k)\| = \left\| \sum_{i=1}^m r_i(x_k) \nabla^2 r_i(x_k) \right\|.$$

- This error is proportional to the size of the residuals $r_i(x_k)$.

Convergence of Gauss-Newton

- **Zero-residual case:** If $r(x^*) = 0$, then near x^* , the residuals $r(x_k)$ are $\mathcal{O}(\|x_k - x^*\|)$. Hence $\Delta_k = \mathcal{O}(\|x_k - x^*\|)$.
- Substituting into the error bound:

$$\|x_{k+1} - x^*\| \leq \mathcal{O}(\|x_k - x^*\|^2).$$

- Thus Gauss-Newton achieves **quadratic convergence** when the model fits the data perfectly at the solution.
- **Large-residual case:** If $r(x^*) \neq 0$, then Δ_k is roughly constant near x^* , and the bound gives linear convergence:

$$\|x_{k+1} - x^*\| \leq \beta \|r(x^*)\| \|x_k - x^*\| + \text{higher-order terms}.$$

- The convergence rate degrades as the residuals increase.

Guiding Example: Fitting a Damped Oscillation

- Recall our model: $h(x, t) = e^{x_1 t} \cos(x_2 t)$.
- Given measurements (t_i, y_i) , define residuals

$$r_i(x) = y_i - e^{x_1 t_i} \cos(x_2 t_i), \quad i = 1, \dots, m.$$

- Jacobian entries:

$$\frac{\partial r_i}{\partial x_1} = -t_i e^{x_1 t_i} \cos(x_2 t_i), \quad \frac{\partial r_i}{\partial x_2} = t_i e^{x_1 t_i} \sin(x_2 t_i).$$

- The Gauss-Newton method uses only these first derivatives – no need for second derivatives of r_i .
- At each iteration, we solve a 2×2 linear system (or a linear least squares problem if $m > 2$).

Example: Numerical Illustration

- Suppose we generate synthetic data with true parameters $x^* = (-0.5, 2.0)$ and add small noise.
- Starting from $x_0 = (-0.2, 1.5)$, Gauss-Newton converges in a few iterations.
- If the noise is small (near zero-residual), we observe quadratic convergence – the error squares each iteration.
- If noise is larger, convergence becomes linear, consistent with the theory.
- The method is widely used in practice because it is simple, fast, and requires only first derivatives.

Practical Considerations

- The matrix $J(x_k)^\top J(x_k)$ may be ill-conditioned or singular.
- In such cases, we can use a **Levenberg-Marquardt** regularization:

$$(J^\top J + \tau I)d = -J^\top r,$$

where $\tau > 0$ is adjusted adaptively.

- This ensures the step is a descent direction even when J is rank-deficient.

Summary: Gauss-Newton

- Gauss-Newton is a Newton-type method for nonlinear least squares, using $B_k = J(x_k)^\top J(x_k)$ as Hessian approximation.
- The approximation error $\Delta_k = \|\sum r_i \nabla^2 r_i\|$ is proportional to the residual size.
- Convergence behavior:
 - **Zero residual:** quadratic convergence.
 - **Small residual:** fast linear convergence.
 - **Large residual:** slower linear convergence; may need regularization.
- It is the method of choice for many parameter estimation problems because it requires only first derivatives and is easy to implement.
- Our guiding example (damped oscillation) illustrates the practical application of this method.

Contents

- Introduction
- Optimality Conditions
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- Summary

Penalty Methods

- Convert constrained problem to unconstrained by adding a penalty for constraint violation.
- For equality constraints $h(x) = 0$, define the quadratic penalty function:

$$Q_\rho(x) = f(x) + \frac{\rho}{2} \|h(x)\|^2.$$

- As $\rho \rightarrow \infty$, the minimizer of Q_ρ approaches a constrained minimizer.
- Solve a sequence of unconstrained problems with increasing ρ .
- Disadvantage: ill-conditioning for large ρ .

Barrier Methods for Inequalities

- For inequality constraints $g_i(x) \leq 0$, we can use a barrier function that keeps iterates inside the feasible region.
- Logarithmic barrier:

$$B_\mu(x) = f(x) - \mu \sum_{i=1}^p \log(-g_i(x)),$$

with $\mu > 0$ (barrier parameter).

- As $\mu \rightarrow 0$, the barrier solution approaches the true optimum.
- This is the foundation of interior-point methods.

SQP Idea

- At each iterate (x_k, λ_k) , build a quadratic programming (QP) subproblem that models the original NLP.
- For equality-constrained problem $\min f(x)$ s.t. $h(x) = 0$, the QP is:

$$\min_d \nabla f(x_k)^\top d + \frac{1}{2} d^\top M_k d \quad \text{s.t.} \quad \nabla h(x_k)^\top d + h(x_k) = 0.$$

- M_k approximates the Hessian of the Lagrangian.
- The QP solution gives a search direction d_k and new multiplier estimates λ_{k+1} .

SQP for General Constraints

- For problems with inequalities $g(x) \leq 0$ and equalities $h(x) = 0$, the QP subproblem is:

$$\begin{aligned} \min_d \quad & \nabla f(x_k)^\top d + \frac{1}{2} d^\top M_k d \\ \text{s.t.} \quad & \nabla g_i(x_k)^\top d + g_i(x_k) \leq 0, \quad i = 1, \dots, p \\ & \nabla h_j(x_k)^\top d + h_j(x_k) = 0, \quad j = 1, \dots, q. \end{aligned}$$

- This is a quadratic program (QP) – can be solved by active-set or interior-point methods.
- The multipliers from the QP become the new estimates.

Globalization of SQP

- SQP steps may not decrease a merit function (e.g., $f(x)$ plus penalty for constraints).
- Use a line search on a merit function, such as the ℓ_1 penalty:

$$\Phi_{\rho}(x) = f(x) + \rho \left(\sum_i \max(0, g_i(x)) + \sum_j |h_j(x)| \right).$$

- The direction d_k from the QP is a descent direction for Φ_{ρ} if ρ is large enough.
- Ensure sufficient decrease via Armijo-like condition.

Interior-Point Methods for NLP

- Extend the barrier idea to problems with inequalities.
- Replace $g_i(x) \leq 0$ by a logarithmic barrier term:

$$\min_x f(x) - \mu \sum_{i=1}^p \log(-g_i(x)) \quad \text{s.t.} \quad h(x) = 0.$$

- For each $\mu > 0$, solve this equality-constrained problem using Newton's method (KKT system).
- As $\mu \rightarrow 0$, the barrier path approaches the solution.
- This is the NLP generalization of the interior-point method for LP.

Primal-Dual Interior-Point Methods

- Consider the inequality-constrained problem $\min f(x)$ s.t. $g(x) \leq 0$.
- Introduce slack variables $s \geq 0$ such that $g(x) + s = 0$.
- The KKT conditions become:

$$\nabla f(x) + \nabla g(x)^\top \lambda = 0,$$

$$g(x) + s = 0,$$

$$\lambda_i s_i = 0, \quad \lambda_i \geq 0, \quad s_i \geq 0.$$

- The condition $\lambda_i s_i = 0$ is combinatorial (either $\lambda_i = 0$ or $s_i = 0$).
- IP methods relax this to $\lambda_i s_i = \tau$ with $\tau > 0$, and drive $\tau \rightarrow 0$.

Primal-Dual Interior-Point Methods

- For a fixed τ , we solve the smooth system:

$$F_{\tau}(x, \lambda, s) = \begin{pmatrix} \nabla f(x) + \nabla g(x)^{\top} \lambda \\ g(x) + s \\ \Lambda S e - \tau e \end{pmatrix} = 0,$$

where $\Lambda = \text{diag}(\lambda)$, $S = \text{diag}(s)$, $e = (1, \dots, 1)^{\top}$.

- This is a square system of equations – Newton's method can be applied directly.
- As $\tau \rightarrow 0$, the solution approaches the true KKT point, typically with superlinear convergence.
- This is the foundation of primal-dual interior-point solvers like IPOPT.

Contents

- Introduction
- Optimality Conditions
- Algorithms for Unconstrained Optimization
- Algorithms for Constrained Optimization
- **Summary**

What We Learned in Chapter 4

- **Optimality conditions:** KKT conditions generalize LP case to nonlinear problems.
- **Unconstrained optimization:** Newton's method, quadratic convergence, line search globalization.
- **Nonlinear least squares:** Gauss-Newton method, connection to Newton.
- **Constrained optimization:** Penalty/barrier methods, SQP, interior-point methods.
- These algorithms are implemented in modern solvers and are essential for real-world applications.

Final Remarks

- Nonlinear programming is a rich and active field.
- Understanding the fundamentals allows you to use advanced solvers effectively.
- The guiding example (parameter estimation) showed how these methods come together.
- In practice, you will often use software like MATLAB's 'fmincon', Python's 'scipy.optimize', or specialized solvers like IPOPT, SNOPT.

Chapter 5

Graph and Network Models

Graph and Network Models

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

Contents

- **Why Graphs?**
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

Graphs Are Everywhere

- Many real-world systems can be represented as networks:
 - **Social networks:** people are vertices, friendships are edges.
 - **Transportation networks:** cities are vertices, roads/flights are edges.
 - **Communication networks:** computers are vertices, connections are edges.
 - **Biological networks:** proteins interact, neurons connect.
- Graphs give us a common language to model connectivity and relationships.
- In this chapter, we'll learn the basics and see how they connect to optimization and differential equations.

Connections to Other Chapters

- **Chapter 3 (Linear Programming):**

- The feasible set of an LP is a polyhedron.
- Vertices and edges of this polyhedron form a graph.
- The simplex method walks along edges from vertex to vertex.

- **Chapter 4 (Nonlinear Programming):**

- Sparse matrices (adjacency matrices) appear in large-scale optimization.

- **Chapter 6 (Differential Equations):**

- State transition graphs model dynamical systems (e.g., SIR epidemic model).
- Numerical methods for PDEs use grids that are graphs.

Contents

- Why Graphs?
- **Basic Concepts of Graph Theory**
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

What is a Graph?

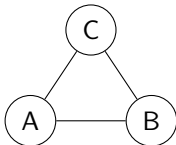
Definition

A **graph** G is an ordered pair (V, E) where:

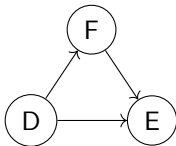
- V is a set of **vertices** (nodes).
- E is a set of **edges**, each connecting two vertices.
- Notation: $n = |V|$, $m = |E|$.
- An edge between u and v is written as uv (unordered).
- If $uv \in E$, u and v are **adjacent** (neighbors).

Types of Graphs

- **Undirected graph:** edges have no direction.



- **Directed graph (digraph):** edges are ordered pairs (u, v) .



- **Weighted graph:** each edge has a numerical value.

For example, these value might model cost, distance, capacity, ...

Degree of a Vertex

- In an undirected graph, $\deg(v)$ = number of edges incident to v .
- In a directed graph, $\deg^\pm(v)$ = number of incoming/outgoing edges

Handshaking Lemma

- For any undirected graph

$$\sum_{v \in V} \deg(v) = 2|E|.$$

- For any directed graph,

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E|.$$

Paths and Cycles

- A **walk** is an alternating sequence of vertices and edges.
- A **trail** is a walk with all edges distinct.
- A **path** is a trail with all vertices distinct.
- A **cycle** is a closed path: a path of length at least 3 that starts and ends at the same vertex, with no other vertex repeated.
- A graph is **connected** if there is a path between any two vertices.
- A **connected component** is a maximal connected subgraph.

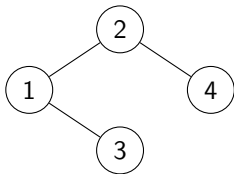
Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- **Trees**
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

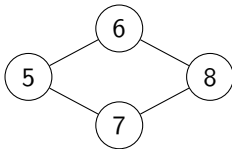
What is a Tree?

- A **tree** is a connected undirected graph with no cycles.
- This simple definition gives rise to many equivalent characterizations.
- For a graph with n vertices, the following are equivalent:
 1. G is a tree (connected and acyclic).
 2. G is connected and has exactly $n - 1$ edges.
 3. G is acyclic and has exactly $n - 1$ edges.
 4. Between any two vertices, there is a unique simple path.
 5. G is minimally connected: removing any edge disconnects it.
 6. G is maximally acyclic: adding any edge creates exactly one cycle.

Examples of Trees



A tree with 4 vertices



Not a tree (cycle 5-6-8-7-5)

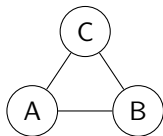
Why Are Trees Important?

- Trees are the simplest connected graphs – they are the "backbone" of many structures.
- They appear in:
 - **Computer science:** file systems, syntax trees, decision trees.
 - **Networks:** minimum cost connection (spanning trees).
 - **Optimization:** branch and bound, dynamic programming on trees.
 - **Biology:** evolutionary trees, phylogenetic trees.
- Many problems that are hard on general graphs become easy on trees (shortest paths are trivial, dynamic programming works well, etc.).

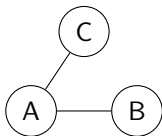
Spanning Trees

- A **spanning tree** of a connected graph G is a subgraph that:
 - Contains all vertices of G .
 - Is a tree (connected and acyclic).
- Every connected graph has at least one spanning tree (usually many).
- Example: A complete graph on 4 vertices has $4^{4-2} = 16$ spanning trees (Cayley's formula).
- Finding a spanning tree is easy: just remove edges until no cycles remain.

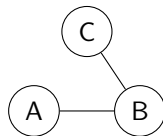
Example: Spanning Trees of a Small Graph



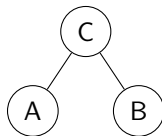
Original



Tree 1



Tree 2



Tree 3

Minimum Spanning Tree (MST)

- In a weighted graph, we often want the spanning tree with smallest total edge weight.
- This is the **minimum spanning tree** (MST).
- Applications:
 - **Network design:** cheapest way to connect cities with fiber optic cable.
 - **Clustering:** find natural groupings in data.
 - **Approximation algorithms:** MST is used as a building block in many advanced algorithms.
- Two classic greedy algorithms: Prim and Kruskal.

Prim's Algorithm – Intuition

- Prim's algorithm grows a tree one vertex at a time.
- Steps:
 1. Start with any vertex as the initial tree.
 2. While the tree does not span all vertices:
 - Consider all edges connecting the tree to a vertex not yet in the tree.
 - Pick the edge with smallest weight.
 - Add that edge and the new vertex to the tree.
- This is like building a tree from a seed, always adding the cheapest possible extension.
- It works because of the **cut property**: the minimum-weight edge crossing any cut belongs to some MST.

The Cut Property

Definition

A **cut** in a graph is a partition of the vertices into two sets: S and $V \setminus S$.

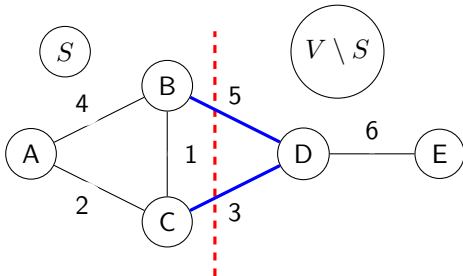
An edge **crosses** the cut if one endpoint is in S and the other in $V \setminus S$.

Cut Property

For any cut, the minimum-weight edge crossing that cut belongs to *some* minimum spanning tree (MST) of the graph.

- This is a key theorem that justifies greedy MST algorithms.
- It says: no matter how you split the vertices, the cheapest connection across that split must be part of at least one optimal tree.

Cut Property – Illustrated

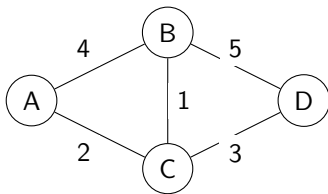


- The cut separates $\{A, B, C\}$ from $\{D, E\}$.
- Crossing edges: B-D (5) and C-D (3).
- The minimum crossing edge is C-D (3). By the cut property, this edge must be in some MST.

How Prim's Algorithm Uses the Cut Property

- Prim's algorithm maintains a growing tree T on a set S of vertices already connected.
- The set S and its complement $V \setminus S$ form a cut.
- All edges from S to $V \setminus S$ are crossing this cut.
- Prim always picks the cheapest such edge.
- By the cut property, this cheapest edge belongs to some MST.
- Therefore, Prim never makes a mistake – it always adds an edge that can be part of an optimal tree.
- This greedy choice leads to a globally optimal MST.

Prim's Algorithm – Example



- Start at A. Edges to neighbors: A-B (4), A-C (2). Pick C (2).
- Tree = A,C. Edges: C-B (1), C-D (3). Cheapest is C-B (1). Add B.
- Tree = A,C,B. New edges: B-D (5) and already have C-D (3). Cheapest is C-D (3). Add D.
- Total weight = $2+1+3 = 6$.

Kruskal's Algorithm – Intuition

- Kruskal's algorithm builds the MST by adding edges in order of increasing weight.
- Steps:
 1. Sort all edges by weight (smallest to largest).
 2. Initialize an empty tree.
 3. For each edge in sorted order:
 - If adding the edge does not create a cycle, add it to the tree.
- This is a "global" approach: always take the cheapest edge that doesn't close a cycle.
- It works because of the **cycle property**: the heaviest edge in any cycle is not in any MST.

Kruskal's Algorithm – Example

Same graph (see the visualization above!):

- Sorted edges: C-B (1), A-C (2), C-D (3), A-B (4), B-D (5).
- Add C-B (1). Tree: C-B.
- Add A-C (2). Tree: C-B, A-C. No cycle.
- Add C-D (3). Tree: C-B, A-C, C-D. No cycle.
- Next edge A-B (4) would create a cycle (A-C-B-A) – skip.
- Next edge B-D (5) would also create a cycle – skip.
- Total weight = $1+2+3 = 6$ – same as Prim's result.

Note: the MST is unique in this particular graph because all edge weights are distinct.

Comparison: Prim vs. Kruskal

- **Prim:**

- Grows a single tree from a starting vertex.
- Good for dense graphs.
- Uses a priority queue.

- **Kruskal:**

- Processes edges globally in sorted order.
 - Good for sparse graphs.
 - Uses a union-find data structure to detect cycles.
- Both are greedy algorithms and run in $O(m \log n)$ time with good data structures.
 - They are conceptually simple and widely used.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- **Graph Representations**
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

Adjacency Matrix

- Label vertices $1, \dots, n$. The **adjacency matrix** A has entries:

$$A_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are connected,} \\ 0 & \text{otherwise.} \end{cases}$$

- For undirected graphs, A is symmetric.
- For weighted graphs, A_{ij} stores the weight of edge (i, j) .

Adjacency Matrix – Counting Walks

- Powers of the adjacency matrix A count walks of given length.
- In detail, $(A^2)_{ij} = \sum_k A_{ik}A_{kj}$.
- If $A_{ik} = 1$ and $A_{kj} = 1$, there is a walk $i \rightarrow k \rightarrow j$.
- Hence $(A^2)_{ij}$ counts the number of walks of length 2 from i to j .
- In general, $(A^m)_{ij}$ counts walks of length m .

Adjacency Matrix – Checking Connectivity

- A graph with adjacency matrix A is connected if and only if

$$(I + A)^{n-1}$$

has all positive entries (because a graph is connected if and only if a walk of length at most $n - 1$ can reach any vertex).

Adjacency Matrix – Counting Triangles

- A **triangle** in an undirected graph is a set of three vertices all mutually connected.
- Consider a triangle with vertices a, b, c . How many closed walks of length 3 start and end at a ?
 - Walk 1: $a \rightarrow b \rightarrow c \rightarrow a$
 - Walk 2: $a \rightarrow c \rightarrow b \rightarrow a$
- Consequently, each vertex in a triangle contributes exactly 2 closed walks of length 3.
- Hence, each triangle contributes $3 \times 2 = 6$ to the trace of A^3 .
- Therefore:

$$\text{number of triangles} = \frac{1}{6} \text{trace}(A^3).$$

Example: A Single Triangle

Graph: vertices 1,2,3 with all edges present.

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Compute A^2 and A^3 :

$$A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad A^3 = A^2 A = \begin{pmatrix} 2 & 3 & 3 \\ 3 & 2 & 3 \\ 3 & 3 & 2 \end{pmatrix}.$$

Trace of $A^3 = 2 + 2 + 2 = 6$. Hence $\# \text{triangles} = 6/6 = 1$, which matches the graph.

The Laplacian Matrix

- Let D be the diagonal **degree matrix**: $D_{ii} = \deg(v_i)$.
- The **Laplacian matrix** is defined as $L = D - A$.
- Example: For the triangle graph:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad L = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}.$$

- Properties:
 - L is symmetric and positive semidefinite.
 - Rows sum to zero: $L\mathbf{1} = 0$ (Recall: Handshaking Lemma).

Counting Spanning Trees

Kirchhoff's Matrix-Tree Theorem

For a connected graph G , the number of spanning trees is given by any cofactor of the Laplacian matrix L .

- A **cofactor** is the determinant of a matrix obtained by deleting one row and one column.
- For the triangle graph, delete row 1 and column 1:

$$L' = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}, \quad \det(L') = (2)(2) - (-1)(-1) = 4 - 1 = 3.$$

- The triangle has exactly 3 spanning trees. Works!

Why Is This Useful?

- **Network reliability:** The number of spanning trees measures how well-connected a network is. More spanning trees means more alternative routes if edges fail.
- **Molecular chemistry:** The number of spanning trees in molecular graphs relates to physical properties.
- **Electrical networks:** The Laplacian appears in equations for current flow (Kirchhoff's current law).
- **Image segmentation:** The second-smallest eigenvalue of L (the Fiedler value) is used to partition graphs into clusters.

Incidence Matrix

- The **incidence matrix** B has rows for vertices, columns for edges.
- For undirected graphs: $B_{v,e} = 1$ if vertex v is incident to edge e (each column has two 1's).
- For directed graphs: $B_{v,e} = -1$ if v is tail, $+1$ if head, 0 otherwise.
- **Why it matters:** In network flow problems (Chapter 3), flow conservation constraints are $Bf = 0$ (for nodes other than source/sink). This connects graph theory directly to LP.

Sparse Matrices

- Most real-world graphs are **sparse**: $m \ll n^2$.
- Adjacency and incidence matrices are sparse – storing only nonzeros saves memory and computation.
- Sparse matrix formats (e.g., compressed sparse row) are essential in numerical computing for large-scale optimization, PDE solvers, and machine learning.
- The sparsity pattern of a matrix can be visualized as a graph:
 $A_{ij} \neq 0$ means a connection between variables i and j . This idea is used in preconditioning and domain decomposition.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- **Markov Chains**
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

What is a Markov Chain?

- A **Markov chain** is a random process that moves through a set of states over time.
- **Key property (Markov property):** The next state depends only on the current state, not on the past.
- Example: Weather can be Sunny or Rainy. Tomorrow's weather depends only on today's weather, not on yesterday's.
- We can model this as a directed graph:
 - Vertices = states.
 - Directed edges = possible transitions, labeled with probabilities.

Transition Matrix

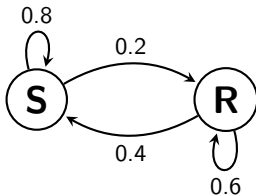
- Suppose we have n states $1, 2, \dots, n$.
- Let P_{ij} = probability of moving from state i to state j in one step.
- The **transition matrix** P is an $n \times n$ matrix with entries P_{ij} .
- Properties:
 - $P_{ij} \geq 0$ (probabilities are nonnegative).
 - $\sum_{j=1}^n P_{ij} = 1$ for each row i (must go somewhere).
- This matrix is exactly the adjacency matrix of a weighted directed graph, where weights are probabilities.

Example: Simple Weather Model

- States: 1 = Sunny, 2 = Rainy.
- Transition probabilities:
 - From Sunny: tomorrow Sunny with prob 0.8, Rainy with prob 0.2.
 - From Rainy: tomorrow Sunny with prob 0.4, Rainy with prob 0.6.
- Transition matrix:

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix}.$$

- Directed graph:



Multi-Step Probabilities

- Let $\pi^{(0)}$ be the initial probability distribution over states (a row vector).
- After one step, the distribution is $\pi^{(1)} = \pi^{(0)}P$.
- After two steps: $\pi^{(2)} = \pi^{(1)}P = \pi^{(0)}P^2$.
- In general, after m steps: $\pi^{(m)} = \pi^{(0)}P^m$.
- The (i, j) entry of P^m is the probability of being in state j after m steps, starting from state i .
- This is exactly the interpretation of powers of the adjacency matrix: they sum over all walks of length m , weighted by the product of probabilities along each walk.

Example: Two-Step Probabilities

- For the weather model, compute P^2 :

$$P^2 = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix} \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix} = \begin{pmatrix} 0.72 & 0.28 \\ 0.56 & 0.44 \end{pmatrix}.$$

- Interpretation:
 - Starting Sunny, probability Sunny after 2 days = 0.72.
 - Starting Sunny, probability Rainy after 2 days = 0.28.
 - Starting Rainy, probability Sunny after 2 days = 0.56.
 - Starting Rainy, probability Rainy after 2 days = 0.44.
- These are sums over all possible 2-day paths (Sunny→Sunny→Sunny, Sunny→Rainy→Sunny, etc.).

Stationary Distribution

- For many Markov chains, as m grows, the distribution $\pi^{(m)}$ converges to a fixed distribution π^* independent of the starting state.
- This is called the **stationary distribution**.
- It satisfies $\pi^* = \pi^* P$.
- Interpretation: long-run fraction of time spent in each state.
- Example: For our weather model, solving $\pi P = \pi$ with $\pi_1 + \pi_2 = 1$ gives $\pi^* = (2/3, 1/3)$.
- Over the long run, it's Sunny $2/3$ of the time, Rainy $1/3$.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- **Dynamic Programming on Graphs**
- Advanced Topics (Brief Overview)
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

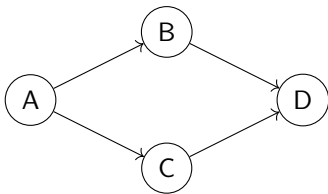
What is Dynamic Programming?

- **Dynamic programming (DP)** is an algorithmic technique for solving optimization problems by:
 1. Breaking the problem into smaller subproblems.
 2. Solving each subproblem once and storing the result.
 3. Combining subproblem solutions to solve the original problem.
- On graphs, DP is often used to find shortest paths, longest paths, or other optimal routes.
- The key is to find a **order** in which to solve subproblems so that when we solve a problem, all its dependencies are already solved.

Why DAGs are Special

- A **Directed Acyclic Graph (DAG)** has no directed cycles.
- This means we can order the vertices so that all edges go from earlier to later vertices.
- Such an ordering is called a **topological order**.
- Example: In a DAG representing course prerequisites, a topological order is a valid sequence to take courses.

Topological Order – Illustration



- A topological order: A, B, C, D or A, C, B, D (both work).
- All edges go forward in the order.
- In a DAG, such an ordering always exists.

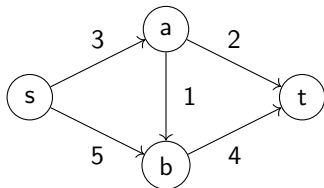
Shortest Paths in a DAG

- Let s be the source vertex and $d[v]$ the shortest distance from s to v .
- Initialize $d[s] = 0$, $d[v] = \infty$ for all other v .
- Process vertices in topological order. For each vertex u , examine all outgoing edges (u, v) :

if $d[v] > d[u] + w(u, v)$ then $d[v] = d[u] + w(u, v)$.

- This works because when we process u , all paths to u have already been considered (since all incoming edges come from earlier vertices).
- Complexity: $O(n + m)$ – very efficient!

DAG Shortest Path – Example



- Topological order: s, a, b, t (assuming edges as shown); set $d[s] = 0$.
- Process s: update $d[a] = 3$, $d[b] = 5$.
- Process a: update $d[b] = \min(5, 3 + 1) = 4$, $d[t] = 3 + 2 = 5$.
- Process b: update $d[t] = \min(5, 4 + 4) = 5$.
- Process t: no outgoing edges. Done: shortest path length = 5.

Why General Graphs Are Harder

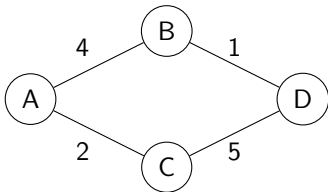
- In a DAG, we process vertices in topological order using DP.
- This works because when we get to v , all its predecessors u have already been processed – their distances are final.
- In a general graph with cycles, this fails:
 - A vertex may have incoming edges from vertices that haven't been processed yet.
 - Distances depend on each other in a circular way.
 - There is no natural order to process vertices.
- We need an algorithm that discovers the correct order *dynamically* as it computes distances.
- For graphs with nonnegative weights, **Dijkstra's algorithm** does exactly this.

Dijkstra's Algorithm – Intuition

- Maintain a set S of vertices whose shortest distance from s is already finalized.
- For each vertex v , maintain a tentative distance $d[v]$ (best known so far).
- Initially $S = \emptyset$, $d[s] = 0$, $d[v] = \infty$ for all $v \neq s$.
- Repeat until S contains all vertices:
 1. Choose the vertex $u \notin S$ with smallest $d[u]$. This distance is now final – add u to S .
 2. For each neighbor v of u :

if $d[v] > d[u] + w(u, v)$ then $d[v] = d[u] + w(u, v)$.

Dijkstra's Algorithm – Example



- Start at A. $d[A] = 0$, $d[B] = 4$, $d[C] = 2$, $d[D] = \infty$.
- Pick C (smallest). Update D: $2 + 5 = 7$ (now $d[D] = 7$).
- Pick B ($d[B] = 4$). Update D: $4 + 1 = 5$ (better, so $d[D] = 5$).
- Pick D ($d[D] = 5$). Done.

Why Dijkstra Works

- The key is the **nonnegative edge weight** assumption.
- When we pick u with smallest $d[u]$ among vertices not in S , any alternative path to u would have to leave S earlier.
- That path would go through some other vertex $x \notin S$ with $d[x] \geq d[u]$, and then to u with nonnegative weights, so its total length is $\geq d[u]$.
- Therefore, $d[u]$ is optimal when we select it.
- This is a form of dynamic programming where subproblems (distances) are solved in increasing order.

What About Negative Weights?

- Dijkstra's algorithm fails when edge weights can be negative – the greedy choice may be wrong.
- Example: A path with a negative edge could become shorter after we thought it was finalized.
- We need an algorithm that works for any weights, even if they are negative.
- The **Bellman-Ford algorithm** does exactly that, at the cost of higher complexity $O(nm)$.

Bellman-Ford – The Idea

- Initialize $d[s] = 0$, $d[v] = \infty$ for all other v .
- Repeat $n - 1$ times (where n is the number of vertices):
 - For every edge (u, v) in the graph:

if $d[v] > d[u] + w(u, v)$ then $d[v] = d[u] + w(u, v)$.

- After $n - 1$ iterations, all shortest paths are correct (if no negative cycles).
- A final check detects negative cycles: if any edge can still be relaxed, a negative cycle exists.
- This is a DP where subproblems are indexed by the number of edges allowed in the path.

DP on Graphs – Summary

- **DAGs:** Process vertices in topological order – simple and fast $O(n + m)$.
- **General graphs (nonnegative weights):** Dijkstra's algorithm – greedy, efficient $O(m \log n)$.
- **General graphs (any weights):** Bellman-Ford – slower $O(nm)$ but handles negatives.
- All these are applications of dynamic programming: they solve larger problems by combining solutions to smaller subproblems.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- **Advanced Topics (Brief Overview)**
- Summary
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

Maximum Flow Problems

- **Problem:** Given a directed graph with edge capacities (e.g., pipeline capacities, data bandwidth), how much flow can be sent from a source node to a sink node?
- **Key concepts:**
 - Each edge has a capacity (maximum flow that can pass through it).
 - Flow must be conserved at intermediate nodes
(what goes in must come out).
 - We want to maximize the total flow from source to sink.
- **Applications:**
 - Transportation networks (goods, traffic)
 - Data routing in computer networks
 - Supply chain logistics

Capacity of a Cut

- Recall: a **cut** (S, T) is a partition of the vertices into two sets. Suppose that S contains the source s , T contains the sink t .
- The **capacity** of a cut is the sum of capacities of all edges going from S to T (direction matters!).
- Intuition: Any flow from s to t must cross from S to T at some point. Therefore:

$$\text{Flow value} \leq \text{Capacity of any cut.}$$

- The cut capacity is an upper bound on the maximum possible flow.

The Max-Flow Min-Cut Theorem

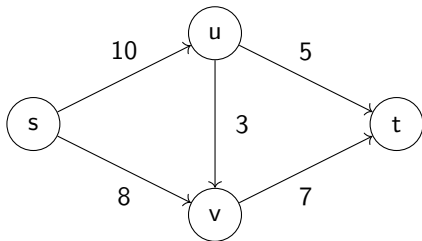
Fundamental Result

For any network, the maximum flow value equals the minimum cut capacity:

$$\max \text{ flow} = \min \text{ cut capacity.}$$

- This means: the biggest bottleneck (the smallest cut) determines exactly how much flow can be sent.
- The theorem tells us that finding the maximum flow is equivalent to finding the minimum cut.

A Simple Example Network

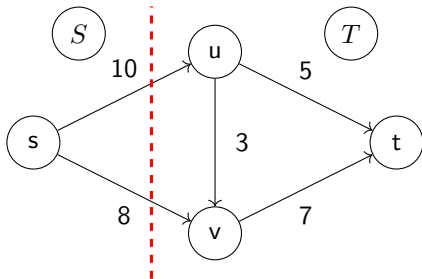


- Source s , sink t . Edge capacities are shown.
- Let's try to find the maximum flow intuitively.

Finding a Feasible Flow

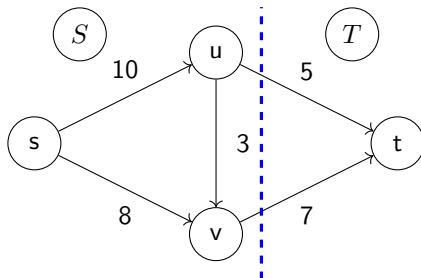
- One possible flow: send 5 units on $s \rightarrow u \rightarrow t$ (uses $u \rightarrow t$ fully).
- Send 7 units on $s \rightarrow v \rightarrow t$ (uses $v \rightarrow t$ fully).
- Total so far: $5 + 7 = 12$ units.
- Can we send more? Edge $s \rightarrow u$ still has capacity 5 left (since it was 10, we used 5).
- Edge $u \rightarrow v$ (capacity 3) could carry additional flow from u to v , then to t via $v \rightarrow t$? But $v \rightarrow t$ is already at capacity 7.
- So maximum flow might be 12. Can we check with a cut?

Identifying a Cut



- Consider the cut $S = \{s\}$, $T = \{u, v, t\}$.
- Edges crossing from S to T : $s \rightarrow u$ (10) and $s \rightarrow v$ (8).
- Cut capacity = $10 + 8 = 18$. This cut says $\text{flow} \leq 18$ – not very tight.

A Tighter Cut



- Cut $S = \{s, u, v\}$, $T = \{t\}$.
- Crossing edges: $u \rightarrow t$ (5) and $v \rightarrow t$ (7).
- Cut capacity = $5 + 7 = 12$.
- This cut says: no flow can exceed 12. We can achieve this bound!
- By the max-flow min-cut theorem, the maximum flow is exactly 12.

Maximum Flow as a Linear Program

- The maximum flow problem can be written as an LP:

$$\begin{array}{ll}\text{maximize} & \sum_{j:(s,j) \in E} x_{sj} \quad (\text{flow out of source}) \\ \text{subject to} & \sum_{j:(i,j) \in E} x_{ij} - \sum_{j:(j,i) \in E} x_{ji} = 0 \quad \forall i \in V \setminus \{s, t\} \\ & 0 \leq x_{ij} \leq c_{ij} \quad \forall (i, j) \in E\end{array}$$

- Variables x_{ij} = flow on edge (i, j) .
- First constraint: flow conservation at intermediate nodes.
- Second constraint: capacity limits and nonnegativity.
- This is a linear program – we could solve it with the simplex method!

The Dual of the Maximum Flow LP

- If we derive the dual of this LP, we get a fascinating problem.
- Each constraint in the primal becomes a variable in the dual.
- The dual variables turn out to correspond to a **cut** in the graph.
- The dual objective is to minimize the capacity of a cut.
- Strong duality tells us:

Maximum flow value = Minimum cut capacity.

- This is exactly the **max-flow min-cut theorem** – a direct consequence of strong duality for LP!

What the Dual Variables Represent

- For each vertex i (except source and sink), the flow conservation constraint gives a dual variable y_i (unrestricted).
- For each edge capacity constraint $x_{ij} \leq c_{ij}$, we get a dual $z_{ij} \geq 0$.
- After some manipulation, the dual becomes:

$$\min \sum_{(i,j) \in E} c_{ij} \cdot \max(0, y_j - y_i)$$

with $y_s = 1$, $y_t = 0$, and $0 \leq y_i \leq 1$.

- The variables y_i define a cut: vertices with $y_i = 1$ are on the source side, $y_i = 0$ on the sink side.
- The expression $\max(0, y_j - y_i)$ is 1 exactly for edges crossing from source side to sink side.
- Thus the dual minimizes the capacity of a cut – beautiful!

LP versus Specialized Algorithms

- **Theoretically:** Yes, we could solve maximum flow using any LP solver (simplex, interior-point).
- **Practically:** There exist a variety of specialized algorithms like Ford-Fulkerson and Edmonds-Karp, which exploit structure and are often faster than general-purpose LP solvers. However, this is an evolving field of research.

My personal guess: in a few years, structure detection in modern LP solvers will become so mature that they will be just as fast.

Traveling Salesman Problem (TSP)

- **Problem:** Given a set of cities and distances between them, find the shortest possible tour that visits each city exactly once and returns to the starting city.
- This is one of the most famous problems in optimization.
- **Why it's hard:** For n cities, there are $(n - 1)!/2$ possible tours. For $n = 20$, that's about 10^{16} possibilities – impossible to check all.
- TSP is **NP-hard**: no known algorithm can solve all instances efficiently (polynomial time) – this is a fundamental open problem in computer science.

TSP – Applications

- Despite its difficulty, TSP and its variants appear everywhere:
 - **Logistics:** delivery route planning, school bus routing
 - **Manufacturing:** drilling holes in circuit boards (minimize movement)
 - **DNA sequencing:** reconstructing DNA fragments
 - **Astronomy:** scheduling telescope observations
 - **Transportation:** airline crew scheduling, vehicle routing
- For practical problems, we use:
 - **Heuristics:** fast algorithms that find good (but not necessarily optimal) solutions (e.g., nearest neighbor, 2-opt, genetic algorithms).
 - **Exact methods:** branch and bound, cutting planes, dynamic programming (for small n). These can solve instances with thousands of cities using advanced solvers.

TSP – Simple Heuristics

- **Nearest neighbor:** Start at a city, repeatedly go to the nearest unvisited city, then return to start.
 - Simple and fast, but can be far from optimal.
- **2-opt:** Repeatedly swap two edges to remove crossings and shorten the tour.
 - A local improvement method that often works well in practice.
- **Christofides algorithm:** For metric TSP (distances satisfy triangle inequality), this guarantees a solution at most 1.5 times the optimal length.
- These heuristics are used in real-world routing software (e.g., Google Maps, delivery route optimization).

What These Problems Teach Us

- **Maximum Flow:** Some graph problems can be solved efficiently with clever algorithms. The max-flow min-cut theorem shows deep structure.
- **TSP:** Some problems are inherently hard – we may need to accept good-enough solutions rather than optimal ones.
- Both illustrate the power and limitations of graph algorithms:
 - Efficient algorithms exist for many practical problems (shortest paths, flows, matchings).
 - But some problems (like TSP) push the boundaries of what we can solve exactly.
- In practice, we choose the right tool for the problem: exact algorithm if possible, heuristic if needed.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- **Summary**
- Appendix: Proof of Kirchhoff's Matrix-Tree Theorem

What We Learned

- **Basic definitions:** vertices, edges, degree, paths, cycles, connectivity.
- **Graph representations:** adjacency and incidence matrices; sparsity.
- **Trees:** properties, spanning trees, MST (conceptually).
- **Dynamic programming on graphs:**
 - DAGs: topological order.
 - Trees: recursion.
 - General graphs (nonnegative weights): Dijkstra.
 - General graphs (negative weights): Bellman-Ford.
- **Advanced topics:** max flow, TSP – just a glimpse.

Contents

- Why Graphs?
- Basic Concepts of Graph Theory
- Trees
- Graph Representations
- Markov Chains
- Dynamic Programming on Graphs
- Advanced Topics (Brief Overview)
- Summary
- **Appendix: Proof of Kirchhoff's Matrix-Tree Theorem**

What We Want to Prove

Kirchhoff's Matrix-Tree Theorem

For a connected undirected graph G with n vertices, let L be its Laplacian matrix. Then the number of spanning trees of G is equal to any cofactor of L (the determinant of L with one row and one column removed).

- We will prove this using the oriented incidence matrix and the Cauchy-Binet formula.
- The proof is elegant and shows the deep connection between linear algebra and graph theory.

Step 1: Oriented Incidence Matrix

- Let G have n vertices and m edges. Assign an arbitrary orientation to each edge (the theorem does not depend on the choice).
- Define the **oriented incidence matrix** $B \in \mathbb{R}^{n \times m}$:

$$B_{v,e} = \begin{cases} +1 & \text{if vertex } v \text{ is the tail of edge } e, \\ -1 & \text{if vertex } v \text{ is the head of edge } e, \\ 0 & \text{otherwise.} \end{cases}$$

- Each column of B contains exactly one $+1$ and one -1 (the two endpoints of the edge).

Step 2: Laplacian as $BB^\top$

- The Laplacian matrix L of G is $L = D - A$, where D is the degree matrix and A the adjacency matrix.
- A key factorization: $L = BB^\top$.
- Why? For a vertex i , $(BB^\top)_{ii}$ is the dot product of the i -th row of B with itself. That row has a $+1$ for each edge incident to i (if i is tail) and a -1 for each edge incident to i (if i is head). The squared entries sum to the degree of i .
- For $i \neq j$, $(BB^\top)_{ij}$ is the dot product of rows i and j . They have a nonzero entry only if they share an edge; then one row has $+1$ and the other -1 , giving product -1 . So $BB^\top$ has -1 for adjacent vertices, 0 otherwise. This matches $L = D - A$.

Step 3: Reduced Laplacian

- Remove the last row of B to obtain $\tilde{B} \in \mathbb{R}^{(n-1) \times m}$. This corresponds to deleting the last vertex.
- Then the reduced Laplacian L' (obtained by deleting the last row and column of L) satisfies:

$$L' = \tilde{B}\tilde{B}^\top.$$

- This holds because $L = BB^\top$, and deleting the last row and column is the same as taking the product of $\tilde{B}$ with its transpose.

Step 4: Determinant via Cauchy–Binet

- The Cauchy–Binet formula tells us that for an $(n - 1) \times m$ matrix $\tilde{B}$ with $m \geq n - 1$,

$$\det(L') = \det(\tilde{B}\tilde{B}^\top) = \sum_S \det(\tilde{B}_S)^2,$$

where the sum runs over all subsets S of $\{1, \dots, m\}$ of size $n - 1$, and $\tilde{B}_S$ is the $(n - 1) \times (n - 1)$ submatrix formed by taking the columns indexed by S .

- Geometrically, we are summing the squares of the determinants of all possible $(n - 1)$ -edge selections.

Step 5: Interpretation of a Subset S

- Each subset S of $n - 1$ edges corresponds to a subgraph H of G with n vertices and $n - 1$ edges.
- What are the possibilities?
 - H could be a spanning tree (connected, no cycles).
 - H could contain a cycle (then it has at least n edges? Wait, $n - 1$ edges cannot have a cycle? Actually a connected graph with $n - 1$ edges is a tree, but a disconnected graph with $n - 1$ edges could have cycles if it has multiple components? For example, two separate triangles would have more than $n - 1$ edges. With exactly $n - 1$ edges, if the graph is connected it's a tree; if it's disconnected, each component is a tree, so no cycles overall. But the issue is that a set of $n - 1$ edges can be disconnected and still have no cycles. However, the determinant $\det(\tilde{B}_S)$ will be zero for any S that does not form a spanning tree. Let's see why.

Step 6: When Is $\det(\tilde{B}_S) \neq 0$?

- The columns of $\tilde{B}_S$ correspond to the chosen edges. The rows correspond to vertices $1, \dots, n-1$ (all but the last vertex).
- If the subgraph H is a spanning tree, then it connects all n vertices, and there is a known fact: the reduced incidence matrix of a spanning tree is square and invertible, with determinant ± 1 .
- If H is not a spanning tree, then either:
 - It is disconnected. Then the rows and columns can be permuted to block form, making the determinant zero.
 - It has a cycle. Then the columns are linearly dependent (as the incidence matrix of a cycle has a linear dependency), so determinant zero.
- Thus $\det(\tilde{B}_S) \neq 0$ exactly when H is a spanning tree, and in that case $\det(\tilde{B}_S) = \pm 1$.

Step 7: Finishing the Proof

- Substituting into the Cauchy-Binet sum:

$$\det(L') = \sum_{S \text{ such that } H \text{ is a spanning tree}} (\pm 1)^2 = \sum_{\text{spanning trees}} 1.$$

- Each spanning tree contributes exactly 1 to the sum.
- Therefore, $\det(L')$ equals the number of spanning trees of G .
- This completes the proof.

Example: Triangle Graph

- Graph: vertices 1,2,3 with edges $e_1 = 12$, $e_2 = 23$, $e_3 = 31$.
- Choose orientation: $e_1 : 1 \rightarrow 2$, $e_2 : 2 \rightarrow 3$, $e_3 : 3 \rightarrow 1$.

- Incidence matrix $B = \begin{pmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix}$.

- Delete last row to get $\tilde{B} = \begin{pmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$.

- Cauchy-Binet sums over the three choices of 2 columns:

$$\det \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}^2 + \det \begin{pmatrix} 1 & -1 \\ -1 & 0 \end{pmatrix}^2 + \det \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}^2 = 3.$$

- Sum = 3, which is indeed the number of spanning trees of a triangle.

Summary

- We proved that the reduced Laplacian's determinant counts spanning trees.
- Key tools: oriented incidence matrix, factorization $L = BB^T$, Cauchy-Binet formula, and the fact that the reduced incidence matrix of a spanning tree has determinant ± 1 .
- This theorem is a beautiful link between linear algebra and combinatorics.

Chapter 6

Ordinary Differential Equation Models

Ordinary Differential Equation Models

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- Existence and Uniqueness Theory
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- Summary

Contents

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- Existence and Uniqueness Theory
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- Summary

What is an Ordinary Differential Equation?

- An **ordinary differential equation (ODE)** relates a function of one variable to its derivatives.
- We focus on **initial value problems (IVPs)**:

$$\dot{x}(t) = f(t, x(t)), \quad x(t_0) = x_0,$$

where:

- $x : [t_0, T] \rightarrow \mathbb{R}^n$ is the unknown function (state).
- $f : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a given function.
- $x_0 \in \mathbb{R}^n$ is the initial state.
- If f does not depend explicitly on t , the system is **autonomous**:
 $\dot{x} = f(x)$.

Integral Formulation

- Integrating the ODE from t_0 to t gives an equivalent integral equation:

$$x(t) = x_0 + \int_{t_0}^t f(s, x(s)) ds.$$

- This form is fundamental for theoretical analysis (existence, uniqueness) and for constructing numerical methods.
- It expresses the solution as a fixed point of an integral operator.

Why Study ODEs?

- ODEs are ubiquitous in science and engineering:
 - Population dynamics (biology)
 - Chemical kinetics (chemistry)
 - Mechanical systems (physics)
 - Finance (some continuous-time models)
- In this chapter we will cover:
 1. Analytic solution methods for special cases.
 2. Existence and uniqueness theory.
 3. Stability analysis (long-term behavior).
 4. Numerical methods for solving ODEs.

Guiding Example: Predator–Prey Model

- The **Lotka–Volterra** model describes two interacting species:

$$\dot{x}_1 = \alpha x_1 - \beta x_1 x_2,$$

$$\dot{x}_2 = -\gamma x_2 + \delta x_1 x_2.$$

- Variables:
 - $x_1(t)$ – population of prey (say, rabbits)
 - $x_2(t)$ – population of predators (say, foxes)
- Parameters:
 - $\alpha > 0$ – prey birth rate
 - $\beta > 0$ – predation rate (prey killed per predator encounter)
 - $\gamma > 0$ – predator death rate
 - $\delta > 0$ – predator growth rate per prey consumed
- This is a system of two coupled nonlinear ODEs.

Contents

- Introduction: Ordinary Differential Equations
- **Explicit Solutions in Special Cases**
- Existence and Uniqueness Theory
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- Summary

Separation of Variables for Scalar ODEs

- Consider a scalar autonomous ODE $\dot{x} = f(x)$.
- If we can write it as $\dot{x} = g(x)$ and $g(x) \neq 0$, we separate variables:

$$\frac{dx}{g(x)} = dt.$$

- Integrate both sides:

$$\int \frac{dx}{g(x)} = \int dt + C.$$

- Then (if possible) solve for $x(t)$.
- Care:** If $g(x_0) = 0$ at the initial point, the solution may be constant (equilibrium). The formula must be handled with caution (division by zero avoided by considering intervals where $g(x) \neq 0$).

Example 1: Exponential Growth/Decay

$$\dot{x} = kx, \quad x(0) = x_0.$$

- For $x \neq 0$, separate: $\frac{dx}{x} = k dt$.
- Integrate: $\ln |x| = kt + C$ yields $x(t) = x_0 e^{kt}$.
- This works even if $x_0 = 0$ (the zero solution is also valid).
- The formula $x(t) = x_0 e^{kt}$ holds for all x_0 (including $x_0 = 0$) - it is the unique solution.

Example 2: Logistic Equation

$$\dot{x} = rx \left(1 - \frac{x}{K}\right), \quad x(0) = x_0 > 0.$$

- Separate (assuming $0 < x < K$):

$$\frac{dx}{x(1 - x/K)} = r dt.$$

- Use partial fractions: $\frac{1}{x} + \frac{1/K}{1 - x/K} = \frac{1}{x} + \frac{1}{K - x}$.
- Integrate: $\ln|x| - \ln|K - x| = rt + C$ yields $\frac{x}{K - x} = Ce^{rt}$.
- Solve: $x(t) = \frac{Kx_0e^{rt}}{K + x_0(e^{rt} - 1)}$.
- As $t \rightarrow \infty$, $x(t) \rightarrow K$ (carrying capacity).

Example 3: Finite Escape Time

$$\dot{x} = x^2, \quad x(0) = 1.$$

- Separate: $\frac{dx}{x^2} = dt$ yields $-\frac{1}{x} = t + C$.
- With $x(0) = 1$, $C = -1$, so $x(t) = \frac{1}{1-t}$.
- Solution exists only for $t < 1$; it blows up as $t \rightarrow 1^-$.
- This shows that even with smooth f , solutions may not exist for all t .

Linear ODEs with Constant Coefficients

- Homogeneous linear system: $\dot{x} = Ax$, $x(0) = x_0$, $A \in \mathbb{R}^{n \times n}$.
- Solution: $x(t) = e^{At}x_0$, where the matrix exponential is defined by

$$e^{At} = \sum_{k=0}^{\infty} \frac{(At)^k}{k!}.$$

- This can, for example, be derived via a Picard iteration.
- For scalar case $A = \lambda$, we recover $x(t) = x_0 e^{\lambda t}$.
- For the predator–prey model (nonlinear), we will need numerical methods.

Contents

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- **Existence and Uniqueness Theory**
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- Summary

Lipschitz Continuity

Definition

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is **Lipschitz continuous** on a set D if there exists a constant $L \geq 0$ such that

$$\|f(x) - f(y)\| \leq L\|x - y\| \quad \forall x, y \in D.$$

- Lipschitz continuity is stronger than ordinary continuity.
- It implies that f cannot oscillate too wildly.
- If f is continuously differentiable on a compact convex set, then it is Lipschitz (by the mean value theorem).
- For the proof of existence/uniqueness, we need a Lipschitz condition in x , uniformly in t .

Gronwall's Lemma

Gronwall's Lemma

Let $u(t)$ be a nonnegative continuous function on $[0, T]$ satisfying

$$u(t) \leq \alpha + \int_0^t \beta(s)u(s) ds,$$

with $\alpha \geq 0$ and $\beta(s) \geq 0$ integrable. Then

$$u(t) \leq \alpha \exp \left(\int_0^t \beta(s) ds \right).$$

In particular, if $\beta(s) = \beta$ constant, $u(t) \leq \alpha e^{\beta t}$.

Proof: Define $v(t) = \alpha + \int_0^t \beta(s)u(s) ds$. Then

$v'(t) = \beta(t)u(t) \leq \beta(t)v(t)$. Hence $\frac{d}{dt} \left(v(t)e^{-\int_0^t \beta(s)ds} \right) \leq 0$, so

$v(t)e^{-\int_0^t \beta(s)ds} \leq v(0) = \alpha$. Since $u(t) \leq v(t)$, we obtain the bound.

Using Gronwall to Prove Uniqueness

- Suppose x and y are two solutions of the IVP with same initial condition and assume that f is continuous in both variables and Lipschitz continuous in x .
- Then $x(t) - y(t) = \int_{t_0}^t [f(s, x(s)) - f(s, y(s))] ds$.
- Taking norms and using Lipschitz:

$$\|x(t) - y(t)\| \leq L \int_{t_0}^t \|x(s) - y(s)\| ds.$$

- Apply Gronwall with $\alpha = 0$, $\beta = L$: $\|x(t) - y(t)\| \leq 0 \cdot e^{Lt} = 0$, hence $x = y$.

Picard–Lindelöf Theorem

Picard–Lindelöf Theorem

Consider the initial value problem

$$\dot{x}(t) = f(t, x(t)), \quad x(t_0) = x_0,$$

where f is continuous in t and uniformly Lipschitz in x on the domain

$$R = [t_0 - a, t_0 + a] \times \overline{B}(x_0, b)$$

for some $a, b > 0$, with Lipschitz constant L , such that

$$\|f(t, x) - f(t, y)\| \leq L\|x - y\| \quad \forall (t, x), (t, y) \in R.$$

Then there exists a unique solution on $[t_0 - \epsilon, t_0 + \epsilon]$ for some $\epsilon > 0$.

Step 1: Integral Formulation

- The IVP is equivalent to the integral equation

$$x(t) = x_0 + \int_{t_0}^t f(s, x(s)) ds.$$

- This is the starting point for Picard iteration.
- Let $M = \sup_{(t,x) \in R} \|f(t, x)\|$ (f is continuous on compact R).
- Choose ϵ such that

$$0 < \epsilon \leq \min \left\{ a, \frac{b}{M} \right\}.$$

- Set $I = [t_0 - \epsilon, t_0 + \epsilon]$ and work on this interval.

Step 2: Picard Iteration

- Define a sequence of functions $\{x_k\}$ on I by

$$x_0(t) \equiv x_0,$$

$$x_{k+1}(t) = x_0 + \int_{t_0}^t f(s, x_k(s)) ds, \quad k \geq 0.$$

- Each x_k is continuous (by induction, using continuity of f and x_k).
- We will show that $\{x_k\}$ converges uniformly to a solution.

Step 3: Bounding the First Difference

- For $t \in I$, we have

$$\|x_1(t) - x_0\| = \left\| \int_{t_0}^t f(s, x_0) ds \right\| \leq M|t - t_0| \leq M\epsilon \leq b.$$

- Hence x_1 stays inside $\overline{B}(x_0, b)$.
- By induction, all x_k stay inside this ball (we'll see this from the estimates).
- Define $\Delta(t) = \max_{s \in [t_0, t]} \|x_1(s) - x_0(s)\|$. From the above,

$$\Delta(t) \leq M|t - t_0|.$$

Step 4: Bounding Successive Differences

- We claim that for all $k \geq 1$ and $t \in I$,

$$\|x_{k+1}(t) - x_k(t)\| \leq \frac{L^k |t - t_0|^k}{k!} \cdot \max_{s \in [t_0, t]} \|x_1(s) - x_0(s)\|.$$

- Proof by induction:

- For $k = 1$, using Lipschitz:

$$\|x_2(t) - x_1(t)\| \leq L \int_{t_0}^t \|x_1(s) - x_0(s)\| ds \leq L|t - t_0| \Delta(t).$$

- Assume true for k , then for $k + 1$:

$$\begin{aligned} \|x_{k+2} - x_{k+1}\| &\leq L \int_{t_0}^t \|x_{k+1}(s) - x_k(s)\| ds \\ &\leq L \int_{t_0}^t \frac{L^k |s - t_0|^k}{k!} \Delta(t) ds \\ &= \frac{L^{k+1}}{k!} \Delta(t) \int_{t_0}^t |s - t_0|^k ds \\ &= \frac{L^{k+1}}{(k+1)!} |t - t_0|^{k+1} \Delta(t). \end{aligned}$$

Step 5: Uniform Cauchy Estimate

- For $n > m$, we have

$$\|x_n(t) - x_m(t)\| \leq \sum_{k=m}^{n-1} \|x_{k+1}(t) - x_k(t)\| \leq \Delta(t) \sum_{k=m}^{n-1} \frac{L^k |t - t_0|^k}{k!}.$$

- Using $\Delta(t) \leq M|t - t_0|$ and $|t - t_0| \leq \epsilon$,

$$\|x_n(t) - x_m(t)\| \leq M\epsilon \sum_{k=m}^{n-1} \frac{(L\epsilon)^k}{k!}.$$

- The remainder of the exponential series $\sum_{k=m}^{\infty} \frac{(L\epsilon)^k}{k!}$ can be made arbitrarily small as $m \rightarrow \infty$ (since it converges).
- Hence $\{x_k\}$ is a Cauchy sequence in the sup norm on I , and converges uniformly to a continuous function x^* .

Step 6: The Limit Satisfies the Integral Equation

- Take the limit as $k \rightarrow \infty$ in the recurrence

$$x_{k+1}(t) = x_0 + \int_{t_0}^t f(s, x_k(s)) ds.$$

- Uniform convergence of x_k to x^* implies $f(s, x_k(s)) \rightarrow f(s, x^*(s))$ uniformly (by continuity of f in x and the fact that $(s, x_k(s))$ stays in the compact set).
- Therefore we can pass the limit inside the integral:

$$x^*(t) = x_0 + \int_{t_0}^t f(s, x^*(s)) ds.$$

- Thus x^* satisfies the integral equation, hence the ODE. And we already know from Gronwall's lemma that the solution is unique.

Summary of the Proof

- We constructed a sequence of approximations (Picard iterates) from the integral form.
- Using Lipschitz continuity, we bounded the differences between successive iterates by terms of an exponential series.
- This showed the sequence is uniformly Cauchy, hence converges to a continuous limit x^* .
- Passing the limit inside the integral showed x^* satisfies the ODE.
- Uniqueness followed from Gronwall's lemma.
- This proof is constructive: the Picard iterates provide an explicit approximation scheme.

Contents

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- Existence and Uniqueness Theory
- **Stability Analysis**
- Numerical Integration: Runge–Kutta Methods
- Summary

What is Stability?

- Consider an ODE $\dot{x} = f(x)$ with a steady state x^* (i.e., $f(x^*) = 0$).
- We want to know what happens to solutions that start near x^* .
- **Stability concepts:**
 - **Lyapunov stable:** solutions starting close stay close for all future time.
 - **Asymptotically stable:** solutions starting close not only stay close but also converge to x^* as $t \rightarrow \infty$.
 - **Unstable:** there exist solutions starting arbitrarily close that move away.
- The simplest way to determine stability is to **linearize** the system around the equilibrium.

The Variational Equation (Sensitivity)

- Consider a solution $x(t)$ and a nearby solution $z(t)$ with $z(0) = x_0 + \delta x_0$.
- Define the perturbation $\delta(t) = z(t) - x(t)$.
- Subtracting the ODEs: $\dot{\delta}(t) = f(z(t)) - f(x(t))$.
- For small δ , Taylor expand:

$$f(z) = f(x) + \frac{\partial f}{\partial x}(x) \delta + o(\|\delta\|).$$

- Thus, to first order, $\dot{\delta} \approx \frac{\partial f}{\partial x}(x(t)) \delta$.
- This linear ODE is the **variational equation**. Its solution gives the sensitivity of the trajectory to initial conditions.
- Gronwall's lemma provides bounds on $\|\delta(t)\|$ in terms of $\|\delta(0)\|$ and the Lipschitz constant.

Linearization Around a Steady State

- If $x(t) \equiv x^*$ is a steady state, the variational equation becomes constant-coefficient:

$$\dot{\delta} = A\delta, \quad A = \frac{\partial f}{\partial x}(x^*).$$

- This is the **linearized system** near x^* .
- The behavior of $\delta(t)$ is governed by the eigenvalues of A .
- Hartman–Grobman theorem (briefly): if none of the eigenvalues have zero real part, the nonlinear system near x^* is topologically equivalent to the linearized one. Thus stability is determined by the linearization.

Stability Theorem for Linear Systems

Theorem

Consider $\dot{z} = Az$.

- If all eigenvalues of A have strictly negative real parts, then $z(t) \rightarrow 0$ for any initial condition. The origin is **asymptotically stable**.
- If any eigenvalue has positive real part, then there exist solutions that grow without bound. The origin is **unstable**.
- If eigenvalues have zero real parts (with others negative), the linearization is inconclusive – nonlinear terms decide stability.

Example 1: A Stable Linear System

$$\dot{x} = -x, \quad \dot{y} = -2y.$$

- Steady state: $(0, 0)$. Jacobian $A = \begin{pmatrix} -1 & 0 \\ 0 & -2 \end{pmatrix}$.
- Eigenvalues: $-1, -2$ (both negative) $\rightarrow$ asymptotically stable.
- Solutions: $x(t) = x_0 e^{-t}$, $y(t) = y_0 e^{-2t}$ – decay to zero.
- This matches the theorem.

Example 2: An Unstable Linear System (Saddle)

$$\dot{x} = x, \quad \dot{y} = -y.$$

- Jacobian $A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$. Eigenvalues: 1 (positive) and -1 (negative).
- The origin is unstable: perturbations along x grow.
- Solutions: $x(t) = x_0 e^t$, $y(t) = y_0 e^{-t}$ – unless $x_0 = 0$, the solution escapes.

Back to Lotka–Volterra: Coexistence Equilibrium

- For the predator–prey model, the coexistence equilibrium is

$$x_1^* = \frac{\gamma}{\delta}, \quad x_2^* = \frac{\alpha}{\beta}.$$

- Jacobian:

$$A = \begin{pmatrix} 0 & -\beta\gamma/\delta \\ \delta\alpha/\beta & 0 \end{pmatrix}.$$

- Eigenvalues: $\lambda = \pm i\sqrt{\alpha\gamma}$ (purely imaginary).
- Linearization predicts a center (closed orbits), but does not determine stability: the nonlinear system may have either periodic orbits (as in Lotka–Volterra) or slowly decaying/growing spirals.
- In fact, $V = \delta x_1 - \gamma \ln x_1 + \beta x_2 - \alpha \ln x_2$ is constant along solutions, so orbits are closed – they are stable but not asymptotically stable.

Summary of Stability Analysis

- The variational equation describes how small perturbations evolve.
- Linearizing around a steady state gives a constant-coefficient system
$$\dot{\delta} = A\delta.$$
- Stability is determined by eigenvalues of A (if none have zero real part).
- Purely imaginary eigenvalues require further investigation (e.g., using Lyapunov functions or first integrals).
- The Lotka–Volterra coexistence equilibrium is an example where linearization is inconclusive; the actual nonlinear behavior is periodic (neutrally stable).

Contents

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- Existence and Uniqueness Theory
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- Summary

Why Numerical Methods?

- Most nonlinear ODEs cannot be solved analytically.
- Even if an analytic solution exists, it may be too complicated to use.
- We need algorithms to approximate $x(t)$ at discrete time points.
- The simplest method is Euler's method:

$$x_{k+1} = x_k + hf(t_k, x_k), \quad t_{k+1} = t_k + h.$$

- Euler is only first-order accurate. Higher-order methods (Runge–Kutta) are more efficient.
- To understand why a method works, we need three key concepts: **consistency**, **stability**, and **convergence**.

Local Truncation Error and Consistency

- Consider a general one-step method $x_{n+1} = x_n + h\Phi(t_n, x_n, h)$.
- The **local truncation error** (LTE) $\tau_n(h)$ is the error made in one step assuming the previous value is exact:

$$\tau_n(h) = \frac{x(t_{n+1}) - [x(t_n) + h\Phi(t_n, x(t_n), h)]}{h}.$$

- A method is **consistent** if $\tau_n(h) \rightarrow 0$ as $h \rightarrow 0$ (uniformly in n).
- The **order** of consistency is p if $\tau_n(h) = \mathcal{O}(h^p)$.

Zero-Stability

- **Zero-stability** (or just stability) ensures that small perturbations in the initial data or round-off errors do not blow up.
- For a one-step method applied to a Lipschitz ODE, zero-stability is automatically satisfied if the method is consistent (a fundamental result).
- More precisely, if Φ is Lipschitz in its arguments, then the method is zero-stable.
- For multi-step methods, stability is more subtle, but for Runge–Kutta methods we are safe.

Convergence Theorem – Statement

Fundamental Theorem of Numerical ODEs

Consider a one-step method $x_{n+1} = x_n + h\Phi(t_n, x_n, h)$ applied to the IVP $\dot{x} = f(t, x)$, $x(t_0) = x_0$. Assume:

1. The method is consistent of order p : local truncation error

$$\|\tau_n(h)\| \leq Ch^p \text{ for all } n.$$

2. The increment function Φ is Lipschitz in x with constant Λ (uniformly in t, h).

Then the method is convergent of order p : there exists $K > 0$ such that for sufficiently small h ,

$$\max_{0 \leq n \leq N} \|x_n - x(t_n)\| \leq Kh^p.$$

Step 1: Define the Global Error

- Let $e_n = x_n - x(t_n)$ be the global error at step n .
- We want to bound $\|e_n\|$ in terms of h and the local truncation error.
- The exact solution satisfies $x(t_{n+1}) = x(t_n) + h\Phi(t_n, x(t_n), h) + h\tau_n$, where τ_n is the local truncation error (vector).
- The numerical solution satisfies $x_{n+1} = x_n + h\Phi(t_n, x_n, h)$.

Step 2: Error Recurrence

- Subtract the two equations:

$$e_{n+1} = e_n + h[\Phi(t_n, x_n, h) - \Phi(t_n, x(t_n), h)] - h\tau_n.$$

- Take norms and use the Lipschitz condition on Φ :

$$\|e_{n+1}\| \leq \|e_n\| + h\Lambda\|e_n\| + h\|\tau_n\|.$$

- Rearranged:

$$\|e_{n+1}\| \leq (1 + h\Lambda)\|e_n\| + h\|\tau_n\|.$$

Step 3: Recursive Bound

- Let $a = 1 + h\Lambda$ and $b_n = h\|\tau_n\|$.
- Then $\|e_{n+1}\| \leq a\|e_n\| + b_n$.
- Unrolling the recursion (induction):

$$\|e_n\| \leq a^n \|e_0\| + \sum_{j=0}^{n-1} a^{n-1-j} b_j.$$

- Since $e_0 = 0$ (initial condition exact), we have

$$\|e_n\| \leq \sum_{j=0}^{n-1} a^{n-1-j} b_j.$$

Step 4: Bounding the Sum

- Note $a^{n-1-j} \leq (1 + h\Lambda)^n \leq e^{\Lambda T}$, where $T = Nh$ (final time).
- Also $b_j = h\|\tau_j\| \leq Ch^{p+1}$ (by consistency of order p).
- Therefore

$$\|e_n\| \leq e^{\Lambda T} \sum_{j=0}^{n-1} Ch^{p+1} = e^{\Lambda T} Cnh^{p+1}.$$

- Since $nh \leq T$, we obtain

$$\|e_n\| \leq CTe^{\Lambda T}h^p.$$

Step 5: Conclusion

- We have shown that for all n ,

$$\|e_n\| \leq \underbrace{CTe^{\Lambda T}}_{=:K} h^p.$$

- Hence $\max_{0 \leq n \leq N} \|x_n - x(t_n)\| \leq Kh^p$.
- This proves convergence of order p .
- The constant K depends on the final time T , the Lipschitz constant Λ , and the consistency constant C , but not on h (for sufficiently small h).

Example: Consistency of Euler's Method

- Euler's method: $x_{n+1} = x_n + hf(t_n, x_n)$.
- Assume exact solution $x(t)$ is twice continuously differentiable.
- Taylor expand $x(t_{n+1})$ around t_n :

$$x(t_{n+1}) = x(t_n) + h\dot{x}(t_n) + \frac{h^2}{2}\ddot{x}(\xi_n).$$

- Since $\dot{x}(t_n) = f(t_n, x(t_n))$, we have

$$x(t_{n+1}) = x(t_n) + hf(t_n, x(t_n)) + \frac{h^2}{2}\ddot{x}(\xi_n).$$

- Therefore the local truncation error is

$$\tau_n(h) = \frac{x(t_{n+1}) - [x(t_n) + hf(t_n, x(t_n))]}{h} = \frac{h}{2}\ddot{x}(\xi_n) = \mathcal{O}(h).$$

- Hence Euler's method is consistent of order 1.

Stability and Convergence of Euler

- Since f is Lipschitz in x , the function $\Phi(t, x, h) = f(t, x)$ is also Lipschitz in x .
- Thus Euler's method is zero-stable.
- By the convergence theorem, the global error is $\mathcal{O}(h)$.
- This means that halving the step size roughly halves the error – not very efficient, which motivates higher-order methods.

General Explicit Runge–Kutta Methods

- An s -stage explicit Runge–Kutta method computes intermediate slopes and combines them:

$$k_1 = f(t_n, x_n),$$

$$k_2 = f(t_n + c_2h, x_n + ha_{21}k_1),$$

$$\vdots$$

$$k_s = f(t_n + c_sh, x_n + h \sum_{j=1}^{s-1} a_{sj}k_j),$$

$$x_{n+1} = x_n + h \sum_{j=1}^s b_jk_j.$$

- The coefficients (a_{ij}, b_j, c_j) are chosen to make the local truncation error as small as possible.

Runge–Kutta 4 (RK4) – The Classic

- The most famous Runge–Kutta method is of order 4:

$$k_1 = f(t_n, x_n),$$

$$k_2 = f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2}, x_n + \frac{h}{2}k_2\right),$$

$$k_4 = f(t_n + h, x_n + hk_3),$$

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

- Its local truncation error is $\mathcal{O}(h^5)$, so it is consistent of order 4.
- It is also zero-stable (as a one-step method), hence convergent of order 4.
- This means that halving h reduces the error by a factor of $2^4 = 16$ – much more efficient than Euler.

Step-Size Control

- To achieve a desired accuracy efficiently, we adapt the step size h during integration.
- Common technique: compute two approximations of different orders (e.g., using a 4th-order and a 5th-order Runge–Kutta pair) and estimate the local error.
- If the estimated error is too large, reduce h ; if it is very small, increase h for efficiency.
- This is implemented in modern solvers like MATLAB's 'ode45', which uses a Runge–Kutta (4,5) pair (the Dormand–Prince method).
- Step-size control makes the method robust and efficient for a wide range of problems.

Contents

- Introduction: Ordinary Differential Equations
- Explicit Solutions in Special Cases
- Existence and Uniqueness Theory
- Stability Analysis
- Numerical Integration: Runge–Kutta Methods
- **Summary**

What We Learned

- **ODEs** model dynamic systems; the integral formulation $x(t) = x_0 + \int f \, ds$ is key.
- **Explicit solutions** exist for some simple ODEs (separable, linear).
- **Lipschitz continuity** guarantees existence and uniqueness (Picard–Lindelöf).
- **Gronwall's lemma** is a powerful tool for error estimates and uniqueness.
- **Stability** is determined by linearization: eigenvalues of the Jacobian decide local behavior (though purely imaginary eigenvalues require further analysis).
- **Numerical methods** (especially RK4) allow us to approximate solutions when analytic formulas are unavailable.